

Secure Single Password Authentication Protocols in Distributed Systems

by

HamidReza Saadi Dadmarzi

A Dissertation Submitted to the
Graduate School of Sciences and Engineering
in Partial Fulfillment of the Requirements for
the Degree of

Master of Science

in

Computer Science and Engineering



KOÇ ÜNİVERSİTESİ

August 3, 2026

**Secure Single Password Authentication Protocols in Distributed
Systems**

Koç University

Graduate School of Sciences and Engineering

This is to certify that I have examined this copy of a master's thesis by

HamidReza Saadi Dadmarzi

and have found that it is complete and satisfactory in all respects,
and that any and all revisions required by the final
examining committee have been made.

Committee Members:

Prof. Alptekin Küpcü (Advisor)

Prof. ErKay Savaş

Assist. Prof. Emre Gürsoy

Date: _____

*To my beloved mother and father,
whose love gave me roots and whose courage taught me to rise.*

ABSTRACT

Secure Single Password Authentication Protocols in Distributed Systems

HamidReza Saadi Dadmarzi

Master of Science in Computer Science and Engineering

August 3, 2026

Passwords remain the predominant mechanism for online authentication because they are familiar, inexpensive to deploy, and easy to use. Conventional systems, however, concentrate password-verification material at login servers and remain vulnerable to password reuse, server compromise, offline dictionary attacks, and insecure credential rotation. These limitations motivate authentication mechanisms that preserve password usability while distributing trust across independent parties.

Single Password Authentication (SPA) allows a user to access multiple independent services using one human-memorable password without revealing the password, or a deterministic function of it, to the login servers. Instead, the password protects high-entropy authentication secrets stored with auxiliary storage providers. During authentication, the user recovers the secret required for the selected service and proves account ownership to its login server. SPA therefore separates the user's remembered credential from the verification information maintained by individual services.

This thesis presents **UpSPA**, an update-enabled threshold SPA framework supporting setup, registration, authentication, login-server-specific secret update, and global password update. Its two-layer state design separates a compact password-protected root record from login-server-specific account records. As a result, the secret of one account can be rotated independently, while changing the remembered password requires updating only the root record rather than every account.

The thesis further introduces **RoUpSPA**, a resilient extension for temporary storage-provider unavailability. It separates the cryptographic threshold needed to recover password-protected state from the quorum needed to complete state-changing operations. Registration and updates can therefore complete with sufficiently many available providers, while stale providers later recover the freshest

committed state. Authenticated encryption, signatures, hidden identifiers, freshness counters, domain binding, and quorum intersection protect against unauthorized creation, replay, rollback, and inconsistent updates.

Security is formalized through real and ideal executions under a static malicious adversarial model. Phase-specific simulators and hybrid arguments reduce security to the threshold oblivious pseudorandom function, authenticated encryption, digital signatures, and collision-resistant hashing. Under the stated corruption and availability assumptions, the constructions prevent offline password testing, cross-service impersonation, and unauthorized updates. Endpoint compromise and denial of service beyond the supported availability bounds remain outside the model.

A proof-of-concept implementation evaluates all protocol phases under different provider counts, thresholds, quorums, and network conditions. It also compares UpSPA with earlier SPA and threshold-authentication schemes and examines quorum-based, public-bulletin-board, and blockchain-backed synchronization alternatives. The results show that local cryptographic costs remain practical, while communication rounds and network latency dominate distributed and resilient deployments. Overall, the thesis demonstrates that single-password authentication can support efficient credential updates and temporary provider failures without exposing the user’s password to login servers or individual storage providers.

ÖZETÇE

Dağıtık Sistemlerde Güvenli Tek Parolalı Kimlik Doğrulama Protokolleri

HamidReza Saadi Dadmarzi

Bilgisayar Bilimleri Ve Mühendisliği, Yüksek Lisans

3 Ağustos 2026

Parolalar; kullanıcılar tarafından iyi bilinmeleri, düşük maliyetle devreye alınabilmeleri ve kolay kullanılabilmesi nedeniyle çevrim içi kimlik doğrulamada baskın mekanizma olmayı sürdürmektedir. Bununla birlikte, geleneksel parola tabanlı sistemler parola doğrulama materyalini giriş sunucularında merkezileştirmekte ve parola tekrar kullanımı, sunucunun ele geçirilmesi, çevrim dışı sözlük saldırıları ve kimlik doğrulama bilgilerinin güvenli olmayan biçimde yenilenmesi gibi tehditlere karşı savunmasız kalmaktadır. Bu sınırlamalar, parolaların kullanılabilirliğini korurken güveni bağımsız taraflar arasında dağıtan kimlik doğrulama mekanizmalarının geliştirilmesini gerekli kılmaktadır.

Tek Parolalı Kimlik Doğrulama (TPKD), kullanıcının insan tarafından hatırlanabilir tek bir parola ile birden fazla bağımsız hizmete erişmesini, bu parolayı veya parolanın deterministik bir fonksiyonunu giriş sunucularına açıklamadan sağlamaktadır. Bunun yerine parola, yardımcı depolama sağlayıcılarında tutulan yüksek entropili kimlik doğrulama gizli değerlerini korumaktadır. Kimlik doğrulama sırasında kullanıcı, seçilen hizmet için gerekli gizli değeri kurtarmakta ve ilgili giriş sunucusuna hesabın sahibi olduğunu kanıtlamaktadır. Böylece SPA, kullanıcının hatırladığı kimlik bilgisini ayrı hizmetler tarafından tutulan doğrulama bilgilerinden ayırmaktadır.

Bu tez, kurulum, kayıt, kimlik doğrulama, giriş sunucusuna özgü gizli değer güncelleme ve genel parola güncelleme aşamalarını destekleyen, güncelleme destekli eşik tabanlı bir SPA çerçevesi olan UpSPA'ı sunmaktadır. İki katmanlı durum tasarımı, kompakt bir parola korumalı kök kaydını giriş sunucularına özgü hesap kayıtlarından ayırmaktadır. Böylece bir hesaba ait gizli değer diğer hesaplardan

bağımsız olarak yenilenebilmekte, hatırlanan parolanın değiştirilmesi ise her hesabın güncellenmesi yerine yalnızca kök kaydının güncellenmesini gerektirmektedir.

Tez ayrıca, depolama sağlayıcılarındaki geçici hizmet kesintilerine karşı dayanıklı bir uzantı olan **RoUpSPA**'yı sunmaktadır. **RoUpSPA**, parola korumalı durumu kurtarmak için gereken kriptografik eşiği, durum değiştiren işlemlerin tamamlanması için gereken yeterli çoğunluktan ayırmaktadır. Böylece kayıt ve güncelleme aşamaları yeterli sayıda erişilebilir sağlayıcıyla tamamlanabilmekte, güncel olmayan sağlayıcılar ise daha sonra kesinleştirilmiş en güncel durumu kurtarabilmektedir. Kimlik doğrulamalı şifreleme, dijital imzalar, gizli tanımlayıcılar, tazelik sayaçları, alan adı bağlama ve yeterli çoğunluk kesişimi; yetkisiz kayıt oluşturmayı, tekrar oynatma saldırılarını, geri alma saldırılarını ve tutarsız güncellemeleri önlemek amacıyla kullanılmaktadır.

Güvenlik, statik kötü niyetli saldırgan modeli altında gerçek ve ideal yürütmeler aracılığıyla biçimsel olarak tanımlanmaktadır. Aşamaya özgü simülasyonlar ve hibrit argümanlar, güvenliği threshold oblivious pseudorandom function'ın, kimlik doğrulamalı şifrelemenin, dijital imzaların ve çakışmaya dayanıklı özetlemenin güvenliğine indirgemektedir. Belirtilen ele geçirme ve erişilebilirlik varsayımları altında yapılar; çevrim dışı parola denemelerini, hizmetler arası kimliğe bürünmeyi ve yetkisiz güncellemeleri önlemektedir. Uç noktaların ele geçirilmesi ve desteklenen erişilebilirlik sınırlarının ötesindeki hizmet engelleme saldırıları modelin kapsamı dışında kalmaktadır.

Bir kavram kanıtlama uygulaması, tüm protokol aşamalarını farklı sağlayıcı sayıları, eşikler, yeterli çoğunluk değerleri ve ağ koşulları altında değerlendirmektedir. Ayrıca **UpSPA**'ı önceki SPA ve eşik tabanlı kimlik doğrulama şemalarıyla karşılaştırmakta; yeterli çoğunluk tabanlı, kamusal ilan panosu tabanlı ve blokzincir destekli eşzamanlama alternatiflerini incelemektedir. Sonuçlar, yerel kriptografik maliyetlerin uygulanabilir düzeyde kaldığını, buna karşılık iletişim turlarının ve ağ gecikmesinin dağıtık ve dayanıklı dağıtımlardaki temel maliyetleri oluşturduğunu göstermektedir. Genel olarak bu tez, tek parolalı kimlik doğrulamanın, kullanıcının parolasını giriş sunucularına veya tekil depolama sağlayıcılarına açığa çıkarmadan verimli kimlik bilgisi güncellemelerini ve geçici sağlayıcı arızalarını destekleyebileceğini göstermektedir.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to my advisor, Prof. Dr. Alptekin Küpçü, for his guidance, patience, and continuous support throughout my graduate studies and the preparation of this thesis. His insightful feedback, encouragement, and high academic standards played an essential role in the development of this work.

I am also grateful for the opportunity to have been a member of the KU-Crypto Research Group at Koç University.

I would also like to thank the members of my thesis jury, Prof. Erkan Savaş, faculty member in Computer Science and Engineering at Sabancı University and Dean of the Faculty of Engineering and Natural Sciences, and Asst. Prof. Emre Gürsoy, faculty member in Computer Science and Engineering and Graduate Program Coordinator at Koç University, for their time, careful evaluation, and valuable comments on this thesis.

I am grateful to Ozan Kaymak for reviewing parts of the thesis, providing helpful comments, and assisting with several project-related tasks. I would also like to thank my former instructor, Dr. Erfan Salavati, Vice President and Quantitative Modeling Lead at JPMorgan, for his guidance and advice on educational and academic matters throughout my studies.

This research was supported by the Scientific and Technological Research Council of Türkiye (TÜBİTAK) under Project No. 123E462, *Tek Parolalı Kimlik Doğrulama Sistemlerinin Geliştirilmesi ve Kullanıcı Deneyiminin İncelenmesi* (Development of Single-Password Authentication Systems and Investigation of User Experience).

TABLE OF CONTENTS

List of Tables	xiii
List of Figures	xv
Abbreviations	xvii
I PRELUDE	1
Chapter 1: Introduction	2
1.1 Thesis Outline and Contributions	2
1.2 Publications	8
Chapter 2: Preliminaries	10
2.1 Notation and Conventions	10
2.2 Standard Definitions and Security Games	15
2.2.1 Basic Cryptographic Language	15
2.2.2 Password-Specific attacks	19
2.2.3 Standard Cryptographic Primitives	23
Chapter 3: Related Work	33
3.1 Single-Password Authentication and Its Threshold Extensions	33
3.2 Password-Protected Secret Sharing and Password-Based Secret Recovery	35
3.3 Threshold Token-Based Single Sign-On	36
3.4 Password-Hardening Services and Updatable Key Management	37
3.5 Quorums, Public Logs, and Resilient State Synchronization	38

3.6	Overall Comparison and Positioning	40
II Updatable Single Password Authentication Variation		42
Chapter 4:	UpSPA: Updatable Single Password Authentication	43
4.1	System Design & Threat Model	43
4.1.1	System Model	44
4.1.2	System Design	47
4.1.3	Threat Model	49
4.1.4	Design Challenges of UpSPA	53
4.2	Construction	54
4.3	Security Analysis	61
4.3.1	Security Model and Proof Organization	61
4.3.2	Simulation Behavior	63
4.3.3	Bad Events Outside the Hybrid Games	74
4.3.4	Hybrid Games, Lemmas, and Proofs	75
4.3.5	Completion of the Theorem	82
Chapter 5:	RoUpSPA: Robust Updatable Single Password Authentication	83
5.1	System Design & Threat Model	83
5.1.1	System Model of RoUpSPA	83
5.1.2	System Design of RoUpSPA	85
5.1.3	Threat Model	86
5.1.4	Design Challenges of RoUpSPA	88
5.2	Construction	89
5.3	Security Analysis	98
5.3.1	Security Model and Notation	98
5.3.2	Main Security Theorem	100
5.3.3	Simulator Overview	101

5.3.4	Bad Events	102
5.3.5	Hybrid Games	103
5.3.6	Synchronization and Counter Consistency	106
5.3.7	Ideal-World Equivalence	109
5.3.8	Final Advantage Bound	109
5.4	Alternative State-Synchronization Mechanisms	110
5.4.1	PBB-based Synchronization	111
5.4.2	Blockchain-based Synchronization	114
5.4.3	Implementation comparison.	115

III Benchmarking and Results 118

Chapter 6: Engineering the SPA Family: From SPA and TSPA to UpSPA 119

6.1	Introduction	119
6.2	Implementation Scope and Design Goals	120
6.3	Shared Software Architecture and Component Decomposition	121
6.4	Cryptographic Primitives and Parameter Choices	122
6.5	Implementing UpSPA and Threshold Variants	123
6.6	Update-Enabled and Resilient Variants	125
6.7	State Management, Record Formats, and Persistence	126
6.8	Service Interfaces, Deployment Model, and Reproducibility	127
6.9	Testing, Benchmarking, and Validation	128
6.9.1	Correctness and Measurement Validation	128
6.9.2	Common Benchmark Environment and Network Profiles	128
6.9.3	UpSPA and TSPA comparison	129
6.9.4	Comparison of UpSPA, PASTA-U and AugSSO	132
6.9.5	Comparison of UpSPA and RoUpSPA	137
6.9.6	Resilient Synchronization Backend Evaluation	139

6.9.7	Receipt-Level EVM Gas Results	147
6.9.8	Foundry Snapshot Cross-Check	149
6.9.9	TRON Implementation and Pending Native Measurements . .	150
6.10	Discussion	150
IV	EPILOG	153
Chapter 7:	Conclusion, Limitations, and Future Work	154
7.1	Conclusion	154
7.2	Limitations	156
7.3	Future Work	157
Appendix A:	PAS-TA-U Baseline and TTG Validation	159
A.1	Purpose and Fairness of the Baseline	159
A.2	Measurement Methodology	160
A.3	Results	162
A.3.1	Isolated TTG Cost	162
A.3.2	Token Generation over Real TCP	163
A.3.3	Password Update over Real TCP	164
A.3.4	OpenSSH Validation	165
A.4	Relative Performance	166
A.5	Fairness of the Main Comparison and Limitations	167
Appendix B:	Latency Diagnostics	168

LIST OF TABLES

2.1	General notation and party notation.	13
2.2	Protocol-state notation.	14
2.3	Cryptographic and proof notation.	15
3.1	Comparison of existing works in terms of security, update support, resilience, recovery, and deployment compatibility.	41
6.1	UpSPA and TSPA deployment-grid median-latency ranges (ms); the last column reports the simplified successful-path stage model.	129
6.2	Local endpoint medians, $q_{sp} = \lceil 0.8n_{sp} \rceil$	144
6.3	Receipt-level EVM gas for deployment and direct RoUpSPA commit- ments.	148
6.4	Receipt-level EVM gas for Merkle publication and proof verification.	148
6.5	Foundry gas for direct commitments.	150
6.6	Foundry gas for Merkle roots.	151
A.1	Isolated TTG costs for the two Rust PAS-TA-U profiles. Values are arithmetic means in milliseconds over three runs of 100 measured executions. The published PAS-TA-U evaluation reports complete token generation rather than isolated TTG operations and therefore has no row in this table.	162
A.2	Complete token-generation latency in milliseconds. The two Rust columns are client-observed real-TCP measurements under Linux <code>tc</code> ; the published column is the arithmetic mean reported for the original Python/OpenSSH implementation. Final application-server verifica- tion is excluded.	163

A.3	Complete password-update latency in milliseconds. Rust values include old- and new-password TOPRF evaluation, server-specific update construction, update-token generation and verification, delivery to all PAS-TA-U servers, record replacement, and acknowledgements.	164
A.4	OpenSSH-agent validation of the paper-style Rust profile, in milliseconds. “Sign service” measures the local agent-side PAS-TA-U threshold signing path; “SSH command” measures a complete fresh <code>ssh . . . true</code> invocation. The published token-generation value is a reference for the signing phase and is not directly comparable with the complete SSH-command column.	165

LIST OF FIGURES

4.1	UpSPA setup phase Π_{Setup}	56
4.2	UpSPA registration phase Π_{Register}	57
4.3	UpSPA authentication phase $\Pi_{\text{Authenticate}}$	58
4.4	UpSPA secret-update phase $\Pi_{\text{SecretUpdate}}$	59
4.5	UpSPA password-update phase $\Pi_{\text{PwdUpdate}}$	60
5.1	The RoUpSPA setup phase Π_{Setup}	90
5.2	The RoUpSPA registration phase Π_{Register}	92
5.3	The RoUpSPA authentication phase $\Pi_{\text{Authenticate}}$	93
5.4	The RoUpSPA secret-update phase $\Pi_{\text{SecretUpdate}}$	94
5.5	The RoUpSPA password-update phase $\Pi_{\text{PwdUpdate}}$	96
6.1	Client registration and authentication, $n_{sp} = 30$	130
6.2	Client setup and updates, $n_{sp} = 30$	131
6.3	LAN registration and authentication, $n_{sp} = 30$	132
6.4	LAN setup and updates, $n_{sp} = 30$	133
6.5	Client registration and authentication, $t_{sp} = 0.6n_{sp}$	134
6.6	Client setup and updates, $t_{sp} = 0.6n_{sp}$	135
6.7	LAN registration and authentication, $t_{sp} = 0.6n_{sp}$	136
6.8	LAN setup and updates, $t_{sp} = 0.6n_{sp}$	137
6.9	Registration latency across benchmark targets	138
6.10	Authentication latency across benchmark targets	139
6.11	Password-update latency across benchmark targets	140
6.12	Registration latency, $t_{sp} = 0.6n_{sp}$	141
6.13	Authentication latency, $t_{sp} = 0.6n_{sp}$	141
6.14	Password-update latency, $t_{sp} = 0.6n_{sp}$	142

6.15	Registration latency: UpSPA and RoUpSPA.	142
6.16	Secret-update latency: UpSPA and RoUpSPA.	143
6.17	Password-update latency: UpSPA and RoUpSPA.	143
6.18	Registration latency, $q_{sp} = \lceil 0.8n_{sp} \rceil$	145
6.19	Secret-update latency, $q_{sp} = \lceil 0.8n_{sp} \rceil$	146
6.20	Password-update latency, $q_{sp} = \lceil 0.8n_{sp} \rceil$	146
6.21	Backend overhead, $q_{sp} = \lceil 0.8n_{sp} \rceil$	147
6.22	EVM gas for direct and Merkle operations.	149
B.1	WAN registration and authentication, $n_{sp} = 10$	168
B.2	WAN registration and authentication, $t_{sp} = 0.6n_{sp}$	169

ABBREVIATIONS

AE	Authenticated Encryption
AES	Advanced Encryption Standard
API	Application Programming Interface
BFT	Byzantine Fault Tolerance
CPU	Central Processing Unit
DB	Database
DE-PAKE	Device-Enhanced Password-Authenticated Key Exchange
DE-PbTA	Device-Enhanced Password-Based Threshold Authentication
DKG	Distributed Key Generation
DPaSE	Distributed Password-Hardened Encryption
EUUF-CMA	Existential Unforgeability under Chosen-Message Attacks
HCR	Hidden Credential Retrieval
IND-CPA	Indistinguishability under Chosen-Plaintext Attacks
INT-CTXT	Ciphertext Integrity
KMS	Key Management System
LAN	Local Area Network
LPbT-SSO	Lattice-Based Password-Based Threshold Single Sign-On
LS	Login Server
MAC	Message Authentication Code
OPRF	Oblivious Pseudorandom Function
OT	Oblivious Transfer
PAKE	Password-Authenticated Key Exchange
PASTA	Password-Based Threshold Authentication

PAS-TA-U	Password-Based Threshold Authentication with Update Support
PBB	Public Bulletin Board
PBFT	Practical Byzantine Fault Tolerance
PHE	Password-Hardened Encryption
PHS	Password-Hardening Service
PIR	Private Information Retrieval
PPT	Probabilistic Polynomial Time
PPSS	Password-Protected Secret Sharing
PRF	Pseudorandom Function
PUOKMS	Password-Protected Updatable Oblivious Key Management System
PQ-PbTA	Post-Quantum Password-Based Threshold Authentication
RAM	Random-Access Memory
RoUpSPA	Robust Updatable Single-Password Authentication
SPA	Single-Password Authentication
SP	Storage Provider
SSO	Single Sign-On
TLS	Transport Layer Security
TOPRF	Threshold Oblivious Pseudorandom Function
TOPPSS	Threshold-OPRF-Based Password-Protected Secret Sharing
TPASS	Threshold Password-Authenticated Secret Sharing
T-PAKE	Threshold Password-Authenticated Key Exchange
TPH-UOKM	Threshold Password-Hardening Updatable Oblivious Key Management
TPHE	Threshold Password-Hardened Encryption
TSPA	Threshold Single-Password Authentication
TSS	Threshold Secret Sharing

TTG	Threshold Token Generation
UOKMS	Updatable Oblivious Key Management System
UpSPA	Updatable Single-Password Authentication
VPS	Virtual Private Server
WAN	Wide Area Network
WASM	WebAssembly

Part I

PRELUDE

Chapter 1

INTRODUCTION

1.1 Thesis Outline and Contributions

This thesis studies how single-password authentication can be extended from a registration-and-authentication mechanism into a complete credential-management framework. The central objective is to allow a user to remember one human-memorable password while keeping the authentication material associated with different login servers independent, updateable, and protected by a distributed set of storage providers. The thesis considers both the cryptographic protection of this state and the operational problems that arise when providers are malicious, temporarily unavailable, or inconsistent.

The work follows a gradual progression. It begins with the original single-password authentication setting and its threshold extension, identifies the missing support for credential updates, and develops **UpSPA** as an update-enabled construction under a full-availability assumption. It then introduces **RoUpSPA**, which retains the cryptographic structure of **UpSPA** but adds a separate quorum-based mechanism for state synchronization and recovery. Finally, the thesis connects the protocol descriptions to concrete software, deployment interfaces, persistent record formats, benchmarking environments, and alternative synchronization backends.

Contributions. The main contributions of this thesis are summarized as follows.

1. **An update-enabled threshold single-password authentication framework.**

The thesis develops **UpSPA** as an extension of the SPA and TSPA line of work. In addition to setup, registration, and authentication, **UpSPA** defines login-server-specific secret update and global password update as explicit protocol

phases. A secret update replaces the high-entropy authentication secret associated with one login server without changing the user's password or the state of unrelated accounts. A password update replaces the password-protected root state while preserving the authentication secrets already registered with the login servers.

This distinction is important because the two operations solve different problems. Secret update is an account-local operation that may be used after suspected exposure or expiration of one login-server-specific secret. Password update is a global operation that changes the user's remembered password without requiring every login-server-specific record to be recreated. The construction therefore provides a credential lifecycle rather than treating password change as an informal re-encryption step.

2. A two-layer state design for efficient and isolated updates.

The thesis introduces a compact password-protected root record containing the high-entropy values needed to derive and manage the remaining protocol state. Login-server-specific authentication secrets are stored separately and are protected using an independent symmetric key recovered from this root record. Consequently, changing the password modifies the compact root layer but does not require re-encrypting the state of every registered login server. Its cost depends on the number of storage providers rather than on the total number of user accounts.

The state design also uses high-entropy provider-side identifiers derived from a secret value rather than directly from public information such as the username and login-server domain. This prevents an outside party from predicting the correct storage locations and creating fraudulent records before the legitimate user registers. Signatures, authenticated encryption, phase-specific message bindings, and freshness counters are used to distinguish authorized updates from arbitrary writes and to separate current records from older but previously valid state.

Together, these mechanisms address several problems that are not handled by ordinary threshold authentication alone: storage-record preemption, unauthorized replacement, cross-context reuse of update messages, replay, and rollback.

3. A security treatment covering the complete protocol lifecycle.

The thesis gives explicit system and threat models for the fully available and resilient constructions. The adversary is malicious and may statically corrupt storage providers, login servers, and users within the stated bounds. In particular, password protection is considered while fewer than t_{sp} storage providers are corrupted and while the target user and target login server remain honest.

Security is expressed through real and ideal executions and is analyzed using phase-specific simulation behavior and hybrid games. The proof structure separates the cryptographic hiding of password-dependent values from the integrity and freshness checks that govern protocol state. It relies on the receiver-side pseudorandomness and receiver-input obliviousness of the TOPRF, the privacy and integrity of authenticated encryption, existential unforgeability of the signature scheme, and collision resistance of the hash function.

Within these assumptions, the analysis addresses resistance to offline dictionary attacks, protection against malicious or honeypot login servers, domain separation across services, and prevention of unauthorized state changes. The thesis also states the boundaries of these guarantees. Direct endpoint compromise, password theft by malware or a keylogger, excessive provider unavailability, and violation of the threshold corruption bound remain outside the cryptographic protection offered by the constructions.

4. A resilient construction separating security thresholds from availability quorums.

The thesis develops RoUpSPA to remove the requirement that every storage provider be available whenever registration or an update modifies persistent state. RoUpSPA distinguishes the cryptographic threshold t_{sp} from the avail-

ability quorum q_{sp} . The first determines how many valid provider contributions are needed for threshold password processing, whereas the second determines when a replicated state-changing operation is considered complete.

The quorum parameters are selected so that successful provider sets have an honest intersection despite the permitted storage-provider corruptions. In particular, the condition $2q_{sp} - n_{sp} > t_{sp} - 1$ ensures that a successful write quorum and a later synchronization or recovery quorum share at least one honest provider. Combined with signatures, counters, record validation, and selection of the freshest valid state, this intersection prevents a completed update from being hidden entirely by corrupted or stale providers.

Authentication remains outside the quorum-finalization path because it is a read-only operation. It proceeds after receiving enough valid threshold responses, while registration and update operations use the stronger completion condition required for persistent state. Providers that miss a completed operation can later recover authenticated ciphertexts and freshness metadata without receiving the user’s password or any decrypted authentication secret.

The thesis additionally studies public bulletin boards and blockchain-backed logs as alternative synchronization mechanisms. These alternatives provide different forms of public history, filtering, and autonomous recovery, but also introduce different availability, flooding, trust, communication, and execution costs. They are therefore treated as optional synchronization layers rather than as requirements of the core RoUpSPA construction.

5. A unified implementation and evaluation of the SPA protocol family.

The thesis presents a proof-of-concept implementation that maps the abstract protocol phases to concrete cryptographic components, service interfaces, persistent records, serialization rules, and deployment configurations. The software architecture separates user-side orchestration, storage-provider processing, login-server interaction, threshold cryptography, state persistence, benchmark recording, and resilient synchronization. This separation makes it

possible to compare related protocols while retaining their different message schedules and trust assumptions.

The evaluation covers client computation, storage-provider work, complete local execution, modeled LAN and WAN environments, threshold scaling, quorum scaling, and communication-round effects. It includes comparisons between UpSPA and TSPA, between UpSPA and the update-enabled threshold SSO constructions PAS-TA-U and AugSSO, and between UpSPA and RoUpSPA. It also evaluates quorum-based synchronization, signed and authorization-enforcing public bulletin boards, and several blockchain deployment profiles.

The results show that local cryptographic work is generally small compared with the cost of sequential network stages. In particular, the additional local work introduced by RoUpSPA is substantially smaller than its WAN overhead, indicating that future optimization should focus primarily on reducing or overlapping communication stages rather than only optimizing signatures or record processing. The comparative experiments also show that no construction or synchronization backend is uniformly best: the preferred design depends on whether the deployment prioritizes local computation, WAN latency, autonomous recovery, write filtering, public auditability, or infrastructure independence.

The implementation chapter also reports receipt-level EVM gas measurements and independent Foundry regression measurements for direct commitments and Merkle-root publication. A TVM-compatible implementation and measurement schema are provided for TRON; however, numerical TRON resource results are not claimed because a complete live and solidified receipt matrix was not available at the time of writing.

Thesis outline. The remainder of the thesis is organized as follows.

Chapter 2 presents the preliminaries required for the protocol and security chapters. It fixes the notation used for users, login servers, storage providers, encrypted

records, threshold parameters, quorum parameters, and protocol phases. It then introduces the cryptographic notions used in the security analysis, including computational indistinguishability, real-ideal security, password-guessing attacks, authenticated encryption, digital signatures, OPRFs, TOPRFs, threshold secret sharing, quorum systems, public bulletin boards, and append-only logs. The chapter concludes with a structured review of SPA and TSPA, password-protected secret sharing, threshold SSO, password-hardening and key-management systems, and resilient distributed storage. This discussion identifies the specific gap addressed in the thesis: existing work provides several forms of threshold password protection or updatable key management, but does not jointly provide the SPA deployment model, login-server-specific secret rotation, password update, and recovery from missed storage-provider updates.

Chapter 3 presents the fully available version, **UpSPA**. It first defines the system architecture, trust boundaries, corruption assumptions, availability requirements, and design challenges. It then describes the setup, registration, authentication, secret-update, and password-update protocols. Special attention is given to the separation of root and account state, hidden storage identifiers, update authorization, domain binding, and freshness. The security section describes the simulator for each protocol phase and gives the hybrid-game argument relating the real execution to the ideal functionality. The chapter ends by identifying the main limitation of this version: although authentication requires only the threshold number of valid providers, state-changing phases depend on the required storage providers being available and therefore do not provide recovery from incomplete or missed writes.

Chapter 4 introduces **RoUpSPA** as the resilient extension of **UpSPA**. The chapter explains the separate roles of t_{sp} and q_{sp} , the quorum-intersection condition, the completion semantics of state-changing phases, and the bounded availability assumptions required for registration and updates. It then presents the resilient protocol phases and the provider-side recovery procedure. The security analysis reuses the cryptographic core of **UpSPA** and adds the arguments needed for signed quorum writes, freshness selection, recovery, and rollback resistance. The chapter also studies public bulletin boards and blockchain-backed logs as alternative state-

synchronization mechanisms and compares their security, liveness, deployment, and performance implications.

Chapter 5 describes the implementation and experimental evaluation. It begins with the implementation scope and design goals and then presents the shared software architecture, cryptographic choices, record formats, persistent state, service interfaces, deployment model, test strategy, and reproducibility requirements. The evaluation compares the relevant SPA, threshold authentication, update-enabled, and resilient constructions under common parameter sets and network profiles. It additionally measures the cost of the alternative synchronization backends and reports the available EVM, Foundry, and TRON implementation status. The final discussion separates cryptographic cost, provider work, communication-round cost, synchronization overhead, and native ledger cost, and explains the trade-offs revealed by the experiments.

Chapter 6 concludes the thesis by bringing together the protocol, security, and implementation findings. It summarizes what is achieved by the fully available and resilient constructions, states the limitations that remain under the current threat and deployment models, and discusses future directions. These directions include reducing the number of sequential communication stages, supporting provider replacement and membership change, strengthening the model against adaptive or mobile corruption, improving endpoint protection, exploring proactive refresh, and developing post-quantum instantiations of the underlying cryptographic components.

1.2 Publications

The research developed in this thesis has resulted in the following three manuscripts:

1. **Secure and Updatable Single Password Authentication** This work introduces UpSPA, an updatable threshold single-password authentication construction supporting secure login-server-specific secret updates and password updates. The manuscript is publicly available through the Cryptology ePrint Archive as Report 2026/784 [\[34\]](#).

2. **Secure and Updatable Single Password Authentication** A workshop version of the UpSPA construction, including its security model, protocol design, formal analysis, implementation, and performance evaluation, has been submitted to the *ACM Workshop on Privacy in the Electronic Society (WPES 2026)*.
3. **Robust Updatable Single Password Authentication** This manuscript extends the updatable SPA framework with quorum-based completion, synchronization, and recovery mechanisms for storage-provider unavailability and stale replicated records. It has been submitted to *ACM Transactions on Privacy and Security (TOPS)*.

The first two manuscripts present the design and evaluation of UpSPA, whereas the third develops RoUpSPA as a storage-resilient extension with separate cryptographic and availability thresholds.

Chapter 2

PRELIMINARIES

2.1 Notation and Conventions

This section fixes the notation used throughout the thesis. The purpose is not to introduce a new protocol, but to make the later descriptions of UpSPA, its resilient variant, and the implementation chapter easier to read. We use the same notation consistently for the fully available and resilient settings, and we introduce protocol-specific symbols only when they are needed later.

Let $\lambda \in \mathbb{N}$ denote the security parameter. Unless stated otherwise, all algorithms are probabilistic polynomial-time algorithms in λ . A function $\text{neg}(\lambda)$ is called negligible if it eventually becomes smaller than the inverse of every positive polynomial in λ . We write $\text{poly}(\lambda)$ for an unspecified polynomial. If two ensembles are computationally indistinguishable, we write $X \stackrel{c}{\approx} Y$; if they are statistically indistinguishable, we write $X \stackrel{s}{\approx} Y$.

For a positive integer n , we write $[n]$ for the set $\{1, \dots, n\}$. If S is a finite set, then $|S|$ denotes its cardinality. The notation $x \leftarrow \mathcal{D}$ means that x is sampled from the distribution $\mathcal{D}$, while $x \leftarrow y$ denotes assignment of the value y to the variable x . In particular, $x \leftarrow \{0, 1\}^\lambda$ denotes uniform sampling of a λ -bit string. We use $x \parallel y$ for concatenation, and we use $\perp$ for an undefined value or failed parsing. Protocol-level failure is written as **FAIL**, while successful completion is written as **OK**. When an execution is stopped intentionally before producing a normal output, we say that the party aborts and write **ABORT**.

The main parties are the user, the login servers, and the storage providers. The user is denoted by $\mathcal{U}$, and when several users are considered we write $\mathcal{U}_l$ for the l -th user. The set of login servers is denoted by $\mathcal{LS}$, and $\mathcal{LS}_j$ denotes the j -th login server. The set of storage providers is denoted by $\mathcal{SP}$, and $\mathcal{SP}_i$ denotes the i -th

storage provider.

We use n_u for the number of users, n_{ls} for the number of login servers, and n_{sp} for the number of storage providers. The storage threshold is denoted by t_{sp} . In the fully available version, the protocol is described with the assumption that the required storage-side parties are available whenever they are needed. In the resilient version, a storage-side quorum size may also be used; we denote it by q_{sp} . When a set $Q \subseteq [n_{sp}]$ is contacted during a phase, we call Q an available set or a quorum, depending on the context. A corrupted set of storage providers is denoted by C , and the corresponding honest set is denoted by $H = [n_{sp}] \setminus C$. These set notations are only conventions. The actual corruption bounds, availability assumptions, and quorum-intersection requirements are stated later in the threat model of each protocol.

The user identity is written as Uid . When the user index is important, we write Uid_l , and when both the user and login-server indices are relevant, we write $\text{Uid}_{l,j}$. The user's current password is denoted by pwd , and a replacement password in the password-update phase is denoted by pwd_{new} . A login-server domain label is written as ls or ls_j . The domain label is treated as part of the cryptographic input whenever the protocol must bind a stored value or verifier to a particular login server. This convention is important because the same user password may be used across different login servers, while the resulting protocol state must not become transferable from one domain to another.

The setup phase creates long-term user state. The value $\mathcal{R}^{sp}$ denotes the random secret used to derive storage-provider-specific identifiers. The value ssk denotes the user's signing secret key, and svk denotes the corresponding signing verification key stored by storage providers. The value K_0 denotes an independent symmetric key used to protect login-server-specific secrets. The value K denotes the full TOPRF key, while k_i denotes the share associated with storage provider $\mathcal{SP}_i$. The master ciphertext protected under the password-derived TOPRF output is denoted by cid . After a password update, the new master ciphertext is denoted by cid_{new} .

For each login server $\mathcal{LS}_j$, the protocol samples a random value as login-server-specific secret $\mathcal{R}^{\text{ls}_j}$. When this secret is refreshed, the new value is denoted by $\mathcal{R}_{\text{new}}^{\text{ls}_j}$. Storage-provider-specific identifiers for login-server records are denoted by

$\text{SUid}_{l,i,j}$. The ciphertext stored under such an identifier is denoted by $\mathbf{c}_j$, and after a secret update the new ciphertext is denoted by $\mathbf{c}_{j,\text{new}}$. The verification information registered at a login server is denoted by $\mathbf{vInfo}_j$, and after a secret update it is denoted by $\mathbf{vInfo}_{j,\text{new}}$. Freshness counters are written as $\mathbf{ctr}_{\text{ac}}$ for login-server-specific records and $\mathbf{ctr}_{\text{id}}$ for the master record. These counters are used only as freshness indicators; they are not secrets.

The main protocol phases are denoted by Π_{Setup} , Π_{Register} , $\Pi_{\text{Authenticate}}$, $\Pi_{\text{SecretUpdate}}$, and $\Pi_{\text{PwdUpdate}}$. Respectively, these stand for setup, registration, authentication, secret update, and password update. We do not introduce a separate ideal functionality for each phase. When a simulation-based proof is discussed, the full protocol functionality is written as $\mathcal{F}_{\text{UpSPA}}$ for UpSPA, and as $\mathcal{F}_{\text{RoUpSPA}}$ when the resilient variant is treated as a separate ideal object. We write **REAL** for a real-world execution and **IDEAL** for an ideal-world execution. The adversary is denoted by $\mathcal{A}$, and the simulator is denoted by **Sim**.

For cryptographic primitives, $\mathbf{H}$ denotes a hash function. The algorithms **Sign.KeyGen**, **Sign**, and **Sign.Verify** denote key generation, signing, and verification for a digital signature scheme. The values $\mathbf{ssk}$ and $\mathbf{svk}$ denote the signing key and verification key. The algorithms **Enc** and **Dec** denote encryption and decryption. The algorithms **TOPRFKeyGen** and **TOPRF** denote threshold OPRF key generation and threshold OPRF evaluation. We write $F_{\mathbf{K}}(\mathbf{pwd})$ for the TOPRF output obtained from password $\mathbf{pwd}$ under the full key $\mathbf{K}$. In an execution, the user obtains this value only after interacting with enough storage providers, while each individual storage provider contributes only its own share.

When discussing proofs, we use a tilde to denote values generated by the simulator rather than by the honest protocol. For example, $\tilde{x}$ denotes a simulated value corresponding to x . A superscript c denotes a value controlled, supplied, or modified by the adversary; for example, x^c is an adversarial version of x . When needed, a superscript o is used to mark honest values. These decorations are used only in security proofs and should not be interpreted as additional protocol fields.

Tables 2.1, 2.2, and 2.3 summarize the notation that appears most frequently in the thesis. Symbols that are local to a single construction step are defined at the

point where they are first used.

Notation	Meaning
λ	Security parameter.
$\text{neg}(\lambda)$	Negligible function in the security parameter.
$\text{poly}(\lambda)$	An unspecified polynomial in λ .
$[n]$	The set $\{1, \dots, n\}$.
$ S $	Cardinality of the finite set S .
$x \leftarrow \mathcal{D}$	Sampling x from distribution $\mathcal{D}$.
$x \leftarrow y$	Assignment of value y to variable x .
$x \parallel y$	Concatenation of strings x and y .
$\perp$	Undefined value, invalid parsing, or failed decryption output.
OK, FAIL, ABORT	Successful completion, protocol-level failure, and explicit abort.
$\mathcal{U}, \mathcal{U}_l$	The user, and the l -th user when users are indexed.
$\mathcal{LS}, \mathcal{LS}_j$	The set of login servers, and the j -th login server.
$\mathcal{SP}, \mathcal{SP}_i$	The set of storage providers, and the i -th storage provider.
n_u, n_{ls}, n_{sp}	Number of users, login servers, and storage providers.
t_{sp}	Storage-provider threshold.
q_{sp}	Quorum size used in the resilient construction.
C, H, Q	Corrupted set, honest set, and contacted/available provider set.

Table 2.1: General notation and party notation.

Finally, whenever an interactive protocol is written in compact form, the inputs of a party are placed inside parentheses and the outputs are written after the protocol execution. For example, a two-party protocol between parties P_0 and P_1 may be

Notation	Meaning
$\text{Uid}, \text{Uid}_l, \text{Uid}_{l,j}$	User identifier, indexed user identifier, and user/login-server indexed identifier.
$\text{pwd}, \text{pwd}_{\text{new}}$	Current password and new password.
ls, ls_j	Login-server domain label.
$\mathcal{R}^{sp}$	Random secret used to derive provider-specific identifiers.
$\mathcal{R}^{\text{ls}_j}, \mathcal{R}_{\text{new}}^{\text{ls}_j}$	Login-server-specific secret and its refreshed value.
$\mathcal{S}^{sp}, \sigma$	User signing secret and corresponding verification material.
ssk, svk	Signing key and verification key.
K_0	Symmetric key used for login-server-specific encrypted records.
K, k_i	Full TOPRF key and the share held by $\mathcal{SP}_i$.
$\text{cid}, \text{cid}_{\text{new}}$	Master ciphertext before and after password update.
$\text{SUid}_{l,i,j}$	Storage-provider-specific identifier for a login-server record.
$\text{c}_j, \text{c}_{j,\text{new}}$	Login-server-specific ciphertext before and after secret update.
$\text{vInfo}_j, \text{vInfo}_{j,\text{new}}$	Login-server verification information before and after secret update.
$\text{ctr}, \text{ctr}_{\text{root}}$	Freshness counter for site-specific state and master state.

Table 2.2: Protocol-state notation.

written as $P_0[\text{out}_0], P_1[\text{out}_1] \leftarrow \text{Protocol}(P_0[\text{in}_0], P_1[\text{in}_1])$. This notation is used only to describe the direction of inputs and outputs. It does not by itself impose security, fairness, or availability. Those properties are stated separately in the relevant threat model and security analysis sections.

Notation	Meaning
H	Hash function.
Enc, Dec	Encryption and decryption algorithms.
$\text{Sign.KeyGen}, \text{Sign}, \text{Sign.Verify}$	Digital-signature key generation, signing, and verification.
$\text{TOPRFKeyGen}, \text{TOPRF}$	Threshold OPRF key generation and evaluation.
$F_K(\text{pwd})$	Password-derived TOPRF output under the full key K .
$\Pi_{\text{Setup}}, \Pi_{\text{Register}}, \Pi_{\text{Authenticate}}$	Setup, registration, and authentication protocols.
$\Pi_{\text{SecretUpdate}}, \Pi_{\text{PwdUpdate}}$	Secret-update and password-update protocols.
$\mathcal{F}_{\text{UpSPA}}, \mathcal{F}_{\text{RoUpSPA}}$	Ideal functionality for UpSPA and, when needed, for the resilient variant.
$\text{REAL}, \text{IDEAL}$	Real-world and ideal-world executions.
$\mathcal{A}, \text{Sim}$	Adversary and simulator.
$\tilde{x}, x^c, x^o$	Simulated value, adversarial/corrupted value, and honest value.

Table 2.3: Cryptographic and proof notation.

2.2 Standard Definitions and Security Games

2.2.1 Basic Cryptographic Language

We first recall the basic cryptographic language used in the rest of the thesis. These notions are standard in modern cryptography, and we mainly use them to make the later protocol definitions and security proofs precise. The adversarial model itself, including which parties may be corrupted and how many corrupted parties are tolerated, is not fixed in this subsection. Those assumptions are stated separately in the threat model of each construction.

Security parameter and efficient algorithms. Let $\lambda \in \mathbb{N}$ be the security parameter. All algorithms in this thesis are assumed to receive 1^λ as an implicit input, even when this input is not written explicitly. A probabilistic polynomial-time algorithm, abbreviated as PPT, is a randomized algorithm whose running time is bounded by a polynomial in λ . We use PPT algorithms to model honest protocol parties, adversaries, simulators, and distinguishers. This computational viewpoint is standard in complexity-based cryptography and appears throughout the foundational treatments of modern cryptography [45].

Negligible functions. A function $\text{neg}(\lambda)$ is negligible if for every positive polynomial $p(\lambda)$ there exists a value λ_0 such that for every $\lambda > \lambda_0$, $\text{neg}(\lambda) < 1/p(\lambda)$. Informally, a negligible function is smaller than any inverse polynomial once the security parameter is large enough. In security statements, events that occur only with negligible probability are treated as cryptographically insignificant. This convention is used to separate feasible attacks from events that vanish faster than any inverse polynomial in the security parameter [26].

Experiments and probabilities. Most security definitions are written as experiments between a challenger and an adversary. An experiment fixes how public parameters or keys are generated, which information is given to the adversary, which oracle queries the adversary may ask, and what condition makes the adversary win. If $\text{EXP}(\mathcal{A}, \lambda)$ is such an experiment, then $\Pr[\text{EXP}(\mathcal{A}, \lambda) = 1]$ denotes the probability that the experiment outputs 1, where the probability is taken over all randomness used by the challenger, the adversary, and the honest algorithms. This experiment-based style is already present in early formalizations of encryption security and later became the standard way to state concrete cryptographic security notions [30].

Adversarial advantage. Security is usually not stated only by saying that an adversary wins with small probability. Instead, the adversary's success probability is compared with a natural baseline. For example, in a left-or-right indistinguishability game, the adversary receives a challenge generated using one of two possible

worlds and tries to guess which world was used. If the hidden bit is uniform, random guessing succeeds with probability $1/2$. The adversarial advantage is therefore written as

$$\mathcal{A}_{\Pi}^{\text{sec}}(\mathcal{A}, \lambda) = \left| \Pr[\text{EXP}_{\Pi}^{\text{sec}}(\mathcal{A}, \lambda) = 1] - \frac{1}{2} \right|.$$

The exact superscript **sec** depends on the security notion under discussion. A scheme is secure for that notion if the advantage of every PPT adversary is negligible in λ . This way of measuring advantage is closely related to the indistinguishability-based definitions used in probabilistic encryption and pseudorandomness [52].

Computational indistinguishability. Let $X = \{X_{\lambda}\}_{\lambda \in \mathbb{N}}$ and $Y = \{Y_{\lambda}\}_{\lambda \in \mathbb{N}}$ be two probability ensembles. We say that X and Y are computationally indistinguishable, written $X \stackrel{c}{\approx} Y$, if every PPT distinguisher $\mathcal{S}$ has only negligible advantage in telling them apart:

$$\left| \Pr[\mathcal{S}(1^{\lambda}, X_{\lambda}) = 1] - \Pr[\mathcal{S}(1^{\lambda}, Y_{\lambda}) = 1] \right| \leq \text{neg}(\lambda)$$

. This notion is used whenever a proof replaces one distribution by another distribution that should look the same to every efficient adversary. Computational indistinguishability is one of the central notions behind pseudorandomness and semantic security, and it goes back to the early complexity-theoretic foundations of cryptography [59]. In this thesis, computational indistinguishability is the default notion unless statistical indistinguishability is explicitly stated.

Oracle access and adaptive queries. Some security experiments give the adversary oracle access to honest algorithms. For example, in a signature security game, the adversary may ask a signing oracle to sign messages of its choice before attempting to forge a new signature. An adversary is adaptive if it can choose each new query after seeing the answers to previous queries. Adaptive oracle access is important because many attacks on authentication protocols are interactive: the adversary may first observe honest executions and then choose malicious messages

depending on what it has learned. The adaptive chosen-message formulation for signatures is a classical example of this style [31].

Views and transcripts. The view of a party in a protocol execution consists of its input, its randomness, the messages it receives, and any local state it stores during the execution. We write View_P^Π for the view of party P in an execution of protocol Π , whenever the party and protocol must be made explicit. The transcript of an execution is the ordered list of messages exchanged between parties. A transcript is usually public to the parties who participate in the execution, but a view may also include private randomness and secret local state. The idea of comparing views is central in simulation-based security, starting from zero-knowledge and secure computation [29].

Correctness. Correctness says that when all parties follow the protocol honestly and the required availability assumptions hold, the intended output is produced except with negligible probability. For an authentication protocol, correctness means that a legitimate user who enters the correct password and obtains the required storage-provider responses is accepted by the login server. For update protocols, correctness means that an honestly completed update leads later honest executions to use the updated state. Correctness is separate from security: correctness concerns honest behavior, while security concerns the behavior of adversarial parties. This separation between functional correctness and adversarial security is standard in secure computation definitions [6].

Games and hybrids. Security proofs in this thesis often use a sequence of games $G_0, G_1, \dots, G_m$, where G_0 is the real attack experiment and G_m is an experiment in which the adversary's success probability is easy to bound. Each neighboring pair of games is shown to be indistinguishable. A typical proof therefore has the form

$$|\Pr[G_0 = 1] - \Pr[G_m = 1]| \leq \sum_{r=0}^{m-1} |\Pr[G_r = 1] - \Pr[G_{r+1} = 1]|,$$

This style is useful because complex protocol security can be reduced to simpler assumptions, such as pseudorandomness, ciphertext privacy, ciphertext integrity,

signature unforgeability, or collision resistance. We use this proof language later for UpSPA and its resilient variant. The game-hopping style is widely used in modern cryptographic proofs and is treated explicitly by Shoup and by Bellare and Rogaway [9].

Real and ideal executions. For protocol-level security, we sometimes compare a real execution of a protocol with an ideal execution in which a trusted party performs the intended task. We write $\text{REAL}_{\Pi, \mathcal{A}}(\lambda)$ for the real execution of protocol Π with adversary $\mathcal{A}$, and $\text{IDEAL}_{\mathcal{F}, \text{Sim}}(\lambda)$ for the ideal execution of functionality $\mathcal{F}$ with simulator Sim . The goal is to show that for every efficient real-world adversary $\mathcal{A}$, there exists an efficient simulator Sim such that

$$\text{REAL}_{\Pi, \mathcal{A}}(\lambda) \stackrel{c}{\approx} \text{IDEAL}_{\mathcal{F}, \text{Sim}}(\lambda).$$

This means that whatever an efficient adversary can cause or observe in the real protocol can also be explained in the ideal execution. In this thesis, this language is used only at the level of the full protocol functionality, namely $\mathcal{F}^{\text{UpSPA}}$ or $\mathcal{F}^{\text{RoUpSPA}}$, and not as a separate functionality for each protocol phase. The detailed corruption and availability assumptions are stated later in the corresponding threat model. This real/ideal viewpoint follows the older simulation-based tradition of secure computation, especially the works of Goldreich, Micali, and Wigderson; Micali and Rogaway; and Beaver [27].

2.2.2 Password-Specific attacks

We next fix the password-specific terminology used throughout the thesis. The protocols studied here are not designed for high-entropy cryptographic keys memorized by users. They are designed for ordinary human passwords, and this changes the security goal: the protocol cannot make password guessing impossible, but it should prevent the adversary from testing guesses offline.

Low-entropy password. A password is called low entropy when it is chosen from a space that is small enough, structured enough, or biased enough that an adversary

may try likely candidates. In practice, this means that the password should not be modeled as a uniformly random λ -bit secret. The user may choose it from natural language words, personal patterns, reused passwords, or other non-uniform distributions. We denote the user's password by `pwd` and treat it as a human-memorable secret rather than as a cryptographic key. This distinction is important because password-based protocols are expected to remain meaningful even when the password space is small enough to be enumerated by an offline adversary [8].

Online dictionary attack. An online dictionary attack is an attack in which the adversary must interact with an honest party, such as a login server or storage provider, for each password guess. For example, the adversary may submit a guessed password or a value derived from it and observe whether the login attempt succeeds. Such attacks cannot be completely eliminated in password-based authentication, because a legitimate user must also be allowed to try a password. However, online attacks are naturally constrained by rate limits, account lockout policies, server-side monitoring, and network delay. Therefore, in this thesis, an online guessing attack is not considered a cryptographic break by itself; the goal is to ensure that the adversary cannot transform it into a faster offline test [1].

Offline dictionary attack. An offline dictionary attack is an attack in which the adversary obtains some stored or observed data and can test many password candidates locally, without further interaction with an honest party. This is the main attack that UpSPA and its resilient variant are designed to prevent. If a server stores a deterministic value derived from the password, or if a ciphertext gives the adversary a recognizable correctness test after decryption, then the adversary may enumerate password candidates and check them offline. Preventing this kind of local verification is central in password-based cryptography and was already a main motivation for encrypted key exchange [10].

Password guessing probability. Since passwords may be chosen from arbitrary human distributions, we do not assume that every password has the same probability.

Instead, we use a guessing bound. Let $\text{ProbGuess}(N)$ denote the probability that an adversary, using its prior knowledge about the user or password distribution, guesses the correct password within N attempts. Equivalently, if the adversary outputs N candidate passwords $\text{pwd}'_1, \dots, \text{pwd}'_N$, then $\text{ProbGuess}(N)$ captures the probability that the real password pwd is contained in this list. Security statements in this thesis use this term to separate unavoidable online guessing from additional cryptographic leakage. Ideally, the adversary's success probability should be no more than $\text{ProbGuess}(N)$ plus a negligible term [1].

Password-derived encryption of random-looking values. A recurring idea in SPA-style protocols is that the password, or a value derived from it through an OPRF or TOPRF, may be used to protect a secret only when the protected plaintext is random-looking. If decrypting with a wrong password produces an output that is obviously malformed, then the ciphertext itself becomes an offline password test. In contrast, when every password candidate decrypts the ciphertext to a value that looks syntactically valid, the adversary still needs some external verification step. In our setting, that verification step should require interaction with the login server or enough storage providers. This is why UpSPA encrypts random-looking secrets, shares, or keys, rather than encrypting structured user data directly under the password [13].

Phishing attack. A phishing attack occurs when an adversary presents a malicious website or interface as an honest login server and induces the user to enter pwd or execute the authentication protocol under the wrong origin. In the SPA setting, the main concern is that an interaction initiated for the phishing site may be reused to authenticate to an honest login server. UpSPA and its resilient variant address this threat at the protocol level by binding storage identifiers and login-server verification information to the login-server domain ls_j . Consequently, when the trusted client derives these values for a phishing domain ls'_j , the resulting records and verification information differ from those expected by the honest login server and cannot be transferred to it. This guarantee assumes that the client or browser

extension correctly identifies the authenticated origin and that the user endpoint remains trusted; it does not replace TLS or browser-origin validation, nor does it protect information entered outside the authentication protocol [35].

Honeypot attack. A honeypot attack is mounted by a malicious login server that offers an apparently legitimate service primarily to collect reusable user credentials. Unlike a phishing site, a honeypot does not need to impersonate an existing login server, because the user may intentionally register with and authenticate to the malicious service. In a conventional password system, such a server learns either `pwd` or a deterministic password-derived value that may support password guessing or cross-site impersonation. In UpSPA, the malicious login server receives only high-entropy verification information that is independent of `pwd` and bound to its own domain. Its protocol view should therefore provide neither an offline password test nor a credential that can be transferred to another honest login server, provided that the adversary remains within the stated storage-provider corruption bound [1].

Keylogger attack. A keylogger is malicious software or hardware that records the user's input at the endpoint on which the password is entered. Because the user must provide `pwd` to the trusted client before the OPRF or TOPRF evaluation can be initiated, directly observing this input bypasses the password-hiding properties of the protocol. Distributing authentication state among storage providers protects against compromise of stored records, login servers, and fewer than threshold-many providers, but it cannot conceal a password that is read directly from the user's device. The core threat models of UpSPA and its resilient variant therefore assume that the client is trusted while `pwd` is entered. A separate trusted input device may reduce this exposure, but such a device is not part of the core constructions considered in this thesis [35].

Denial-of-service attack. A denial-of-service attack aims to prevent an honest user or service from completing an operation rather than to recover `pwd` or forge valid authentication state. The adversary may exhaust computational or commu-

nication resources, delay or drop messages, withhold provider responses, or return malformed values that force an honest execution to abort. In the protocols of this thesis, authentication cannot complete unless at least t_{sp} valid storage-provider responses are obtained, while a state-changing operation in the resilient variant cannot complete unless the required q_{sp} acknowledgements are collected. The resilient construction therefore tolerates partial provider unavailability only while the relevant threshold or quorum remains reachable. An abort caused by insufficient responses is an availability failure, not by itself a password-recovery, impersonation, or state-forgery attack [53].

2.2.3 Standard Cryptographic Primitives

This subsection recalls the cryptographic primitives used by UpSPA and its resilient variant. The definitions are stated at the level needed for the later constructions and proofs. For the primitives that directly appear in the proof reductions, we also give the corresponding security games. For primitives used mainly as building blocks or system abstractions, such as OPRFs, threshold secret sharing, oblivious transfer, public bulletin boards, quorums, and blockchains, we give mathematical definitions without adding full security games.

Collision-resistant hash functions. A hash function maps arbitrary-length inputs to fixed-length outputs. Formally, we write $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$. In the thesis notation, we suppress any public index or description key of the hash function and simply write H . A hash function is collision resistant if no efficient adversary can find two distinct inputs with the same hash value. This is the only hash-function property that is explicitly needed in the later security arguments, because hash values are used to bind identifiers, domain names, counters, ciphertexts, and verification information to the intended protocol context [57].

Collision Resistance of H

Experiment

Challenger *chal*. Give 1^λ to $\mathcal{A}$.

Adversary $\mathcal{A}$. Output $x, x' \in \{0, 1\}^*$.

Challenger *chal*. Returns 1 iff $x \neq x' \wedge H(x) = H(x')$; otherwise returns 0.

The collision-resistance advantage of $\mathcal{A}$ is $\text{Adv}_H^{\text{cr}}(\mathcal{A}, \lambda) = \Pr[\text{Exp}_H^{\text{cr}}(\mathcal{A}, \lambda) = 1]$. We say that H is collision resistant if for every PPT adversary $\mathcal{A}$, the value $\text{Adv}_H^{\text{cr}}(\mathcal{A}, \lambda)$ is negligible in λ .

Digital signatures. A digital signature scheme is a public-key primitive consisting of three PPT algorithms ($\text{Sign.KeyGen}, \text{Sign}, \text{Sign.Verify}$). The key-generation algorithm outputs a signing key and a verification key, written as $(\text{ssk}, \text{svk}) \leftarrow \text{Sign.KeyGen}(1^\lambda)$. The signing algorithm takes a signing key and a message and outputs a signature, written as $\sigma \leftarrow \text{Sign}(\text{ssk}, m)$. The verification algorithm takes a verification key, a message, and a signature and outputs a bit, written as $b \leftarrow \text{Sign.Verify}(\text{svk}, m, \sigma) \in \{0, 1\}$. Correctness requires that every honestly generated signature verifies, namely

$$\Pr[\text{Sign.Verify}(\text{svk}, m, \text{Sign}(\text{ssk}, m)) = 1 : (\text{ssk}, \text{svk}) \leftarrow \text{Sign.KeyGen}(1^\lambda)] = 1.$$

The security notion used in this thesis is existential unforgeability under adaptive chosen-message attack, usually denoted $\text{EUF} - \text{CMA}$. This is the standard notion introduced by Goldwasser, Micali, and Rivest [31].

EUF-CMA Security of Digital Signatures

Setup

Challenger *chal*. Sample $(\text{ssk}, \text{svk}) \leftarrow \text{Sign.KeyGen}(1^\lambda)$, set $\mathcal{Q} \leftarrow \emptyset$, and give svk to $\mathcal{A}$.

Signing oracle $\mathcal{O}_{\text{Sign}}(m)$

Adversary $\mathcal{A}$. Submit a message m .

Challenger chal . Compute $\sigma \leftarrow \text{Sign}(\text{ssk}, m)$, update $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{m\}$, and return σ . This exchange may be repeated adaptively.

Forgery

Adversary $\mathcal{A}$. Output (m^*, σ^*) .

Challenger chal . Returns 1 iff $\text{Sign.Verify}(\text{svk}, m^*, \sigma^*) = 1 \wedge m^* \notin \mathcal{Q}$; otherwise returns 0.

The EUF – CMA advantage of $\mathcal{A}$ is $\text{Adv}_{\text{Sig}}^{\text{EUF-CMA}}(\mathcal{A}, \lambda) = \Pr[\text{Exp}_{\text{Sig}}^{\text{EUF-CMA}}(\mathcal{A}, \lambda) = 1]$. A digital signature scheme is EUF – CMA-secure if this advantage is negligible for every PPT adversary $\mathcal{A}$. In UpSPA and the resilient construction, this prevents an adversary from forging update authorizations or authenticated state records without the user’s signing key.

Authenticated symmetric encryption. An authenticated symmetric encryption scheme is a symmetric-key primitive that provides both confidentiality and ciphertext integrity. We write it as a triple of PPT algorithms $(\text{KeyGen}, \text{Enc}, \text{Dec})$. The key-generation algorithm samples a secret key $\text{sk} \leftarrow \text{KeyGen}(1^\lambda)$. The encryption algorithm takes a key and a message $m \in \mathcal{M}$ and outputs a ciphertext $c \leftarrow \text{Enc}_{\text{sk}}(m)$. The decryption algorithm takes a key and a ciphertext and outputs either a message or the failure symbol $\perp$, written as $m' \leftarrow \text{Dec}_{\text{sk}}(c) \in \mathcal{M} \cup \{\perp\}$. Correctness requires $\Pr[\text{Dec}_{\text{sk}}(\text{Enc}_{\text{sk}}(m)) = m : \text{sk} \leftarrow \text{KeyGen}(1^\lambda)] = 1$. Authenticated encryption is secure when it provides both message privacy and ciphertext integrity [7]. In the protocols of this thesis, privacy hides random-looking secrets and secret records, while ciphertext integrity ensures that malformed or adversarially modified records are rejected.

IND-CPA Security of Symmetric Encryption

Setup

Challenger *chal*. Sample $\text{sk} \leftarrow \text{KeyGen}(1^\lambda)$ and give 1^λ to $\mathcal{A}$.

Encryption oracle $\mathcal{O}_{\text{Enc}}(m)$

Adversary $\mathcal{A}$. Submit a message m .

Challenger *chal*. Return $c \leftarrow \text{Enc}_{\text{sk}}(m)$. The oracle remains available both before and after the challenge.

Challenge

Adversary $\mathcal{A}$. Output m_0, m_1 with $|m_0| = |m_1|$.

Challenger *chal*. Sample $b \xleftarrow{\$} \{0, 1\}$, compute $c^* \leftarrow \text{Enc}_{\text{sk}}(m_b)$, and return c^* to $\mathcal{A}$.

Guess

Adversary $\mathcal{A}$. After any further oracle queries, output $b' \in \{0, 1\}$.

Challenger *chal*. Returns 1 iff $b' = b$; otherwise returns 0.

The IND – CPA advantage of $\mathcal{A}$ is $\text{Adv}_{\text{AE}}^{\text{IND-CPA}}(\mathcal{A}, \lambda) = |\Pr[\text{Exp}_{\text{AE}}^{\text{IND-CPA}}(\mathcal{A}, \lambda) = 1] - 1/2|$. We say that the encryption scheme satisfies IND – CPA privacy if this advantage is negligible for every PPT adversary $\mathcal{A}$.

INT-CTXT Security of Authenticated Encryption

Setup

Challenger *chal*. Sample $\text{sk} \leftarrow \text{KeyGen}(1^\lambda)$, set $\mathcal{C} \leftarrow \emptyset$, and give 1^λ to $\mathcal{A}$.

Encryption oracle $\mathcal{O}_{\text{Enc}}(m)$

Adversary $\mathcal{A}$. Submit a message m .

Challenger *chal*. Compute $c \leftarrow \text{Enc}_{\text{sk}}(m)$, update $\mathcal{C} \leftarrow \mathcal{C} \cup \{c\}$, and return c . This exchange may be repeated adaptively.

Forgery

Adversary $\mathcal{A}$. Output a ciphertext c^* .

Challenger chal. Returns 1 iff $c^* \notin \mathcal{C} \wedge \text{Dec}_{\text{sk}}(c^*) \neq \perp$; otherwise returns 0.

The ciphertext-integrity advantage of $\mathcal{A}$ is

$$\text{Adv}_{\text{AE}}^{\text{int-ctxt}}(\mathcal{A}, \lambda) = \Pr[\text{Exp}_{\text{AE}}^{\text{int-ctxt}}(\mathcal{A}, \lambda) = 1].$$

We say that authenticated encryption provides ciphertext integrity if this advantage is negligible for every PPT adversary $\mathcal{A}$. When associated data is used, the encryption and decryption algorithms additionally receive a public string ad ; the same definitions apply, except that ciphertext validity is checked relative to the same associated data.

Pseudorandom functions. A pseudorandom function family is a keyed family of deterministic functions $\{F_K : X \rightarrow Y\}_{K \in \mathcal{K}}$. Given a key K , the function maps an input $x \in X$ to an output $F_K(x) \in Y$. Informally, the family is secure if an efficient algorithm with oracle access cannot distinguish oracle access to $F_K(\cdot)$ for a uniformly sampled key K from oracle access to a truly random function with the same domain and range. We use PRFs only through OPRF and TOPRF evaluations, where the user obtains a password-derived value without revealing the password to the storage providers [28].

Oblivious pseudorandom functions. An oblivious pseudorandom function is a two-party protocol between a sender and a receiver. The sender holds a PRF key K , the receiver holds an input x , and the receiver learns $F_K(x)$. The sender should learn nothing about x , and the receiver should learn nothing about K or about $F_K(x')$ for other inputs $x' \neq x$. We write this interaction abstractly as $F_K(x) \leftarrow \text{OPRF}(\text{Receiver}[x], \text{Sender}[K])$. In the password-authentication setting, the receiver is the user and the input is usually the password pwd . The storage provider contributes the PRF key material but should not learn pwd or the resulting PRF output [22].

Threshold oblivious pseudorandom functions. A threshold oblivious pseudorandom function distributes the key of a pseudorandom function among several

servers. A t -out-of- n TOPRF consists of a key-generation algorithm and an interactive evaluation protocol. The key-generation algorithm is written as $(K, \{k_i\}_{i \in [n]}) \leftarrow \text{TOPRF.KeyGen}(1^\lambda, n, t)$, where K is the underlying full PRF key and k_i is the share held by server S_i . During evaluation, a receiver holding an input $x \in \mathcal{X}$ interacts with a participating set $U \subseteq [n]$. When $|U| \geq t$ and the evaluation succeeds, the receiver obtains $F_K(x)$ without revealing x or the resulting output to the servers.

The security analysis in this thesis uses two distinct TOPRF properties. Receiver-side pseudorandomness protects the output obtained on a fresh input, whereas receiver-input obliviousness protects the receiver's input from the participating corrupted servers. These notions are stated separately because pseudorandomness of the output does not by itself hide the interactive transcript, and input obliviousness does not by itself make the output pseudorandom.

TOPRF Receiver-Side Pseudorandomness

Corruption and setup

Adversary $\mathcal{A}$. Chooses a corrupted set $C \subseteq [n]$ with $|C| \leq t - 1$.

Challenger chal . Samples

$$(K, \{k_i\}_{i \in [n]}) \leftarrow \text{TOPRF.KeyGen}(1^\lambda, n, t),$$

initializes $Q \leftarrow \emptyset$, and gives $\{k_i\}_{i \in C}$ to $\mathcal{A}$.

Evaluation oracle $\mathcal{O}_{\text{Eval}}(x, U)$

Adversary $\mathcal{A}$. Submits an input $x \in \mathcal{X}$ and a participating set $U \subseteq [n]$ satisfying $|U| \geq t$.

Challenger chal . Executes the TOPRF evaluation with the honest servers in $U \setminus C$ and lets $\mathcal{A}$ control the corrupted servers in $U \cap C$. It returns the receiver output y . If $y \neq \perp$, it updates $Q \leftarrow Q \cup \{x\}$.

Challenge

Adversary $\mathcal{A}$. Chooses a fresh input $x^* \in \mathcal{X} \setminus Q$.

Challenger chal . Samples $b \leftarrow \{0, 1\}$. If $b = 0$, it sets $y^* \leftarrow F_K(x^*)$; otherwise, it samples $y^* \leftarrow \mathcal{Y}$ uniformly. It returns y^* to $\mathcal{A}$.

Guess

Adversary $\mathcal{A}$. Outputs $b' \in \{0, 1\}$.

Challenger chal . Returns 1 iff $b' = b$; otherwise returns 0.

The receiver-side pseudorandomness advantage is

$$\text{Adv}_{\text{TOPRF}}^{\text{prf}}(\mathcal{A}, \lambda) = \left| \Pr[\text{Exp}_{\text{TOPRF}}^{\text{prf}}(\mathcal{A}, \lambda) = 1] - \frac{1}{2} \right|.$$

A TOPRF satisfies receiver-side pseudorandomness if this advantage is negligible for every PPT adversary corrupting fewer than t servers.

TOPRF Receiver-Input Obliviousness**Corruption and setup**

Adversary $\mathcal{A}$. Chooses a corrupted set $C \subseteq [n]$ with $|C| \leq t - 1$.

Challenger chal . Samples

$$(K, \{k_i\}_{i \in [n]}) \leftarrow \text{TOPRF.KeyGen}(1^\lambda, n, t)$$

and gives $\{k_i\}_{i \in C}$ to $\mathcal{A}$.

Ordinary execution oracle $\mathcal{O}_{\text{View}}(x, U)$

Adversary $\mathcal{A}$. Submits $x \in \mathcal{X}$ and $U \subseteq [n]$ with $|U| \geq t$.

Challenger chal . Runs the evaluation with an honest receiver on x , simulates the honest servers in $U \setminus C$, and gives $\mathcal{A}$ all messages and local views of the corrupted servers in $U \cap C$. The receiver output is not revealed. The oracle may be queried polynomially many times and adaptively.

Challenge

Adversary $\mathcal{A}$. Outputs two equal-length inputs $x_0, x_1 \in \mathcal{X}$ and a participating set $U^* \subseteq [n]$ satisfying $|U^*| \geq t$.

Challenger chal . Samples $b \leftarrow \{0, 1\}$, runs the evaluation with an honest receiver on x_b , and gives $\mathcal{A}$ the messages and local views of the corrupted servers in $U^* \cap C$. The receiver output is not revealed.

Guess

Adversary $\mathcal{A}$. Outputs $b' \in \{0, 1\}$.

Challenger chal . Returns 1 iff $b' = b$; otherwise returns 0.

The receiver-input-obliviousness advantage is

$$\text{Adv}_{\text{TOPRF}}^{\text{rio}}(\mathcal{A}, \lambda) = \left| \Pr[\text{Exp}_{\text{TOPRF}}^{\text{rio}}(\mathcal{A}, \lambda) = 1] - \frac{1}{2} \right|.$$

A TOPRF satisfies receiver-input obliviousness if this advantage is negligible for every PPT adversary corrupting fewer than t servers.

The UpSPA and RoUpSPA security proofs invoke the two notions above. Evaluation robustness is related to the verifiability and consistency property of V-OPRFs, which ensures that every non-rejecting evaluation is computed according to the function fixed during setup [37]. The inability of a subthreshold coalition to obtain an additional TOPRF output without another threshold interaction is captured by the ticket-based UC TOPRF functionality and threshold one-more security treatment of TOPPSS [38]. These auxiliary properties are not used directly in the main reductions.

Threshold secret sharing. A t -out-of- n threshold secret sharing scheme distributes a secret S among n parties so that any set of at least t shares reconstructs S , while any set of fewer than t shares reveals no information about S . We write the sharing and reconstruction algorithms as $\{s_i\}_{i \in [n]} \leftarrow \text{TSS.share}(S, n, t)$ and $S \leftarrow \text{TSS.reconst}(\{s_i\}_{i \in I})$ for any $I \subseteq [n]$ with $|I| \geq t$. Correctness requires that reconstruction from any authorized set returns the original secret. Privacy requires that for every unauthorized set $J \subseteq [n]$ with $|J| < t$, the joint distribution $\{s_i\}_{i \in J}$ is independent of S . In Shamir's construction, this privacy is information-theoretic [58].

Verifiable secret sharing. A verifiable secret sharing scheme augments threshold secret sharing with a mechanism for checking that shares are consistent with a single

underlying secret. In the notation of this thesis, the sharing algorithm outputs both shares and verification material, written as $\{(s_i, \pi_i)\}_{i \in [n]} \leftarrow \text{TSS.share}(S, n, t)$, where π_i is a proof or verification object attached to share s_i . A verification procedure checks whether a share is well-formed, and reconstruction uses a sufficient number of valid share/proof pairs, written as $S \leftarrow \text{TSS.reconst}(\{(s_i, \pi_i)\}_{i \in I})$. The purpose of verifiability is to prevent a malicious dealer, storage provider, or recovery procedure from causing different honest parties to reconstruct inconsistent secrets. In this thesis, VSS-style language is used only to express that invalid shares can be detected before they affect reconstruction [55].

Oblivious transfer. A one-out-of-two oblivious transfer protocol is a two-party protocol between a sender and a receiver. The sender holds two messages (m_0, m_1) , and the receiver holds a choice bit $b \in \{0, 1\}$. At the end of the protocol, the receiver learns m_b and should learn nothing about m_{1-b} , while the sender should learn nothing about b . We write this abstractly as $m_b \leftarrow \text{OT}(\text{Receiver}[b], \text{Sender}[m_0, m_1])$. More generally, one-out-of- N OT lets the receiver obtain one selected item from a sender's database without revealing the selected index. In this thesis, OT is not a core primitive of UpSPA, but it is useful for describing privacy-preserving retrieval variants in the SPA literature [19].

Public bulletin board. A public bulletin board is modeled as a public append-only map $\mathcal{PBB} : \mathcal{T} \rightarrow \mathcal{M}^*$, where $\mathcal{T}$ is a tag space and $\mathcal{M}^*$ denotes finite sequences of messages. The operation $\mathcal{PBB}.\text{write}(\tau, m)$ appends message m to the sequence stored under tag τ , and $\mathcal{PBB}.\text{read}(\tau)$ returns the ordered sequence currently stored under that tag. The append-only property means that if a read at time r returns a sequence L , then any later read under the same tag returns a sequence for which L is a prefix. The bulletin board is public: any party may read it, and adversarial parties may attempt to write to it. Therefore, it provides neither confidentiality nor authenticity by itself. In the resilient construction, authenticity is instead obtained from signatures and freshness is obtained from counters. A blockchain can be viewed as one possible implementation of such a public append-only synchronization

layer [23].

Quorum systems. Let the storage providers be indexed by $[n_{sp}]$. A quorum system is a collection $\mathcal{Q} \subseteq 2^{[n_{sp}]}$ of subsets of providers, where each $Q \in \mathcal{Q}$ is called a quorum. The basic consistency requirement is intersection: for all $Q, Q' \in \mathcal{Q}$, $Q \cap Q' \neq \emptyset$. For uniform quorums of size q_{sp} , every two quorums satisfy $|Q \cap Q'| \geq 2q_{sp} - n_{sp}$. Thus, the condition $2q_{sp} > n_{sp}$ guarantees non-empty intersection. If at most f storage providers may be corrupted and we need an honest provider in every intersection, then the stronger condition $2q_{sp} - n_{sp} > f$ is sufficient. In the resilient construction, this mathematical overlap is what allows a later read or recovery phase to observe information written by a previous successful update. Counters are then used to choose the freshest valid record among the records returned by the contacted quorum [25].

Blockchain as an append-only log. A blockchain is a hash-linked sequence of blocks $\mathcal{B} = (B_0, B_1, \dots, B_h)$, where each block B_r contains a batch of records and, for $r > 0$, a digest of the previous block. Abstractly, one can write $B_r = (\text{payload}_r, H(B_{r-1}), \text{meta}_r)$. This hash chaining makes the order of accepted records explicit: changing an older block changes its digest and therefore invalidates later links unless the later chain is recomputed or replaced according to the underlying consensus rule. In this thesis, we do not prove blockchain consensus security. We use the blockchain only as a high-level example of an append-only public log suitable for implementing $\mathcal{PBB}$ -style synchronization [54].

Chapter 3

RELATED WORK

Password authentication has developed along several lines. Some works strengthen traditional two-party login, others move part of the trust to a helper device or storage provider, and another group replaces direct login with Single Sign-On (SSO) or threshold SSO tokens. The works closest to this thesis preserve an experience close to ordinary password authentication while preventing offline password guessing after server or storage compromise. Accordingly, this chapter is organized by function: single-password authentication, password-protected secret recovery, threshold SSO, password-hardening and key-management services, and the synchronization mechanisms motivating the resilient construction. Table 3.6 summarizes the main distinctions for UpSPA and RoUpSPA.

3.1 *Single-Password Authentication and Its Threshold Extensions*

Traditional password authentication stores either the password or a deterministic password-derived value at the login server. If the server is malicious or its database is leaked, an adversary can test guesses offline, and password reuse may extend one compromise to other services. Large-scale studies confirm that reuse is common [21], while comparative work identifies deployability and user familiarity as major barriers to replacing passwords [12]. PAKE protocols, beginning with encrypted key exchange [10], reduce risks associated with transmitting passwords, but the SPA line addresses a stronger requirement: the login server (service provider) should learn neither the password nor a reusable deterministic function of it.

Acar, Belenkiy, and Küpçü introduced Single Password Authentication (SPA) [1]. A user remembers one password for several services while each service receives only verification material for an independently generated high-entropy authentication

secret. The corresponding secret is kept by a helper storage component in password-protected form and is recovered during authentication. Thus, the login server never receives the password, and the storage provider alone has no direct offline test for password guesses.

The cloud constructions build on hidden credential retrieval, which allows a reusable password to recover a random-looking credential from untrusted storage without exposing an offline verification test [13]. SPA also considers mobile-helper variants in which the device stores or reconstructs the secret and the long-term password is not entered into an untrusted terminal. In either case, the helper is a single cloud provider or mobile device rather than a threshold set. The principal limitation is therefore collusion: if the login server and storage provider are both controlled, their information can be combined for an offline dictionary attack. SPA also lacks native distributed password update and login-server-specific secret update, and some constructions require additional cryptographic support at the login server.

Browser-based password-manager designs such as Bicakci et al. [11] likewise reduce reuse by deriving or managing site-specific secrets on the client side, with an emphasis on usability and deployment. They do not distribute state across a threshold set or define native distributed password and secret updates. Device-enhanced protocols, particularly DE-PAKE [40], strengthen password authentication with one auxiliary device and provide strong online/offline protection against an active network adversary without changing ordinary password-over-TLS servers. However, DE-PAKE is not a t -of- n storage protocol and does not maintain threshold-distributed per-login-server state. Replacing a device-side key may change a hardened site credential, but this differs from UpSPA’s authenticated update of distributed login-server-specific state.

Threshold SPA (TSPA) [35] addresses the collusion weakness of single-storage SPA by distributing state across n_{sp} storage providers with threshold t_{sp} . The user authenticates through at least t_{sp} providers, while an offline dictionary attack requires the login server together with at least t_{sp} providers. The user secret-shares a random authentication secret, encrypts each share under a provider-specific OPRF output on the password, and registers verification information derived from the

reconstructed secret and login-server domain. Fewer than t_{sp} providers cannot reconstruct the secret, and the login server with fewer than t_{sp} providers still lacks an offline password test.

This construction also shows why merely secret-sharing an SPA credential is insufficient. Direct share release to the login server, or ordinary password-derived authentication at each provider, would recreate an offline test. TSPA instead makes each retrieval depend on an OPRF evaluation: the provider contributes key material without learning the password or the OPRF output. DiSPP [36] later generalizes this approach to broader provider configurations and participation policies, but, like TSPA, focuses on registration and authentication rather than a complete update lifecycle.

UpSPA [34] extends this line with first-class password update and login-server-specific secret update. It preserves threshold resistance to offline guessing, remains compatible with ordinary login servers, and prevents false account creation or unauthorized record replacement through high-entropy identifiers, authenticated records, and explicit update authorization. RoUpSPA adds resilience: authentication uses any valid set of at least t_{sp} provider responses, while registration and login-server-specific secret update complete after the login-server step and acknowledgment from a valid q_{sp} provider quorum. Password update has no login-server step and completes after the q_{sp} write quorum acknowledges the new record. Providers that were unavailable can later recover the freshest valid state. Thus, the progression is from one helper in SPA, to threshold authentication in TSPA/DiSPP, to native updates in UpSPA, and finally quorum-based completion and missed-provider recovery in RoUpSPA.

3.2 Password-Protected Secret Sharing and Password-Based Secret Recovery

Password-protected secret sharing (PPSS) protects a high-entropy secret using a low-entropy password and a set of servers. The servers should learn neither the password nor the secret, and a coalition below the threshold should obtain no offline verification test; successful guessing remains bounded by online interactions and the

password distribution. Bagherzandi et al. [4] formalized PPSS as a threshold secret-recovery primitive. Its output is a reconstructed secret, not authentication to an ordinary login server, and it does not define the SPA update lifecycle.

Later works improve robustness, composability, or round complexity. Memento [15] reconstructs secrets from one password in a hostile environment while protecting the password against malicious recovery servers. Round-optimal PPSS and T-PAKE [37] connect password-protected sharing with password-authenticated key exchange and reduce communication rounds in the password-only model. TOPPSS [38] gives efficient threshold-OPRF-based recovery with a single reconstruction round and low client and server cost. These schemes provide secret recovery or key establishment rather than per-service authentication state, native password update, or login-server-specific secret rotation.

More specialized systems include TPASS [60], which supports efficient threshold password-authenticated secret recovery for cloud computing, and the construction of Huszti et al. [33], which provides scalable password-based threshold authentication and mutually authenticated key agreement with key confirmation for smart homes. Their service models differ from the unmodified multi-service SPA setting. TSPA, UpSPA, and RoUpSPA draw on PPSS and OPRF-based recovery ideas, but adapt them to authentication across independent login servers, with per-login-server state, password and secret updates, and, in RoUpSPA, recovery for providers that missed a completed operation.

3.3 Threshold Token-Based Single Sign-On

Threshold SSO reduces password burden through a different architecture. The user authenticates to a logical identity-provider function distributed among several identity servers, which jointly issue a token such as a threshold signature or MAC. The relying service must accept this token, so authentication and update logic are centered at the IdP layer rather than stored as per-login-server SPA state.

PASTA [2] is the main starting point for password-based threshold token authentication. Any t of n identity servers can authenticate the user and issue a token,

while fewer than t cannot mount offline dictionary attacks or forge valid tokens. PASTA provides threshold availability for authentication and token generation, but no native user-password-update protocol. Later schemes strengthen other properties: PESTO [5] adds proactive and adaptive security through share refresh; PROTECT [61] addresses mobile users and perpetual leakage with identity-server key renewal; TSAPP [62] adds privacy goals such as anonymity and unlinkability; and DE-PbTA [43] introduces auxiliary-device protection against long-lived credential leakage.

Post-quantum variants retain the token-based architecture under lattice assumptions. PQ-PbTA [44] provides quantum-resistant threshold SSO with an updatable server private key, while LPbT-SSO [16] builds password-based threshold SSO from LWE. Their update mechanisms mainly concern server-side cryptographic keys rather than the user's password. PAS-TA-U [56] is closer to this thesis because it adds native password update to PASTA: the old password authorizes replacement by new IdP records protected under the new password. AugSSO [42] supports efficient password-update processing, identity-server share renewal, and popular-password collection.

These systems are not weaker than SPA; they protect a different architecture and object. PESTO, PROTECT, TSAPP, and DE-PbTA improve proactive security, leakage resilience, privacy, or device assistance; PQ-PbTA and LPbT-SSO change the underlying assumptions; and PAS-TA-U and AugSSO update IdP-side credentials or keys. None maintains login-server-specific authentication secrets at independent storage providers or recovers a provider that missed registration or an update. Moreover, all require relying-party support for IdP-issued tokens, whereas UpSPA and RoUpSPA preserve the ordinary login-server model.

3.4 Password-Hardening Services and Updatable Key Management

Password-hardening services (PHS) and updatable key-management systems (KMS) protect password-derived values, encryption keys, or key-wrapping state rather than SPA authentication records. Pythia [20] is a verifiable partially oblivious PRF ser-

vice that hardens password-derived values and supports efficient bulk rotation after compromise. Simple password-hardened encryption [47] uses one external service for rate limiting, stronger password-derived encryption keys, and key rotation, while Threshold PHE [14] distributes that service across several servers. These constructions protect PRF outputs or encryption keys, not per-login-server authentication state.

Updatable oblivious KMS schemes move this functionality to encrypted storage. UOKMS [39] protects KMS key material and identifiers, supports a threshold realization, and refreshes compact per-object headers or wrapping material without re-encrypting bulk data. PUOKMS [49] adds password protection and distributes assistance across identity and key servers. TPH-UOKM [41] combines threshold password hardening with updatable oblivious key management, supporting password update through cloud-side transformation tokens and proactive renewal of key-server shares. DPaSE [18] derives many independent symmetric keys from one password with distributed server assistance, but does not provide an SPA password-update or login-server-secret-update mechanism.

The distinction is the protected object. Pythia rotates a PRF output or hardening value; PHE and Threshold PHE protect password-derived encryption keys; and UOKMS, PUOKMS, and TPH-UOKM update keys, ciphertext headers, or wrapping objects for stored data. In UpSPA, the protected objects are master records, provider-specific identifiers, and login-server-specific authentication secrets. Its password update changes password-protected authentication state while preserving access to ordinary login servers, and its secret update rotates one login-server-specific authentication secret. The later PHS and KMS systems therefore offer strong distribution and update mechanisms, but not the SPA registration, authentication, and resilient update lifecycle.

3.5 Quorums, Public Logs, and Resilient State Synchronization

The resilient construction is also related to distributed storage, quorum systems, and append-only public logs. Classical robust shared memory [3] uses quorum-based

reads and writes so that later operations observe completed writes under its failure model. Byzantine quorum systems [50] strengthen the intersection requirement to preserve honest information despite malicious replicas. PBFT [17] goes further by establishing an agreed total order for a general replicated state machine.

RoUpSPA uses a lighter abstraction. A registration or update completes after q_{sp} providers validate, store, and acknowledge the same record. Two sets of size q_{sp} intersect in at least $2 \cdot q_{sp} - n_{sp}$ providers; with at most $t_{sp} - 1$ corrupted providers, the condition $2 \cdot q_{sp} - n_{sp} > t_{sp} - 1$ ensures that a completed write quorum and a later read or recovery quorum share at least one honest provider. Signed records, monotone counters, validation, highest-valid-state selection, and repair of stale providers complement this overlap. The honest intersection member preserves validated state and does not decrease freshness, while the selection rules prevent an arbitrary Byzantine response from becoming authoritative. The purpose is therefore recovery of small authenticated records, not general consensus.

Public logs offer an alternative recovery channel. Certificate Transparency [48] makes certificate issuance auditable, and CONIKS [51] applies a related idea to end-user key bindings and consistency. A $\mathcal{PBB}$ -based variant can similarly expose the freshest authenticated RoUpSPA record. With an end-to-end authenticated but otherwise untrusted $\mathcal{PBB}$, providers verify the user's signature, phase, and freshness counter themselves; malformed and duplicate records can be ignored, although scanning cost and flooding remain concerns. An authorization-enforcing $\mathcal{PBB}$ filters unauthorized writes before storage, reducing this burden but adding the board's admission policy and availability to the trust and liveness assumptions. Both variants support autonomous catch-up but introduce a global component for posting and recovery.

A blockchain gives stronger public ordering at greater cost. Bitcoin [54], the Bitcoin Backbone analysis [23], and Ouroboros [46] illustrate consensus-backed ledgers under different assumptions. A blockchain-based RoUpSPA variant could publish commitments, counters, phase labels, or pointers to encrypted recovery bundles, but would inherit finality delay, transaction fees, censorship risk, smart-contract assumptions, and public metadata leakage. Thus, robust memory and Byzantine

quorums motivate the core overlap condition; PBFT is stronger than necessary; transparency systems motivate public recovery histories; and $\mathcal{PBB}$ or blockchain variants exchange local provider synchronization for a global liveness and metadata boundary.

3.6 Overall Comparison and Positioning

The closest lineage is SPA, TSPA, and DiSPP. SPA separates the password from the login server by protecting high-entropy authentication state through one helper. TSPA distributes that state and removes the single-provider bottleneck, while DiSPP broadens the distributed framework. UpSPA adds native password update, login-server-specific secret update, and protection against false account creation and unauthorized replacement. RoUpSPA further allows registration and updates to complete through a valid q_{sp} quorum and lets unavailable providers recover the freshest valid state. PPSS systems instead recover secrets; threshold SSO issues IdP tokens; PHS and KMS schemes protect password-derived keys or encrypted-storage metadata; and quorum systems and public logs supply synchronization mechanisms rather than password authentication.

Table 3.6 uses the malicious-corruption setting of UpSPA and RoUpSPA as the full-comparison point. The triangle and dagger marks therefore denote a different security model, mixed guarantees within a grouped column, or a different protected object, not the absence of formal security. Threshold support also does not imply update support: TSPA/DiSPP and PPSS-based systems provide threshold authentication or recovery without native SPA updates; base PASTA-style SSO lacks user-password update; PAS-TA-U and AugSSO add IdP-side updates; Threshold PHE rotates keys without user-password update; and TPH-UOKM supports password and key updates for cloud-storage ciphertexts. Only UpSPA and RoUpSPA combine password update with login-server-specific secret rotation in the SPA setting, while only RoUpSPA additionally provides threshold-resilient registration, resilient updates, and recovery after a provider misses a completed operation.

Deployment remains another dividing line. UpSPA, RoUpSPA, TSPA/DiSPP,

and DE-PAKE can work with ordinary password-based login servers, while the exact requirements of SPA constructions vary. PPSS is not a login-server framework, threshold SSO requires relying parties to verify IdP-issued tokens, and PHS/KMS systems protect encrypted objects rather than implement the login interface. RoUpSPA therefore obtains resilience by synchronizing SPA storage-provider state, not by replacing login servers with an SSO or KMS layer.

Property \ Solution	Our Work RoUpSPA	Single-Password / Password-Based Authentication					SSO		PHS and KMS	
		UpSPA [34]	SPA [1, 11]	DE-PAKE [40]	TSPA/DISPP [35, 36]	PPSS/TOPPS/TPASS [4, 37, 38, 60]	PbTA/Threshold SSO [2, 5, 61, 62, 43, 44, 16]	Update-Oriented SSO [56, 42]	Pythia/PHE/TPHE [20, 47, 14]	TPH-UOKM [41]
Security under malicious corruption	✓	✓	△	△ [†]	△	△	△	✗	△	✗
Threshold security (t -of- n)	✓	✓	✗	✗	✓	✓	✓	✓	△	✓
Secret/key rotation supported	✓	✓	✗	△ [‡]	✗	✗	△	△	✓	✓
Password update supported	✓	✓	✗	✗	✗	✗	✗	✓	✗	✓
Prevents false account creation at storage provider	✓	✓	✗	✗	✗	✗	—	—	—	—
Threshold-resilient registration	✓	✗	✗	✗	✗	✗	✗	✗	—	—
Threshold-resilient secret/password update	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
Recovery after a provider misses registration/update	✓	✗	✗	✗	✗	—	✗	✗	✗	✗
Compatible with ordinary password-based login servers	✓	✓	△	✓	✓	—	✗	✗	—	—

Table 3.1: Comparison of existing works in terms of security, update support, resilience, recovery, and deployment compatibility.

Notes. — denotes that the property is not applicable. △ indicates that the property is supported only by some schemes in the grouped column, is formalized under a different security model, or applies to a different protected object. † indicates that DE-PAKE provides security against active network adversaries and compromise within its authentication model, assuming secure initialization. ‡ indicates that DE-PAKE permits replacement of a device-side key and may therefore change a hardened site credential, but does not provide a native password-update or threshold-distributed SPA-state-update protocol.

Part II

Updatable Single Password Authentication Variation

Chapter 4

UPSPA: UPDATABLE SINGLE PASSWORD AUTHENTICATION

4.1 System Design & Threat Model

UpSPA builds on the progression from SPA to TSPA. SPA allows a user to authenticate to multiple login servers with one remembered password while keeping the corresponding high-entropy authentication material at a separate storage provider. TSPA distributes this storage role among multiple providers and requires threshold-many of them during authentication. On top of this threshold architecture, UpSPA adds update support, allowing the user to replace the secret of one login-server account or change the remembered password without re-registering the remaining accounts [1, 35, 34].

This section fixes the system contract, architectural choices, and adversarial assumptions of UpSPA. It identifies the state owned by each party, separates persistent records from transient client values, and states the availability and completion conditions under which the security analysis is interpreted. The exact algorithms, message order, acceptance checks, and finalization steps of the five protocol phases are intentionally given only in Section 4.2, which is the authoritative protocol description.

The same model also provides the cryptographic and architectural baseline for RoUpSPA. The resilient construction inherits the password-protected identification state, the login-server-specific account state, the TOPRF interface, the login-server separation, the domain-bound verifier, and the main confidentiality and authorization goals established here, while changing the replica-completion and synchronization rules [34].

4.1.1 System Model

UpSPA is an update-enabled threshold single-password authentication protocol. It allows a user to access multiple ordinary login servers while remembering one low-entropy password **pwd**. Password-protected state is kept at independent storage providers, whereas each login server receives only high-entropy verification information specific to its own domain. This follows the deployment philosophy of SPA and TSPA: a login server should not receive **pwd** or a deterministic password-derived value that supports offline testing, and fewer than threshold-many storage providers should not recover the value protecting the user's long-term state [1, 35].

Parties and public parameters. The system contains users $\mathcal{U}_1, \dots, \mathcal{U}_{n_u}$, storage providers $\mathcal{SP}_1, \dots, \mathcal{SP}_{n_{sp}}$, and login servers $\mathcal{LS}_1, \dots, \mathcal{LS}_{n_{ls}}$. A user $\mathcal{U}_l$ has a public or guessable identity Uid_l and remembers **pwd**. Each storage provider stores encrypted protocol records and one share of a TOPRF key. A login server $\mathcal{LS}_j$ is identified by an authenticated canonical domain label ls_j and stores only the verification information associated with the account under Uid_l . It neither participates in the TOPRF nor communicates directly with the storage providers.

The public parameters include the security parameter λ , the number of storage providers n_{sp} , and the TOPRF threshold t_{sp} , where $1 \leq t_{sp} \leq n_{sp}$. Provider identities, provider endpoints, and the binding between $\mathcal{LS}_j$ and ls_j are public configuration.

Trusted client and transient state. The user executes UpSPA through a trusted client, such as a browser extension, mobile application, or local application. The client receives **pwd** when protected state must be opened, executes the TOPRF receiver algorithm, checks authenticated-decryption results, derives record identifiers, and applies the validation rules defined in Section 4.2. The password, TOPRF output, decrypted identification plaintext, signing key, identifier secret, account-encryption key, and login-server-specific secret are transient client values and need not be stored between executions.

The client should erase reconstructed secrets after use to the extent supported

by the implementation platform. The formal model does not treat best-effort memory clearing as a cryptographic primitive. A process dump, keylogger, malicious extension, or operating-system compromise that exposes these values is endpoint compromise and lies outside the theorem.

Persistent state. Each $\mathcal{SP}_i$ maintains one identification record for every successfully initialized user and one account record for every login server registered by that user. The identification record is indexed by Uid_l and contains the verification key svk , the identification ciphertext cid , and the provider’s TOPRF share k_i . A login-server-specific account record is indexed by $\text{SUid}_{l,i,j}$ and contains c_j , whose plaintext contains $\mathcal{R}^{\text{ls}_j}$ and the freshness counter ctr_{ac} .

The provider state machine distinguishes committed records from tentative records. Ordinary reads expose only committed records. Tentative records exist only for the serialized state-changing execution that created them and are resolved according to the exact finalization rules in Section 4.2. This distinction is protocol state, but it is not a separate user-visible call of $\mathcal{F}_{\text{UpSPA}}$.

A login server $\mathcal{LS}_j$ stores one account verifier under Uid_l . At the protocol level this verifier is $\text{vInfo}_j = \text{H}(\mathcal{R}^{\text{ls}_j} \parallel \text{ls}_j)$. A concrete website may process this high-entropy value through its ordinary password-verifier interface, but $\mathcal{LS}_j$ does not store pwd , $\text{H}(\text{pwd})$, $F_K(\text{pwd})$, $\mathcal{R}^{\text{sp}}$, K_0 , $\mathcal{R}^{\text{ls}_j}$, or any TOPRF share.

Canonical login-server identity and account ownership. The client derives protocol values from the canonical authenticated label ls_j . Redirect targets, aliases, internationalized names, and subdomains must be mapped to this label by public deployment rules before cryptographic processing. The protocol assumes that the trusted client obtains the intended label from an authenticated context and does not infer it from page content supplied by an untrusted website.

The login server remains responsible for its ordinary account-creation and recovery policy. Email confirmation, organizational onboarding, CAPTCHA challenges, abuse controls, and account recovery are outside UpSPA. Likewise, because the initial identification record is indexed by public Uid_l , a deployment should protect

first-time setup using a provider-side ownership or onboarding mechanism.

Protocol surface and authoritative description. UpSPA exposes the five user-visible calls `Setup`, `Register`, `Authenticate`, `SecretUpdate`, and `PwdUpdate`. Their purpose, exact inputs, messages, validation rules, and success conditions are defined only in Section 4.2. The present subsection does not define an alternative phase flow.

Availability and completion contract. Authentication requires at least t_{sp} valid provider responses. Setup, registration, secret update, and password update belong to the fully available baseline and require all n_{sp} providers at every provider boundary prescribed by the construction. A missing, invalid, or late required response prevents a successful phase output and is treated as denial of service rather than as an accepted authentication or update.

State-changing executions for the same Uid_i are serialized. A phase is completed only at the completion point defined in Section 4.2; an intermediate provider acknowledgement, login-server response, or tentative record is not itself a successful phase result. The security theorem concerns completed executions in which the honest client remains active through the required finalization. Interrupted setup, client loss during finalization, permanent provider failure, and repair of partially delivered state are outside this baseline.

Communication and privacy scope. Honest parties communicate through secure, server-authenticated channels in a synchronous model with bounded message delay and the clock assumptions required by the password-update timestamp. The adversary may delay, reorder, omit, or suppress messages and thereby cause an abort.

The system uses ordinary username-based accounts. It therefore does not claim anonymity or identity privacy against login servers. Its privacy goal is narrower: neither a login server nor fewer than t_{sp} storage providers should obtain a protocol-derived offline predicate for testing guesses of `pwd`, and authentication material for

one login server should not be reusable at another.

4.1.2 System Design

The design of UpSPA extends threshold single-password authentication with isolated account-secret update and global password update. The construction separates the cryptographic organization of the user's state from the provider-side committed/-tentative state machine. This subsection explains that organization and its purpose without repeating the phase algorithms of Section 4.2.

Deployment architecture. All threshold password processing occurs between the user and the storage providers. A login server does not evaluate the TOPRF, receive TOPRF shares, or communicate with a provider. It receives only a high-entropy, domain-specific authentication value. This value may be supported through a native verifier interface or through an ordinary password field, preserving the deployment model of SPA and TSPA.

The storage providers are intended to be administratively independent. The formal corruption bound is numerical, but its deployment meaning depends on compromise of one provider not automatically implying compromise of the remaining providers.

Two-layer protected state. The user state has a compact identification layer and a login-server-specific account layer. The identification plaintext contains the signing key $\mathbf{ssk}$, the identifier secret $\mathcal{R}^{sp}$, and the account-encryption key $\mathbf{K}_0$, and is protected inside $\mathbf{cid}$ under a TOPRF-derived value. The public verification key $\mathbf{svk}$ and the provider-specific TOPRF share $\mathbf{k}_i$ remain available to the corresponding provider.

For each login server, the account layer contains an independent high-entropy secret $\mathcal{R}^{ls_j}$ and counter $\mathbf{ctr}_{ac}$ protected by $\mathbf{K}_0$. The associated login-server verifier is bound to ls_j . Consequently, the remembered password protects the compact identification layer, while $\mathbf{K}_0$ protects the collection of account records. This separation allows a password change to avoid rewriting all login-server-specific ciphertexts and

allows one account secret to change without affecting unrelated accounts.

Record naming and domain separation. The provider-specific account identifier is derived from the hidden high-entropy value $\mathcal{R}^{sp}$ together with $\mathbf{ls}_j$ and the provider index. This prevents public values such as Uid_l and $\mathbf{ls}_j$ from directly determining the unused account namespace after successful setup. Including the domain label separates login servers, while including the provider index separates the storage namespaces of different providers.

The login-server verifier is derived independently for each domain from $\mathcal{R}^{\mathbf{ls}_j}$ and $\mathbf{ls}_j$. These bindings provide cross-domain separation in the authenticated-domain model. They do not by themselves protect against endpoint compromise, DNS compromise, deceptive domain presentation, session theft, or malicious account-recovery procedures.

Authorization and freshness. The account-encryption key, hidden identifier secret, current login-server verifier, signing key, authenticated encryption, account counter, and password-update timestamp serve distinct authorization or freshness roles. Possession of a public identity or a storage identifier alone is not sufficient to authorize a state change. The exact checks, signed field order, counter handling, and committed/tentative transitions are part of the construction and must be implemented exactly as written in Section 4.2.

Authenticated encryption prevents acceptance of a newly modified ciphertext under an honest key, but it does not establish recency by itself. Account counters distinguish current account state from older valid ciphertexts. Password-update freshness is supplied by the signed timestamp and the provider's state checks; the fully available UpSPA does not use a separate identification counter.

Update and storage cost. Because K_0 is protected once inside the compact identification record, changing pwd replaces only the identification-layer material at the providers and has cost $\mathcal{O}(n_{sp})$, independent of n_{ls} . Rotating one account secret affects only the replicated account record for the selected login server and also costs

$\mathcal{O}(n_{sp})$. Each provider stores one identification record per user and one account record per registered login server, giving $\mathcal{O}(1 + n_{ls})$ persistent storage per user at one provider and $\mathcal{O}(n_{sp}(1 + n_{ls}))$ across all providers.

Fully available baseline. Requiring all providers for state-changing operations is restrictive, but it gives every completed write a simple common-state invariant. The security analysis can therefore establish the core TOPRF, authenticated-encryption, signature, hash-binding, authorization, and domain-separation arguments without introducing replica status, recovery certificates, or provider synchronization. RoUpSPA reuses this cryptographic organization and addresses partial provider availability separately.

4.1.3 Threat Model

We model the adversary as a **static probabilistic polynomial-time malicious adversary** $\mathcal{A}$. The corruption set is fixed at the beginning of the complete execution and does not change between phases. Corruption gives $\mathcal{A}$ full control over the corrupted party and reveals its local state, including databases, TOPRF shares, ciphertexts, verification keys, tentative records, protocol messages, and retained randomness. Corrupted parties may deviate arbitrarily, return malformed or inconsistent responses, replay messages, refuse to participate, and coordinate their actions.

Corruption bounds and honest target. Let $C_{sp} \subseteq [n_{sp}]$ denote the corrupted storage providers, with $|C_{sp}| \leq t_{sp} - 1$. The adversary may additionally corrupt up to $n_{ls} - 1$ login servers and $n_u - 1$ users, leaving at least one honest target user $\mathcal{U}_l$ and one honest target login server $\mathcal{LS}_j$. The theorem protects the account of this honest pair. A corrupted target login server is outside the guarantee because it controls its account database, authentication decisions, sessions, and recovery mechanisms. Corrupted login servers for other domains may observe their own account material, but their joint view should not reveal an offline password-testing predicate or reusable authentication material for the honest $\mathcal{LS}_j$.

Communication, availability, and completed phases. Honest messages use the authenticated channels, delay bound, and clock assumptions of the system model. The adversary may schedule, delay, reorder, or drop messages and may prevent corrupted parties from responding. The acceptance and completion conditions are exactly those of Section 4.2. Executions that do not reach those conditions may fail or remain unresolved, but they do not create an ideal success.

The theorem is therefore a safety statement for completed phases rather than a liveness guarantee. Client crashes during finalization, permanent provider failures, partially created setup records, and operational repair of inconsistent state are outside the completed-phase theorem. Tentative records and finalization commands are real-world storage mechanics reproduced by the simulator; they are not recovery or availability calls of the ideal functionality.

Trusted endpoint. The honest user’s client device and runtime are trusted while executing UpSPA. Malware, keyloggers, memory scraping, malicious browser extensions, compromised operating systems, and malicious client code are outside the cryptographic model. If the endpoint reveals `pwd`, the decrypted identification state, `ssk`, $\mathcal{R}^{sp}$, K_0 , or $\mathcal{R}^{ls_j}$, the corresponding guarantees need not hold. Trusted software distribution, system isolation, secure input handling, and reliable domain presentation are deployment requirements rather than properties proved by UpSPA.

Cryptographic assumptions. The analysis assumes receiver-side TOPRF pseudorandomness against every coalition of fewer than t_{sp} storage providers and receiver-input obliviousness against malicious senders. Authenticated encryption provides IND – CPA confidentiality, ciphertext integrity, and the key-binding property required by the construction. The signature scheme is EUF – CMA-secure, and H is collision resistant for the identifier and verifier bindings used by the protocol.

Collision resistance alone does not imply that the output on an undisclosed high-entropy input is unpredictable. A claim that an unused $H(\mathcal{R}^{sp} \parallel ls_j \parallel i)$ or fresh $H(\mathcal{R}^{ls_j} \parallel ls_j)$ cannot be predicted before disclosure additionally requires hidden-

input output unpredictability for inputs containing the fresh λ -bit values $\mathcal{R}^{sp}$ or $\mathcal{R}^{ls_j}$. Without that requirement, the corresponding guarantee is limited to collision-resistant binding.

Password and protected-state confidentiality. Within the corruption bounds, the adversary’s view should not provide a locally verifiable predicate for testing password guesses offline. A corrupted login server receives a high-entropy verifier independent of pwd , while fewer than t_{sp} corrupted providers obtain only subthreshold TOPRF shares, encrypted identification and account records, public verification keys, and their protocol transcripts. Under the TOPRF and authenticated-encryption assumptions, this coalition cannot compute $F_K(\text{pwd}')$ for an arbitrary guess pwd' and test whether cid decrypts to the honest identification state.

The theorem does not make the adversary’s prior probability of guessing a human-memorable password negligible. It states that the protocol view adds no offline verification predicate beyond prior information and the online attempts allowed by the ideal execution. Rate limiting, account lockout, monitoring, and non-revealing error handling may restrict such attempts in a deployment, but they are not cryptographically enforced by UpSPA. Within the same bounds, $\mathcal{A}$ should not learn $\text{ssk} \parallel \mathcal{R}^{sp} \parallel K_0$ from cid or $\mathcal{R}^{ls_j} \parallel \text{ctr}_{ac}$ from c_j except through an authorized execution by an honest client.

Cross-site and domain-bounded security. A malicious login server may observe the verifier and authentication values for its own domain, but it should learn neither pwd nor a deterministic function of pwd , and its material should not be reusable at another login server. This guarantee is defined within the authenticated-domain model. It does not cover DNS compromise, a client interface that misrepresents the domain, endpoint malware, account-recovery abuse, session theft, or intentional export of reconstructed authentication material.

Metadata leakage. A corrupted provider learns that it was contacted, the identifier presented to it, message lengths, timing within the network model, and the

outcome visible to that provider. A login server learns the public user identity and whether a protocol call was attempted for its domain. The theorem does not provide anonymity, unlinkability, traffic-analysis resistance, or concealment of the number of registered accounts.

Record creation, integrity, and authorized transitions. After successful setup, the adversary should not be able to predict and occupy the provider-side namespace of a future login-server-specific account under the stated hidden-input assumption. This claim does not cover the initial identification record, which is indexed by public Uid_i and may be preempted before honest setup reaches a provider.

A malicious provider may return malformed, substituted, stale, tentative, or inconsistent records. Such behavior may cause an abort, but it should not make an honest client successfully accept an unauthorized record or state transition. An accepted transition must pass the exact current-state, decryption, verification, signature, freshness, and finalization checks assigned to that transition in Section 4.2. Replays of older honestly generated ciphertexts are governed by those freshness rules because ciphertext integrity alone does not establish recency.

Malicious users and login servers. Corrupted users may choose arbitrary passwords, keys, ciphertexts, identifiers, and update messages. Honest providers and the honest target login server apply the construction’s validation and uniqueness checks to those values. A malicious login server may reject its own execution and thereby cause denial of service, but it does not communicate directly with the providers and cannot independently commit provider state.

Remaining exclusions. The protocol does not provide protection for malicious account-recovery policies, post-login session security, permanent provider replacement, forgotten-password recovery, or repair after a client crash during finalization. These are limitations of the model and not implicit features of UpSPA.

4.1.4 Design Challenges of UpSPA

The design challenges below summarize why the state organization and trust boundaries are needed. They do not restate the phase algorithms; the corresponding protocol mechanisms are specified only in Section 4.2.

Challenge 1: keeping deterministic password information away from login servers. A login server must authenticate the user without storing `pwd`, $H(\text{pwd})$, or another deterministic password-derived value. UpSPA instead gives each domain independent high-entropy authentication material and confines password processing to the user-provider TOPRF interaction.

Challenge 2: preventing a subthreshold provider coalition from testing passwords offline. The password-protected identification layer must remain inaccessible to fewer than t_{sp} providers. Distributing the TOPRF key and using its output as the identification-encryption key separates unavoidable online guesses from an offline test based only on stored provider state.

Challenge 3: supporting two different updates without coupling all account records. A login-server-specific secret update should affect one account, whereas a password update should change the remembered password without rewriting every account ciphertext. The two-layer state design separates the compact identification state from the account records and keeps password-update cost independent of n_{ls} .

Challenge 4: preventing predictable record creation and cross-domain reuse. Provider-side account names must not be determined only by public identities, and authentication material for one domain must not serve another. Hidden $\mathcal{R}^{sp}$ -derived provider identifiers and domain-bound, independently sampled account secrets provide these separations after successful setup.

Challenge 5: authorizing replacement and distinguishing fresh state from stale state. Possession of a public identity or lookup key must not authorize an

account or identification replacement. UpSPA therefore combines protected identification state, authenticated encryption, a user signing key for identification replacement, domain-specific verifier checks, account counters, and signed timestamp freshness. Their exact use and field bindings are fixed by the construction.

Challenge 6: preserving one committed outcome under the fully available assumption. A multi-party state change must not become visible as current state before the construction reaches its completion condition. The committed/tentative provider state, serialized executions, idempotent exact-message finalization, and all-provider completion rule give the fully available protocol a single committed-state invariant for completed phases without claiming general crash recovery.

Challenge 7: retaining ordinary login-server deployment while stating the availability cost honestly. The login server should not execute threshold cryptography or communicate with the providers. Keeping its interface close to ordinary username–password authentication improves deployability, but the fully available baseline permits one refusing provider to block a state-changing phase. This is an explicit liveness limitation rather than a hidden security claim.

Contract inherited by RoUpSPA. RoUpSPA inherits the cryptographic state separation, password-security boundary, domain binding, and authorization principles above. Its design discussion should therefore address only the new obligations created by partial provider availability, including quorum completion, replica status, synchronization, and the separation of safety from liveness [34].

4.2 Construction

This section is the sole authoritative message-level description of the fully available UpSPA construction. The preceding system-design and threat-model section fixes only the architecture, trust assumptions, and security goals. The parties are a user $\mathcal{U}$ with public identifier Uid and password pwd , n_{sp} storage providers $\{\mathcal{SP}_i\}_{i \in [n_{sp}]}$, and a login server $\mathcal{LS}_j$ identified by the public domain label ls_j . The cryptographic

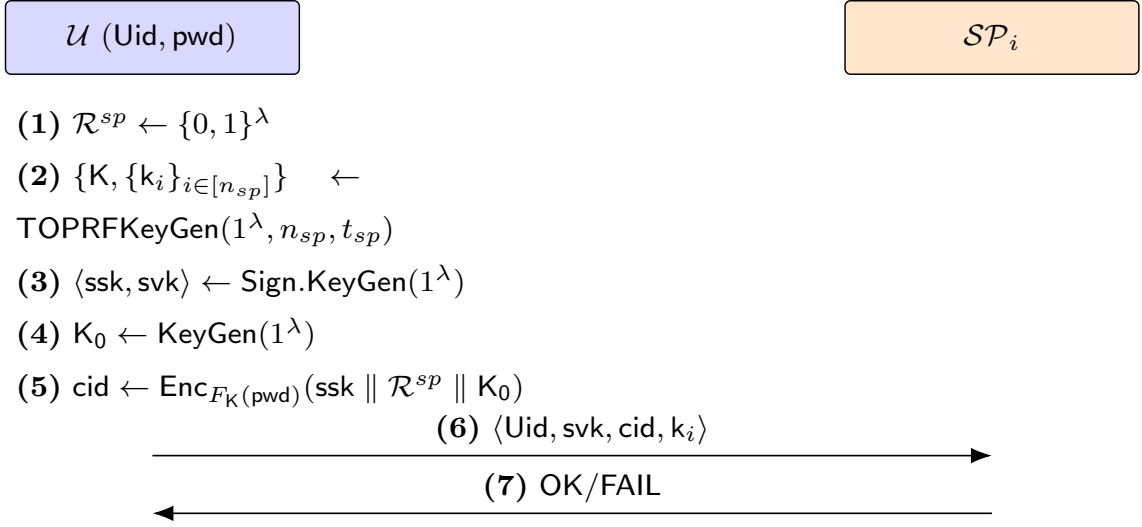
threshold t_{sp} determines how many storage-provider shares are needed in a TOPRF evaluation. By contrast, the fully available construction assumes that every storage provider is reachable when a state-changing phase is executed. Consequently, setup, registration, secret update, and password update complete only after the required response has been received from all n_{sp} providers. Authentication needs only t_{sp} valid provider responses.

The construction distinguishes a *pending* record from a *committed* record. A pending record is never returned by an ordinary read. In registration and secret update, the providers first acknowledge a pending write; the login server then decides whether its own operation succeeds; finally, $\mathcal{U}$ sends **Store** or **Discard**. Password update has no login-server step and therefore uses a pending write followed by **Store**. A provider applies a decision only to the pending tuple belonging to that execution and acknowledges the resulting commit or removal. These commands model the commit/abort decision used by the protocol; they do not claim a complete ACID transaction or a general crash-recovery mechanism.

Setup. The one-time setup phase Π_{Setup} is shown in Figure 4.1. First, $\mathcal{U}$ samples the high-entropy identifier secret $\mathcal{R}^{sp}$. It then generates a TOPRF key K , distributes the corresponding shares k_i , generates a signing key pair $\langle \text{ssk}, \text{svk} \rangle$, and samples the independent symmetric key K_0 . The value K_0 protects all login-server-specific records and is deliberately independent of **pwd**; this is what allows a later password update to avoid re-encrypting every site record.

The password-protected master ciphertext is $\text{cid} \leftarrow \text{Enc}_{F_K(\text{pwd})}(\text{ssk} \parallel \mathcal{R}^{sp} \parallel K_0)$. For each $i \in [n_{sp}]$, $\mathcal{U}$ sends $\langle \text{Uid}, \text{svk}, \text{cid}, k_i \rangle$ to $\mathcal{SP}_i$, using the share intended for that provider. The provider accepts only if no setup record is already stored under **Uid**. On acceptance it stores $(\text{Uid}, \text{svk}, \text{cid}, k_i)$ and returns **OK**; otherwise it returns **FAIL**. Setup succeeds only after all n_{sp} providers return **OK**. No **Store** or **Discard** step is used in setup because this is the initial creation of the master record and no login-server decision is involved.

Registration. In Π_{Register} , shown in Figure 4.2, $\mathcal{U}$ creates the state associated with one login server $\mathcal{LS}_j$. The user first asks every provider for the setup record. After all providers confirm its existence, $\mathcal{U}$ selects any t_{sp} valid providers for the

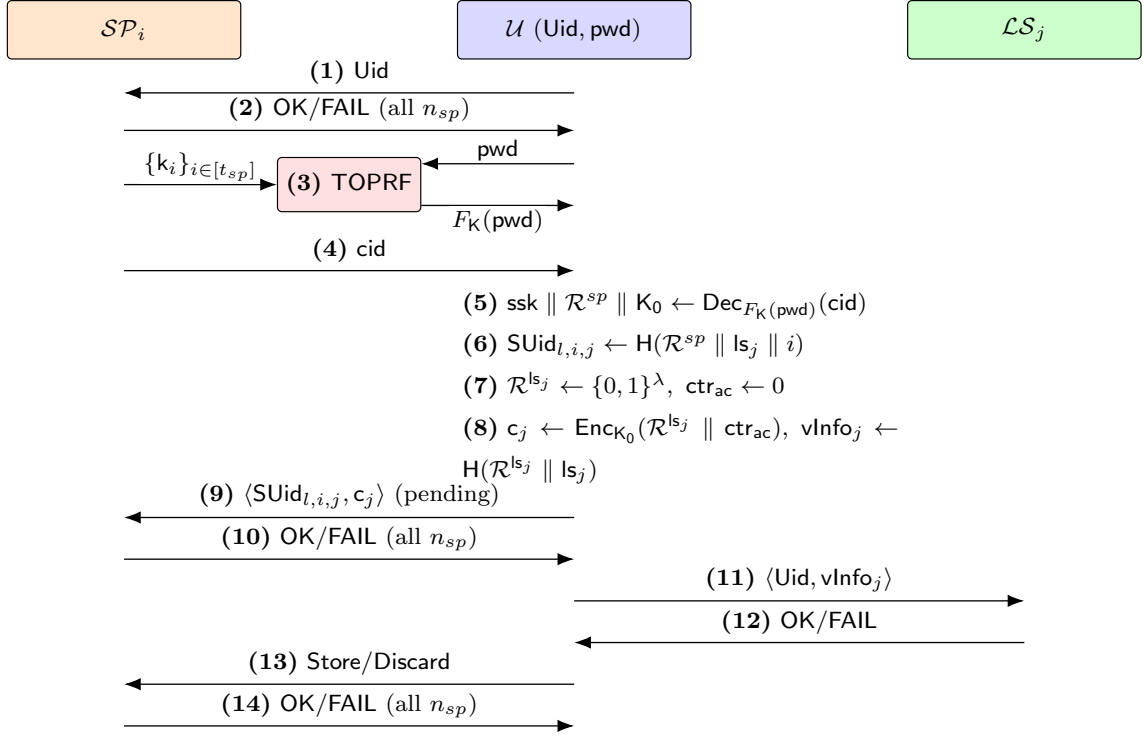
Figure 4.1: UpSPA setup phase Π_{Setup} .

TOPRF evaluation on pwd , obtains $F_K(\text{pwd})$, and decrypts one valid cid to recover $\text{ssk}, \mathcal{R}^{sp}, K_0$. Authenticated decryption rejects malformed or password-inconsistent master ciphertexts.

For each provider $\mathcal{SP}_i$, $\mathcal{U}$ derives a different routing identifier $\text{SUid}_{l,i,j} \leftarrow H(\mathcal{R}^{sp} \parallel \text{ls}_j \parallel i)$. The provider index is essential in this fully available version: records for the same login server are stored under different, independently derived identifiers at different providers. Next, $\mathcal{U}$ samples $\mathcal{R}^{\text{ls}_j} \leftarrow \{0, 1\}^\lambda$, initializes $\text{ctr}_{\text{ac}} = 0$, and computes $\text{c}_j \leftarrow \text{Enc}_{K_0}(\mathcal{R}^{\text{ls}_j} \parallel \text{ctr}_{\text{ac}})$, $\text{vInfo}_j \leftarrow H(\mathcal{R}^{\text{ls}_j} \parallel \text{ls}_j)$.

The ciphertext is the provider-side state, whereas vInfo_j is the high-entropy verifier presented to $\mathcal{LS}_j$. $\mathcal{U}$ sends $\langle \text{SUid}_{l,i,j}, \text{c}_j \rangle$ to the corresponding provider. If neither a committed nor a pending record exists under $\text{SUid}_{l,i,j}$, the provider stores the tuple as pending and returns **OK**; otherwise it returns **FAIL**. The user proceeds only after all n_{sp} pending acknowledgements arrive. If this condition is not met, $\mathcal{U}$ sends **Discard** for the accepted pending writes and aborts before contacting $\mathcal{LS}_j$.

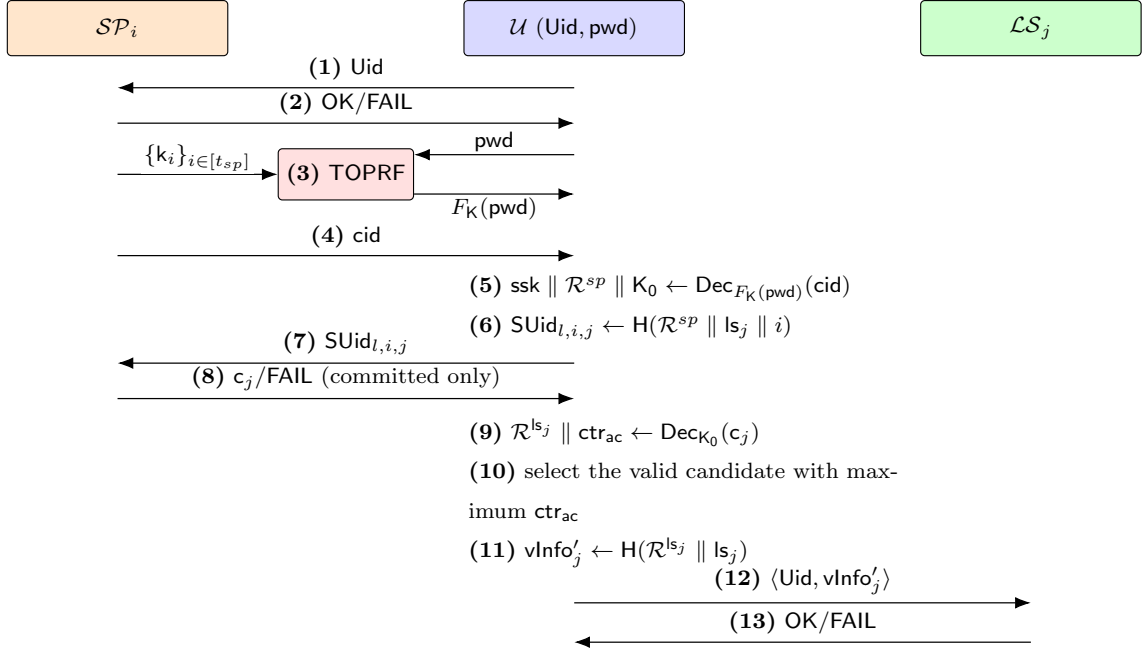
After a complete pending round, $\mathcal{U}$ sends $\langle \text{Uid}, \text{vInfo}_j \rangle$ to $\mathcal{LS}_j$. The login server stores the verifier only if no account already exists under Uid . An **OK** response causes $\mathcal{U}$ to send **Store**; a **FAIL** response causes it to send **Discard**. Each provider commits or removes only the matching pending tuple and returns a final acknowledgement.

Figure 4.2: UpSPA registration phase Π_{Register} .

Registration is reported as successful only when $\mathcal{LS}_j$ accepted and all n_{sp} providers acknowledged the **Store** decision.

Authentication. The authentication phase in Figure 4.3 is a read operation and therefore does not execute **Store** or **Discard**. The user obtains at least t_{sp} valid setup responses, runs the TOPRF on **pwd**, and recovers $\mathcal{R}^{sp}$ and K_0 from an authenticated decryption of **cid**. It then recomputes the provider-specific $\text{SUid}_{l,i,j}$ values and asks the corresponding providers for the site record. Providers return only committed records; pending and discarded values are not visible to this phase.

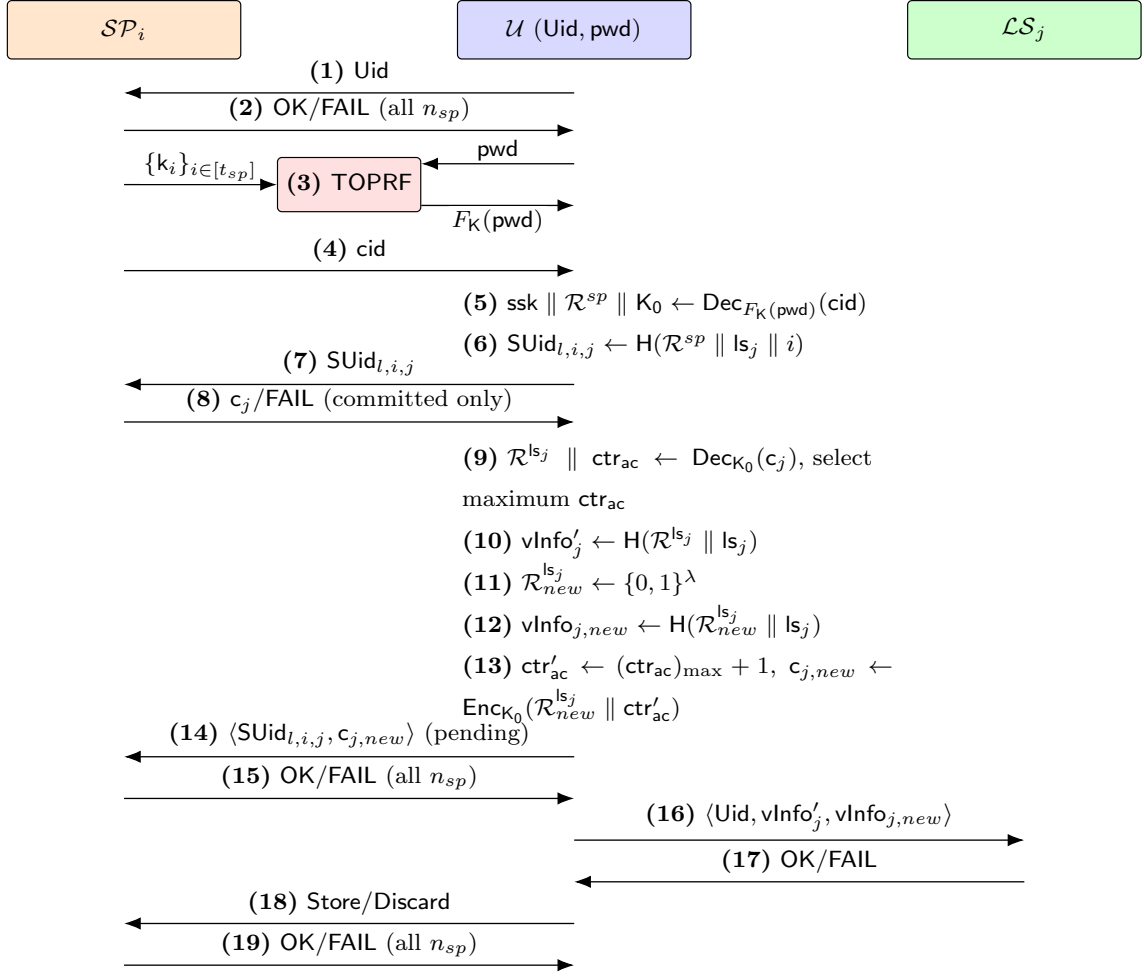
For every valid response, $\mathcal{U}$ decrypts c_j as $\mathcal{R}^{ls_j} \parallel \text{ctr}_{ac}$. If several authenticated candidates are available, it selects the one with the largest counter. The phase fails if fewer than t_{sp} valid provider responses are obtained or no candidate passes authenticated decryption. Finally, $\mathcal{U}$ computes $\text{vlnfo}'_j = H(\mathcal{R}^{ls_j} \parallel ls_j)$ and sends $\langle \text{Uid}, \text{vlnfo}'_j \rangle$ to $\mathcal{LS}_j$, which accepts exactly when the received verifier matches the verifier currently stored for that account.

Figure 4.3: UpSPA authentication phase $\Pi_{\text{Authenticate}}$.

Secret update. The phase in Figure 4.4 rotates the secret associated with one login server while leaving pwd unchanged. The user contacts all n_{sp} providers, uses any t_{sp} valid shares to reconstruct the password-derived key, decrypts cid , and recomputes the provider-specific identifiers. Each provider returns its current committed c_j . After authenticated decryption, $\mathcal{U}$ selects the candidate with the largest counter and obtains the current secret $\mathcal{R}^{\text{ls}_j}$ and counter ctr_{ac} .

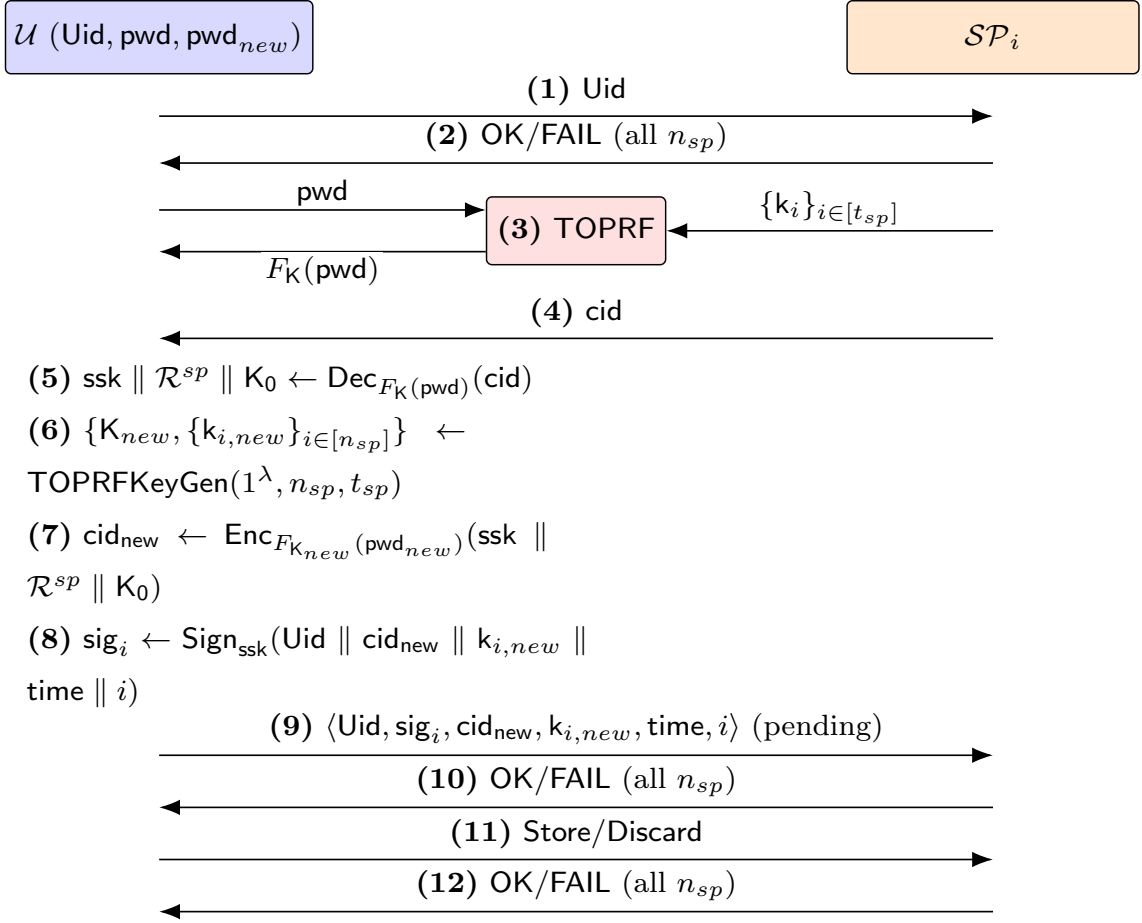
The user derives the current verifier $\text{vInfo}'_j = H(\mathcal{R}^{\text{ls}_j} \parallel \text{ls}_j)$, samples a fresh secret $\mathcal{R}_{\text{new}}^{\text{ls}_j}$, and forms $\text{vInfo}_{j,\text{new}} = H(\mathcal{R}_{\text{new}}^{\text{ls}_j} \parallel \text{ls}_j)$, $\text{c}_{j,\text{new}} = \text{Enc}_{K_0}(\mathcal{R}_{\text{new}}^{\text{ls}_j} \parallel \text{ctr}'_{\text{ac}})$, $(\text{ctr}_{\text{ac}})' = (\text{ctr}_{\text{ac}})_{\text{max}} + 1$. It sends $\langle \text{SUid}_{l,i,j}, \text{c}_{j,\text{new}} \rangle$ to every provider. A provider with an existing committed record under $\text{SUid}_{l,i,j}$ keeps the replacement pending and returns OK; the old committed value remains the value returned to authentication. If all n_{sp} pending acknowledgements do not arrive, $\mathcal{U}$ sends Discard and stops without asking $\mathcal{LS}_j$ to change its verifier.

After a complete pending round, $\mathcal{U}$ sends $\langle \text{Uid}, \text{vInfo}'_j, \text{vInfo}_{j,\text{new}} \rangle$ to $\mathcal{LS}_j$. The login server replaces the stored verifier only if vInfo'_j matches its current value. The user maps the login-server response to Store or Discard, and each provider applies

Figure 4.4: UpSPA secret-update phase $\Pi_{\text{SecretUpdate}}$.

that decision only to the matching tentative replacement. A successful secret update requires the login-server acceptance and a final OK from all n_{sp} providers.

Password update. The password-update phase in Figure 4.5 replaces pwd by pwd_{new} without modifying any login-server verifier or login-server-specific ciphertext. The user first reconstructs $\text{ssk}, \mathcal{R}^{sp}, K_0$ under the current password. It then generates a fresh TOPRF key K_{new} and provider shares $\{k_{i,new}\}_{i \in [n_{sp}]}$, and computes $\text{cid}_{new} \leftarrow \text{Enc}_{F_{K_{new}}}(\text{pwd}_{new})(\text{ssk} \parallel \mathcal{R}^{sp} \parallel K_0)$. The value K_0 is retained, so changing the single password does not require a linear scan over the user's login-server records. For each provider $\mathcal{SP}_i$, $\mathcal{U}$ authenticates the replacement by computing $\text{sig}_i \leftarrow \text{Sign}_{\text{ssk}}(\text{Uid} \parallel \text{cid}_{new} \parallel k_{i,new} \parallel \text{time} \parallel i)$.

Figure 4.5: UpSPA password-update phase $\Pi_{\text{PwdUpdate}}$.

The signed Uid binds the update to the intended master record; the provider index binds the fresh TOPRF share to its recipient; and time supplies the freshness value checked by that provider. The transmitted tuple is $\langle \text{Uid}, \text{sig}_i, \text{cid}_{\text{new}}, k_{i,\text{new}}, \text{time} \rangle$; thus, every value verified by $\mathcal{SP}_i$ is also explicitly present in the request.

After verifying the signature and freshness, each provider keeps $(\text{cid}_{\text{new}}, k_{i,\text{new}}, \text{time})$ as the pending replacement and returns OK. If every provider acknowledges, $\mathcal{U}$ sends Store; otherwise it sends Discard to providers that accepted the pending record and aborts. The phase succeeds only after all n_{sp} providers acknowledge the final Store. No login server participates.

Completed and interrupted executions. For registration and secret update,

the completed path is

Pending writes at all providers $\longrightarrow$ Login-server decision
 $\longrightarrow$ Store/Discard $\longrightarrow$ Final acknowledgements from all providers.

For password update, the login-server step is absent. These rules ensure that ordinary reads use only committed records and that the thesis does not count a state-changing operation as successful before its final acknowledgements. They do not assert global verifiable finality: an execution interrupted after the login server has changed state but before every provider processes the decision may leave the parties at different progress points. Handling such crashes is a deployment concern outside the fully available protocol model.

4.3 Security Analysis

4.3.1 Security Model and Proof Organization

We prove the security of the fully available version of UpSPA in the ideal/real-world model. In the real world, the adversary $\mathcal{A}$ interacts with the parties executing UpSPA. In the ideal world, a simulator Sim interacts with $\mathcal{F}_{\text{UpSPA}}$ on behalf of corrupted parties in the real world and produces the view of the corrupted parties. The goal is to prove that the two views are computationally indistinguishable.

Let $C \subseteq [n_{sp}]$ be the statically corrupted set of storage providers and let $H = [n_{sp}] \setminus C$. We allow $|C| \leq t_{sp} - 1$, at most $n_{ls} - 1$ corrupted login servers, and at most $n_u - 1$ corrupted users. Thus, at least one user and one login server remain honest, and every set of t_{sp} storage providers contains an honest provider.

Completed phases. The proof adopts without modification the committed/tentative state, validation rules, serialization requirement, and phase-completion points defined in Section 4.2. It does not define a second protocol flow. In particular, only committed records are visible to ordinary reads, a state-changing execution for one user is resolved before another such execution begins, and the simulator reports an ideal success only when the corresponding real execution reaches the construction's

final completion condition. A client crash or an interrupted finalization remains an availability event outside the completed-phase theorem.

Proof-only call sets. Let $\mathcal{K}$ be the collection of all full TOPRF keys honestly generated during the execution. Its members are the actual outputs of `TOPRFKeyGen` produced at setup and during password update. For every $k_i \in \mathcal{K}$, the corresponding shares are denoted $\{k_i\}_{i \in [n_{sp}]}$, and $q_K = |\mathcal{K}|$. This collection is only a way to organize the reduction. It introduces no counter, identifier, message field, or additional state into UpSPA.

Let $\mathcal{T}$ be the ordered collection of interactive TOPRF sessions in which an honest user is the receiver, and let $q_T = |\mathcal{T}|$. These sessions occur during registration, authentication, secret update, and the reconstruction step of password update. Setup and the creation of the new identification ciphertext in password update are not interactive sessions: the user has just generated the corresponding full TOPRF key and evaluates the PRF locally.

Let $\mathcal{E}_{id}$ be the ordered collection of honest encryption calls producing `cid` or `cidnew`, and let $\mathcal{E}_{sp}$ be the ordered collection of honest encryption calls producing `cj` or `cj,new`. Define $q_{id} = |\mathcal{E}_{id}|$, $q_{sp} = |\mathcal{E}_{sp}|$. The value q_{sp} counts encryption calls, not the number of copies of one ciphertext sent to different providers.

Theorem 4.3.1 (Security of the fully available UpSPA). Assume that TOPRF satisfies receiver-side PRF security against every coalition of fewer than t_{sp} storage providers and satisfies receiver-input obliviousness against malicious senders. Assume that authenticated encryption is IND – CPA secure, provides ciphertext integrity, and has the key-binding property required by the construction. Assume that `Sign` is EUF – CMA-secure and that `H` is collision resistant. Under the corruption bounds, authenticated channels, and completed-phase conditions above, for every non-uniform PPT adversary $\mathcal{A}$, there exists a PPT simulator `Sim` such that, for every auxiliary input `aux`,

$$\{\text{REAL}_{\text{UpSPA}, \mathcal{A}}(1^\lambda, \text{aux})\}_{\lambda \in \mathbb{N}} \stackrel{c}{\approx} \{\text{IDEAL}_{\mathcal{F}_{\text{UpSPA}}, \text{Sim}}(1^\lambda, \text{aux})\}_{\lambda \in \mathbb{N}}.$$

Proof overview. We first define `Sim` phase by phase. We then define the signature,

hash, and authenticated-encryption bad events outside the confidentiality games. The game sequence has the same structure as the earlier proof: G_0 is the real execution; G_1 replaces TOPRF values and honest-receiver transcripts; G_2 replaces the plaintexts inside identification ciphertexts; G_3 replaces the plaintexts inside account ciphertexts; and G_4 is the ideal execution with $\mathcal{F}_{\text{UpSPA}}$ and Sim . Every game has a corresponding transition lemma and proof.

4.3.2 Simulation Behavior

Local pending and committed state. The functionality $\mathcal{F}_{\text{UpSPA}}$ represents only the logical result of a completed phase and does not maintain pending records, provider acknowledgements, or the **Store/Discard** state. These values are maintained locally by Sim while it simulates the honest parties in the real protocol. A pending record is not returned during authentication. Until a matching **Store** is completed, the previous committed record remains active. A matching **Discard** removes only the pending record. Therefore, Sim sends the final **OK** to $\mathcal{F}_{\text{UpSPA}}$ only after the corresponding real execution reaches its protocol-defined completion point.

Setup Phase: Sim :

1. Upon receiving $\langle \text{Setup}, \text{Uid}_l^o \rangle$ from $\mathcal{F}_{\text{UpSPA}}$ in the ideal world, Sim :
 - (a) Generates a PRF key $\tilde{K}$ and n_{sp} key shares of $\tilde{K}$ as $\{\tilde{k}_i\}_{i \in [n_{sp}]}$ by running the TOPRF key-generation function as $\{\tilde{K}, \{\tilde{k}_i\}_{i \in [n_{sp}]}\} \leftarrow \text{TOPRFKeyGen}(1^\lambda, n_{sp}, t_{sp})$, and generates a signing key and a verification key as $\tilde{\text{ssk}}, \tilde{\text{svk}} \leftarrow \text{Sign.KeyGen}(1^\lambda)$.
 - (b) Assigns $\tilde{\sigma} = \tilde{\text{svk}}$, samples $\widetilde{\mathcal{R}^{sp}} \leftarrow \{0, 1\}^\lambda$ and $\widetilde{K_0} \leftarrow \{0, 1\}^\lambda$, and samples the simulated identification-encryption key $R(\text{pwd}) \leftarrow \{0, 1\}^\lambda$.
 - (c) Computes a simulated identification ciphertext of the same length as in the honest protocol as $\tilde{\text{cid}} \leftarrow \text{Enc}_{R(\text{pwd})}(\{0, 1\}^{|\text{ssk}| + |\mathcal{R}^{sp}| + |K_0|})$, and stores that $\tilde{\text{cid}}$ logically represents $\tilde{\text{ssk}} \parallel \widetilde{\mathcal{R}^{sp}} \parallel \widetilde{K_0}$.
 - (d) Sends one tuple $\langle \text{Uid}_l^o, (\tilde{\sigma}, \tilde{\text{cid}}, \tilde{k}_i) \rangle$ to each of the $t_{sp} - 1$ corrupted $\mathcal{SP}_i$ simulated by $\mathcal{A}$. If any corrupted $\mathcal{SP}_i$ returns **FAIL**, Sim sends **FAIL**.

to $\mathcal{F}_{\text{UpSPA}}$. Otherwise, after the setup tuple has been accepted by all n_{sp} storage providers, **Sim** stores the committed identification tuple and sends OK to $\mathcal{F}_{\text{UpSPA}}$.

2. Upon receiving a setup request for a corrupted user Uid_l^c from $\mathcal{A}$, **Sim** receives $\langle \text{Uid}_l^c, (\sigma_l^c, \text{cid}^c, k_i^c) \rangle$ for each of the $n_{sp} - t_{sp} + 1$ honest $\mathcal{SP}_i$ that it simulates. If a setup record for Uid_l^c already exists at any such honest $\mathcal{SP}_i$, **Sim** sends FAIL to $\mathcal{A}$ and does not complete the ideal setup. Otherwise, **Sim** sends $\langle \text{Setup}, \text{Uid}_l^c \rangle$ to $\mathcal{F}_{\text{UpSPA}}$. If $\mathcal{F}_{\text{UpSPA}}$ returns OK, **Sim** stores $\langle \text{Uid}_l^c, \sigma_l^c, \text{cid}^c, \{k_i^c\}_{i \in [n_{sp} - t_{sp} + 1]} \rangle$ for the corresponding honest providers and sends OK to $\mathcal{A}$. Otherwise, it sends FAIL to $\mathcal{A}$.

Registration phase:

1. **Sim Behavior for Uid_l^o .** The registration request for the honest user Uid_l^o is carried out with the honest $\mathcal{LS}$ or any one of the malicious login servers corrupted by $\mathcal{A}$. The only difference in the simulation is whether the simulated verification information is sent to $\mathcal{A}$.

Sim receives $\langle \text{Register}, \text{Uid}_l^o \rangle$ from $\mathcal{F}_{\text{UpSPA}}$ and sends Uid_l^o to $\mathcal{A}$. Then, **Sim**:

- (a) If any required corrupted $\mathcal{SP}_i$ sends FAIL, **Sim** sends FAIL to $\mathcal{F}_{\text{UpSPA}}$ and aborts the registration in the real world. Otherwise, **Sim** performs the TOPRF interaction using $\tilde{\text{pwd}}$, the shares of the honest storage providers that it simulates, and the partial evaluations returned by the corrupted storage providers. It runs the protocol-defined TOPRF reconstruction algorithm on these contributions. If fewer than t_{sp} valid contributions are available, or the reconstruction detects a malformed or inconsistent contribution and outputs $\perp$, **Sim** sends FAIL to $\mathcal{A}$ and $\mathcal{F}_{\text{UpSPA}}$. A successful reconstruction is represented by the already sampled value $R(\text{pwd})$.
- (b) Receives cid^c from each corrupted $\mathcal{SP}_i$ and checks that every required response is the current committed $\tilde{\text{cid}}$ stored for Uid_l^o . If this check fails, **Sim** sends FAIL to $\mathcal{A}$ and $\mathcal{F}_{\text{UpSPA}}$ and aborts the registration.

- (c) Receives $\langle \text{Register}, \text{SUid}_{l,i,j} \rangle$ from $\mathcal{F}_{\text{UpSPA}}$ for each honest $\mathcal{SP}_i$ that it simulates. For each corrupted $\mathcal{SP}_i$, it computes the corresponding simulated provider-specific identifier as $\tilde{\text{SUid}}_{i,j}^c \leftarrow \text{H}(\widetilde{\mathcal{R}^{sp}} \parallel \text{ls}_j \parallel i)$.
 - (d) Samples $\tilde{\mathcal{R}}^{\text{ls}_j} \leftarrow \{0,1\}^\lambda$, sets $\text{ctr}_{\text{ac}} \leftarrow 0$, and computes $\text{vlnfo}_j \leftarrow \text{H}(\tilde{\mathcal{R}}^{\text{ls}_j} \parallel \text{ls}_j)$. It then computes a simulated account ciphertext of the same length as in the honest protocol as $\tilde{c}_j \leftarrow \text{Enc}_{\tilde{\text{K}}_0}(\{0,1\}^{|\mathcal{R}^{\text{ls}_j}| + |\text{ctr}_{\text{ac}}|})$, and stores that $\tilde{c}_j$ logically represents $\tilde{\mathcal{R}}^{\text{ls}_j} \parallel 0$.
 - (e) Sends $\langle \tilde{\text{SUid}}_{i,j}^c, \tilde{c}_j \rangle$ to each corrupted $\mathcal{SP}_i$ and records the corresponding tuples for the honest $\mathcal{SP}_i$ as pending. If any required provider returns FAIL, Sim sends Discard to the providers that accepted the pending tuple, removes the pending tuple from its local state, sends FAIL to $\mathcal{F}_{\text{UpSPA}}$, and aborts the registration.
 - (f) If all n_{sp} providers acknowledge the pending tuple, Sim obtains the login-server decision. If ls_j is corrupted, Sim sends $\langle \text{Uid}_{l,j}^o, \text{vlnfo}_j \rangle$ to $\mathcal{A}$ and uses the returned OK/FAIL response. If ls_j is honest, Sim obtains the corresponding login-server response from $\mathcal{F}_{\text{UpSPA}}$ without yet reporting the registration phase as completed.
 - (g) On a login-server FAIL response, Sim sends Discard to all providers, removes the pending account tuple, leaves any previous committed state unchanged, sends FAIL to $\mathcal{F}_{\text{UpSPA}}$, and aborts in both worlds. On a login-server OK response, Sim sends Store to all providers. Only after all n_{sp} providers return their final OK does Sim move the account tuple from pending to committed state, store the corresponding vlnfo_j , and send the final OK to $\mathcal{F}_{\text{UpSPA}}$. If any required final acknowledgement is missing or is FAIL, Sim does not report an ideal registration success.
2. **Sim Behavior for Uid_l^c .** $\mathcal{A}$, as a malicious user with Uid_l^c , requests to register either with an honest $\mathcal{LS}$ with ls_j^o simulated by Sim or with a malicious $\mathcal{LS}$ with ls_j^c corrupted by $\mathcal{A}$.

Sim receives Uid_l^c and the selected ls_j from $\mathcal{A}$. If the setup record of Uid_l^c does

not exist, **Sim** aborts. Otherwise, **Sim**:

- (a) Executes TOPRF with $\mathcal{A}$ using the k_i^c values sent during setup as $\mathcal{A}[F_{K^c}(\text{pwd}^c)] \leftarrow \text{TOPRF}\left(\mathcal{A}[\text{pwd}^c], \{\text{Sim}[k_i^c]\}_{i \in [n_{sp}-t_{sp}+1]} \cup \{\mathcal{A}[k_i^c]\}_{i \in [t_{sp}-1]}\right)$, and sends the corresponding committed cid^c values of the honest $\mathcal{SP}_i$ to $\mathcal{A}$.
- (b) Receives $\langle \text{SUid}_{i,j}^c, c_j^c \rangle$ for each honest $\mathcal{SP}_i$. Each simulated honest provider checks that neither a committed nor a pending record exists under the received $\text{SUid}_{i,j}^c$. If any check fails, **Sim** sends **FAIL** to $\mathcal{A}$ and $\mathcal{F}_{\text{UpSPA}}$. Otherwise, it stores the received tuples as pending and returns **OK** to $\mathcal{A}$ on behalf of the honest providers.
- (c) If ls_j is honest, **Sim** receives $\langle \text{Uid}_{l,j}^c, \text{vInfo}_j^c \rangle$ from $\mathcal{A}$ and applies the same account-existence check as the honest login server. If ls_j is corrupted, the login-server decision is the response selected by $\mathcal{A}$.
- (d) If the login-server decision is **FAIL**, **Sim** applies **Discard** to the pending tuples of the honest providers, sends **FAIL** to $\mathcal{A}$, and leaves the previous committed state unchanged. If the decision is **OK**, **Sim** waits for the matching **Store** command and the required final acknowledgements. It commits the pending tuples only after the same all-provider completion point as in the real protocol. When ls_j is honest, it also stores vInfo_j^c as the login information of $\text{Uid}_{l,j}^c$. At that point, **Sim** sends the corresponding $\langle \text{Register}, \text{Uid}_l^c, \text{ls}_j \rangle$ request to $\mathcal{F}_{\text{UpSPA}}$ for either an honest or a corrupted login server and relays the matching final **OK/FAIL** response to $\mathcal{A}$.

Authentication phase:

1. **Sim Behavior for Uid_l^o .** The authentication request for the honest user Uid_l^o could be to the honest $\mathcal{LS}$ or any one of the malicious login servers corrupted by $\mathcal{A}$. The only difference between these two sub-cases is whether the simulated verification information is sent to $\mathcal{A}$.

Sim receives $\langle \text{Authenticate}, \text{Uid}_l^o \rangle$ from $\mathcal{F}_{\text{UpSPA}}$ and sends Uid_l^o to $\mathcal{A}$. Then, **Sim**:

- (a) Collects the storage-provider responses used in the TOPRF execution. A FAIL response from one corrupted $\mathcal{SP}_i$ is ignored when at least t_{sp} valid contributions remain. Sim performs the TOPRF interaction using pwd , its honest contributions, and the valid partial evaluations returned by $\mathcal{A}$, and runs the protocol-defined reconstruction algorithm. If fewer than t_{sp} valid contributions remain or reconstruction outputs $\perp$, Sim sends FAIL to $\mathcal{A}$ and $\mathcal{F}_{\text{UpSPA}}$. A successful reconstruction uses the same stored value $R(\text{pwd})$.
- (b) Receives the identification-ciphertext responses. A response from a corrupted $\mathcal{SP}_i$ is valid only if it is the current committed $\tilde{\text{cid}}$ stored for Uid_i^o . Invalid or missing corrupted responses are ignored. If fewer than t_{sp} valid committed identification responses remain, Sim sends FAIL to $\mathcal{A}$ and $\mathcal{F}_{\text{UpSPA}}$ and aborts the authentication.
- (c) Receives $\langle \text{Authenticate}, \text{SUid}_{l,i,j} \rangle$ from $\mathcal{F}_{\text{UpSPA}}$ for each honest $\mathcal{SP}_i$ and computes $\tilde{\text{SUid}}_{i,j}^c \leftarrow H(\widetilde{\mathcal{R}}^{sp} \parallel \text{ls}_j \parallel i)$ for each corrupted $\mathcal{SP}_i$. It sends the corresponding $\tilde{\text{SUid}}_{i,j}^c$ values to $\mathcal{A}$ and receives a candidate c_j^c or FAIL from each corrupted $\mathcal{SP}_i$. A returned ciphertext is used only when it matches a current committed account record in Sim's database.
- (d) Combines the committed account records returned by the honest providers with the valid committed records returned by the corrupted providers. Pending records and malformed ciphertexts are ignored. If fewer than t_{sp} valid committed account responses remain, Sim sends FAIL to $\mathcal{A}$ and $\mathcal{F}_{\text{UpSPA}}$. Otherwise, it decrypts the known simulated records according to the logical values stored in its database and selects the valid record having the largest counter ctr_{ac} . It then computes $\tilde{\text{vInfo}}_j \leftarrow H(\tilde{\mathcal{R}}^{\text{ls}_j} \parallel \text{ls}_j)$ from the selected committed account state.
- (e) If ls_j is honest, Sim returns the corresponding OK/FAIL authentication result to $\mathcal{F}_{\text{UpSPA}}$. If ls_j is corrupted, Sim sends $\langle \text{Uid}_{l,i,j}^o, \tilde{\text{vInfo}}_j \rangle$ to $\mathcal{A}$ and relays the returned OK/FAIL result to $\mathcal{F}_{\text{UpSPA}}$. Authentication creates no pending record and uses no Store/Discard command.

2. **Sim Behavior for Uid_l^c .** $\mathcal{A}$, as a malicious user with Uid_l^c , requests to authenticate with either an honest $\mathcal{LS}$ with ls_j^o simulated by Sim or a malicious $\mathcal{LS}$ with ls_j^c corrupted by $\mathcal{A}$.

Sim receives Uid_l^c and the selected ls_j from $\mathcal{A}$. If the setup record of Uid_l^c does not exist, Sim aborts. Otherwise, Sim :

- (a) Executes TOPRF with $\mathcal{A}$ using the k_i^c values sent during setup as $\mathcal{A}[F_{\text{K}^c}(\text{pwd}^c)] \leftarrow \text{TOPRF}\left(\mathcal{A}[\text{pwd}^c], \{\text{Sim}[\text{k}_i^c]\}_{i \in [n_{sp}-t_{sp}+1]} \cup \{\mathcal{A}[\text{k}_i^c]\}_{i \in [t_{sp}-1]}\right)$, and sends the corresponding committed cid^c values of the honest providers to $\mathcal{A}$.
- (b) Receives $\{\text{SUid}_{i,j}^c\}_{i \in [n_{sp}-t_{sp}+1]}$ for the honest $\mathcal{SP}_i$. If a received identifier matches a committed account record in the corresponding simulated provider, Sim sends the associated c_j^c to $\mathcal{A}$; otherwise, it sends FAIL for that provider.
- (c) If ls_j is honest, Sim receives $\langle \text{Uid}_{l,j}^c, \text{vInfo}_j^c \rangle$ from $\mathcal{A}$ and sends the corresponding $\langle \text{Authenticate}, \text{Uid}_l^c, \text{ls}_j \rangle$ request to $\mathcal{F}_{\text{UpSPA}}$. It relays the resulting OK/FAIL response to $\mathcal{A}$. If ls_j is corrupted, $\mathcal{A}$ determines the login-server response; Sim invokes the corresponding ideal authentication after the provider interaction and relays that response to $\mathcal{F}_{\text{UpSPA}}$ without performing an additional honest login-server check.

Secret update phase:

1. **Sim Behavior for Uid_l^o .** The secret update request for the honest user Uid_l^o could be to the honest login server ls_j^o or any one of the malicious login servers ls_j^c corrupted by $\mathcal{A}$.

Sim receives $\langle \text{SecretUpdate}, \text{Uid}_l^o \rangle$ from $\mathcal{F}_{\text{UpSPA}}$ and sends Uid_l^o to $\mathcal{A}$. Then, Sim :

- (a) If any required corrupted $\mathcal{SP}_i$ sends FAIL or the committed setup record of Uid_l^o does not exist, Sim sends FAIL to $\mathcal{F}_{\text{UpSPA}}$ and aborts the secret update. Otherwise, it performs the TOPRF interaction using $\tilde{\text{pwd}}$, its

honest contributions, and the partial evaluations returned by $\mathcal{A}$. It runs the protocol-defined reconstruction algorithm. If reconstruction outputs $\perp$, Sim sends FAIL to $\mathcal{A}$ and $\mathcal{F}_{\text{UpSPA}}$. A successful reconstruction uses the same current value $R(\text{pwd})$ used in registration and authentication.

- (b) Receives cid^c from each corrupted $\mathcal{SP}_i$ and checks that every required response is the current committed $\tilde{\text{cid}}$. If a required response is inconsistent, Sim sends FAIL to $\mathcal{A}$ and $\mathcal{F}_{\text{UpSPA}}$ and aborts the secret update.
- (c) Receives $\langle \text{SecretUpdate}, \text{SUid}_{l,i,j} \rangle$ from $\mathcal{F}_{\text{UpSPA}}$ for each honest $\mathcal{SP}_i$ and computes $\tilde{\text{SUid}}_{i,j}^c \leftarrow H(\widetilde{\mathcal{R}}^{sp} \parallel \text{ls}_j \parallel i)$ for each corrupted $\mathcal{SP}_i$. It sends these identifiers to $\mathcal{A}$ and receives the corresponding candidate c_j^c records. A returned ciphertext is used only when it matches a committed account record in Sim's database.
- (d) Decrypts the valid committed records according to the logical account values in its database and selects the current record with the largest counter. If no valid committed record is recovered or any required provider response is missing, Sim sends FAIL to $\mathcal{A}$ and $\mathcal{F}_{\text{UpSPA}}$. Otherwise, it obtains $\tilde{\mathcal{R}}^{\text{ls}_j} \parallel \text{ctr}_{\text{ac}}$ and computes $\text{vlnfo}'_j \leftarrow H(\tilde{\mathcal{R}}^{\text{ls}_j} \parallel \text{ls}_j)$.
- (e) Samples $\tilde{\mathcal{R}}_{\text{new}}^{\text{ls}_j} \leftarrow \{0, 1\}^\lambda$, computes $\text{vlnfo}_{j,\text{new}} \leftarrow H(\tilde{\mathcal{R}}_{\text{new}}^{\text{ls}_j} \parallel \text{ls}_j)$, and computes a simulated replacement ciphertext of the correct length as $\tilde{\text{c}}_{j,\text{new}} \leftarrow \text{Enc}_{\widetilde{\text{K}}_0}(\{0, 1\}^{|\mathcal{R}^{\text{ls}_j}| + |\text{ctr}_{\text{ac}}|})$. It stores that $\tilde{\text{c}}_{j,\text{new}}$ logically represents $\tilde{\mathcal{R}}_{\text{new}}^{\text{ls}_j} \parallel (\text{ctr}_{\text{ac}} + 1)$.
- (f) Sends $\langle \tilde{\text{SUid}}_{i,j}^c, \tilde{\text{c}}_{j,\text{new}} \rangle$ to each corrupted $\mathcal{SP}_i$ and records the corresponding replacement tuple at each honest $\mathcal{SP}_i$ as pending while retaining the old committed account record. If any required provider rejects the pending replacement, Sim sends Discard to the providers that accepted it, removes the pending tuple, leaves the old committed account record active, sends FAIL to $\mathcal{F}_{\text{UpSPA}}$, and aborts the secret update.
- (g) If all n_{sp} providers acknowledge the pending replacement, Sim obtains the login-server decision. If ls_j is corrupted, it sends $\langle \text{Uid}_{l,j}^o, \text{vlnfo}'_j, \text{vlnfo}_{j,\text{new}} \rangle$ to $\mathcal{A}$ and uses the returned OK/FAIL response. If ls_j is honest, it obtains

the corresponding response from $\mathcal{F}_{\text{UpSPA}}$ without yet reporting the secret update as completed.

- (h) On a login-server **FAIL** response, **Sim** sends **Discard** to all providers, removes the pending replacement, leaves the old committed account state unchanged, and sends **FAIL** to $\mathcal{F}_{\text{UpSPA}}$. On a login-server **OK** response, **Sim** sends **Store** to all providers. Only after all n_{sp} final acknowledgements are **OK** does it replace the committed account tuple by the pending tuple, remove the pending entry, store $\mathbf{vInfo}_{j,new}$ as the current verifier, and send the final **OK** to $\mathcal{F}_{\text{UpSPA}}$. If the required final acknowledgements are not obtained, **Sim** does not report an ideal secret-update success.

2. **Sim Behavior for Uid_l^c .** $\mathcal{A}$, as a malicious user with Uid_l^c , requests the secret update with either an honest $\mathcal{LS}$ with ls_j^o simulated by **Sim** or a malicious $\mathcal{LS}$ with ls_j^c corrupted by $\mathcal{A}$.

Sim receives Uid_l^c and the selected ls_j from $\mathcal{A}$. If the setup record of Uid_l^c does not exist, **Sim** aborts. Otherwise, **Sim**:

- (a) Executes TOPRF with $\mathcal{A}$ using the $\mathbf{k}_i^c$ values sent during setup as $\mathcal{A}[F_{K^c}(\text{pwd}^c)] \leftarrow \text{TOPRF}\left(\mathcal{A}[\text{pwd}^c], \{\text{Sim}[\mathbf{k}_i^c]\}_{i \in [n_{sp}-t_{sp}+1]} \cup \{\mathcal{A}[\mathbf{k}_i^c]\}_{i \in [t_{sp}-1]}\right)$, and sends the corresponding committed cid^c values of the honest providers to $\mathcal{A}$.
- (b) Receives $\text{SUid}_{i,j}^c$ for each honest $\mathcal{SP}_i$. If the identifier matches the corresponding committed account entry, **Sim** sends $\mathbf{c}_j^c$ to $\mathcal{A}$; otherwise, it sends **FAIL** and aborts the secret update.
- (c) Receives $\langle \text{SUid}_{i,j}^c, \mathbf{c}_{j,new}^c \rangle$ for each honest $\mathcal{SP}_i$. Each simulated honest provider checks that the identifier refers to its committed account record and stores the replacement only as pending. If any required pending write fails, **Sim** applies **Discard** to the accepted pending tuples, sends **FAIL** to $\mathcal{A}$ and $\mathcal{F}_{\text{UpSPA}}$, and leaves the committed account record unchanged.
- (d) If ls_j is honest, **Sim** receives $\langle \text{Uid}_{l,j}^c, \mathbf{vInfo}_j^c, \mathbf{vInfo}_{j,new}^c \rangle$ from $\mathcal{A}$ and applies the same login-server check, which accepts only when $\mathbf{vInfo}_j^c$ matches the

current login information of $\text{Uid}_{l,j}^c$. If ls_j is corrupted, the login-server decision is selected by $\mathcal{A}$.

- (e) On a login-server **FAIL** response, **Sim** applies **Discard** and sends **FAIL** to $\mathcal{A}$. On a login-server **OK** response, **Sim** waits for the matching **Store** command and the final acknowledgements. It replaces the committed account tuple only after the same all-provider completion point as in the real protocol. When ls_j is honest, it also replaces the stored vInfo_j^c by $\text{vInfo}_{j,\text{new}}^c$. At that point, it sends the corresponding $\langle \text{SecretUpdate}, \text{Uid}_l^c, \text{ls}_j \rangle$ request to $\mathcal{F}_{\text{UpSPA}}$ for either an honest or a corrupted login server and relays the matching final **OK/FAIL** response to $\mathcal{A}$.

Password update phase:

1. **Sim Behavior for Uid_l^o .** The password update request is for the honest user Uid_l^o . **Sim** receives $\langle \text{PwdUpdate}, \text{Uid}_l^o \rangle$ from $\mathcal{F}_{\text{UpSPA}}$ and sends Uid_l^o to $\mathcal{A}$. Then, **Sim**:
 - (a) If any required corrupted $\mathcal{SP}_i$ replies with **FAIL** or the committed setup record of Uid_l^o does not exist, **Sim** sends **FAIL** to $\mathcal{F}_{\text{UpSPA}}$ and aborts the password update. Otherwise, it performs the TOPRF interaction using $\tilde{\text{pwd}}$, its honest contributions, and the partial evaluations returned by $\mathcal{A}$. It runs the protocol-defined reconstruction algorithm. If reconstruction outputs $\perp$, **Sim** sends **FAIL** to $\mathcal{A}$ and $\mathcal{F}_{\text{UpSPA}}$. A successful reconstruction uses the same current value $R(\text{pwd})$.
 - (b) Receives cid^c from each corrupted $\mathcal{SP}_i$ and checks that every required response is the current committed $\tilde{\text{cid}}$. If this check fails, **Sim** sends **FAIL** to $\mathcal{A}$ and $\mathcal{F}_{\text{UpSPA}}$ and aborts the password update. Otherwise, it recovers the same logical values $\tilde{\text{ssk}} \parallel \widetilde{\mathcal{R}^{sp}} \parallel \widetilde{\mathbf{K}_0}$ associated with the committed $\tilde{\text{cid}}$.
 - (c) Generates a fresh TOPRF key and its provider shares as $\tilde{\mathbf{K}}_{\text{new}}, \{\tilde{\mathbf{k}}_{i,\text{new}}\}_{i \in [n_{sp}]} \leftarrow \text{TOPRFKeyGen}(1^\lambda, n_{sp}, t_{sp})$. It does not generate a new signing and verification key pair. It samples an independent simulated identification-encryption key for the new password as $R(\text{pwd}_{\text{new}}) \leftarrow \{0, 1\}^\lambda$.

- (d) Computes $\tilde{\text{cid}}_{\text{new}} \leftarrow \text{Enc}_{R(\text{pwd}_{\text{new}})}(\{0, 1\}^{|\text{ssk}|+|\mathcal{R}^{sp}|+|\mathbf{K}_0|})$, and stores that $\tilde{\text{cid}}_{\text{new}}$ logically represents the unchanged identification plaintext $\text{ssk} \parallel \widetilde{\mathcal{R}^{sp}} \parallel \widetilde{\mathbf{K}_0}$.
- (e) For each provider index $i \in [n_{sp}]$, computes $\tilde{\text{sig}}_i \leftarrow \text{Sign}(\text{ssk}, \tilde{\text{cid}}_{\text{new}} \parallel \tilde{\mathbf{k}}_{i,\text{new}} \parallel \text{time} \parallel i)$. For each corrupted $\mathcal{SP}_i$, it sends the corresponding pending tuple $\langle \tilde{\text{sig}}_i, \tilde{\text{cid}}_{\text{new}}, \tilde{\mathbf{k}}_{i,\text{new}}, \text{time} \rangle$ to $\mathcal{A}$. Each simulated honest provider verifies the same signed string under $\tilde{\text{ssk}}$, checks the freshness of time , and stores the new identification ciphertext and its own new TOPRF share only as pending while retaining the previous committed identification tuple.
- (f) If any required provider rejects the pending tuple, **Sim** sends **Discard** to every provider that accepted it, removes the pending identification tuple, leaves the previous committed identification tuple unchanged, sends **FAIL** to $\mathcal{F}_{\text{UpSPA}}$, and aborts the password update.
- (g) If all n_{sp} providers acknowledge the pending tuple, **Sim** sends **Store** to all providers. Only after all n_{sp} providers return their final OK does **Sim** replace the committed identification ciphertext, full TOPRF key, provider shares, and current password-derived value by $\tilde{\text{cid}}_{\text{new}}, \tilde{\mathbf{K}}_{\text{new}}, \{\tilde{\mathbf{k}}_{i,\text{new}}\}_{i \in [n_{sp}]}$, and $R(\text{pwd}_{\text{new}})$, respectively. It then removes the pending identification tuple and sends the final OK to $\mathcal{F}_{\text{UpSPA}}$. If any required final acknowledgement is missing or is **FAIL**, **Sim** does not report an ideal password-update success.

2. **Sim Behavior for Uid_l^c .** $\mathcal{A}$, as a malicious user with Uid_l^c , requests the password update.

Sim receives Uid_l^c from $\mathcal{A}$. If the setup record of Uid_l^c does not exist, **Sim** aborts. Otherwise, **Sim**:

- (a) Executes TOPRF with $\mathcal{A}$ using the $\mathbf{k}_i^c$ values associated with Uid_l^c as $\mathcal{A}[F_{\mathbf{K}^c}(\text{pwd}^c)] \leftarrow \text{TOPRF}(\mathcal{A}[\text{pwd}^c], \{\text{Sim}[\mathbf{k}_i^c]\}_{i \in [n_{sp}-t_{sp}+1]} \cup \{\mathcal{A}[\mathbf{k}_i^c]\}_{i \in [t_{sp}-1]}),$

and sends the corresponding committed cid^c values of the honest providers to $\mathcal{A}$.

- (b) For every honest provider index i , receives $\langle \text{sig}_i^c, \text{cid}_{new}^c, \text{k}_{i,new}^c, \text{time} \rangle$ from $\mathcal{A}$. It verifies $\text{Sign.Verify}(\text{svk}_i^c, \text{sig}_i^c, \text{cid}_{new}^c \parallel \text{k}_{i,new}^c \parallel \text{time} \parallel i)$, where $\text{svk}_i^c = \sigma_i^c$, and checks that time is fresh. If any required verification fails, Sim sends **FAIL** to $\mathcal{A}$ and $\mathcal{F}_{\text{UpSPA}}$ and does not change the committed identification tuple. Otherwise, each simulated honest provider stores the received replacement only as pending.
- (c) If the required pending acknowledgements are obtained and $\mathcal{A}$ sends the matching **Store** command, each simulated honest provider promotes its pending identification tuple and returns the final response. Only after the same all-provider final-acknowledgement point as in the real protocol does Sim send $\langle \text{PwdUpdate}, \text{Uid}_i^c \rangle$ to $\mathcal{F}_{\text{UpSPA}}$ and relay the matching final **OK/FAIL** response to $\mathcal{A}$. If $\mathcal{A}$ sends **Discard** or the pending round fails, Sim removes the pending tuple, leaves the previous committed identification tuple unchanged, and sends **FAIL** to $\mathcal{A}$.

Sim retains a previous committed value only while a replacement is pending. A matching **Discard** removes the pending replacement and leaves the previous committed value active, whereas a completed **Store** makes the replacement the current committed value. Sim does not retain an unbounded history of previous versions.

Corrupted users. When the acting user is corrupted, Sim does not replace the values selected by $\mathcal{A}$. It executes the honest storage-provider and honest login-server checks on those values, including authenticated decryption, identifier lookup, counter and timestamp checks, signature verification, tentative-state rules, and **Store/Discard** finalization. It invokes the corresponding ideal phase only when the same phase would be accepted by the honest real-world parties. Thus, corrupted users receive the same success, failure, and permitted-abort behavior in both worlds.

4.3.3 Bad Events Outside the Hybrid Games

The following events are monitored in every game. Hash evaluation, signature generation and verification, authenticated-decryption checks, counters, timestamps, and finalization rules are otherwise unchanged throughout the game sequence.

- **Signature forgery** **BadSig**. The event occurs if an honest storage provider accepts (m, σ) under an honest user's verification key, where $\text{Sign.Verify}_{\text{svk}}(\sigma, m) = 1$ but m was never signed by the honest user under the corresponding signing key. The relevant password-update message is exactly $m = \text{Uid}_l \parallel \text{cid}_{\text{new}} \parallel \text{k}_{i,\text{new}} \parallel \text{time} \parallel i$. An exact replay of an honestly signed message is not **BadSig**; it is rejected by the timestamp rule. Changing any signed component creates a new message and therefore requires a forgery.
- **Hash collision** **BadHash**. The event occurs if two distinct relevant protocol inputs $x \neq x'$ satisfy $H(x) = H(x')$ and that equality changes an identifier lookup, verifier comparison, or phase outcome. The relevant inputs have the forms $\mathcal{R}^{sp} \parallel \text{ls}_j \parallel i$ and $\mathcal{R}^{\text{ls}_j} \parallel \text{ls}_j$. The simulator samples $\mathcal{R}^{sp}$ and $\mathcal{R}^{\text{ls}_j}$ from the same distributions as the real protocol and evaluates H honestly. No hash output is replaced in a game. Collision resistance is therefore used only for unambiguous binding.
- **Authenticated-encryption failure** **BadCtxt**. The event occurs if an honest party accepts an adversarial ciphertext that was not produced by an honest encryption under the relevant honest key, or if a ciphertext honestly produced under k is accepted under a different key $k' \neq k$ in a way that changes a phase outcome. Replaying an honestly generated ciphertext is not itself **BadCtxt**; current-state, counter, timestamp, and finalization rules determine whether the replay is effective.

Let q_{dec} be the number of adversarial ciphertext candidates processed under honest keys, q_{kb} the number of cross-key acceptance attempts, and q_{vk} the

number of honest signature verification keys generated in the execution. Define $\epsilon_{\text{bad}}(\lambda) = q_{\text{dec}} \cdot \text{Adv}_{\text{AE}}^{\text{int-ctxt}}(\mathcal{B}_{\text{ctxt}}) + q_{\text{kb}} \cdot \text{Adv}_{\text{AE}}^{\text{kb}}(\mathcal{B}_{\text{kb}}) + q_{\text{vk}} \cdot \text{Adv}_{\text{Sign}}^{\text{euf-cma}}(\mathcal{B}_{\text{sig}}) + \text{Adv}_{\text{H}}^{\text{cr}}(\mathcal{B}_{\text{hash}})$.

Lemma 4.3.2 (Bad-event bound). In either the real or ideal execution, $\Pr[\text{BadSig} \vee \text{BadHash} \vee \text{BadCtxt}] \leq \epsilon_{\text{bad}}(\lambda)$.

Proof. If **BadSig** occurs, a reduction guesses the affected honest verification key, forwards honest password-update signatures as signing-oracle queries, and outputs the first newly accepted pair as an **EUF – CMA** forgery. The factor q_{vk} accounts for guessing the affected key; the **EUF – CMA** experiment itself already permits polynomially many adaptive signing queries under that key.

If **BadHash** occurs, the simulator knows every relevant hash input it generated or received. It outputs the two distinct inputs that produced the same hash value, thereby breaking collision resistance.

If the first part of **BadCtxt** occurs, the accepted fresh ciphertext gives an authenticated-encryption integrity forgery. If the second part occurs, the accepted ciphertext and the two different keys give a key-binding violation. Guessing the affected verification key, decryption attempt, or cross-key attempt gives exactly the factors in $\epsilon_{\text{bad}}(\lambda)$. A union bound completes the proof. $\square$

Stopped-game convention. Every game below is equipped with the same monitor and returns a fixed bit when one of the three bad events first occurs. This keeps the confidentiality reductions independent of integrity and binding failures. By Lemma 4.3.2, replacing an unstopped real or ideal execution by its stopped form changes the output probability by at most $\epsilon_{\text{bad}}(\lambda)$.

4.3.4 Hybrid Games, Lemmas, and Proofs

The following sequence of games establishes that the view of $\mathcal{A}$ in its interaction with **Sim** is computationally indistinguishable from its view in an interaction with the real-world honest parties.

For each game G_i , let $p_i = \Pr[\mathcal{A} \text{ outputs } 1 \text{ in } G_i]$, $\Delta_i = |p_i - p_{i+1}|$. All games preserve the same public message formats, ciphertext lengths, phase scheduling,

provider failures, login-server decisions, counters, timestamps, and **Store/Discard** behavior. Registration, secret update, and password update keep replacement records tentative until the corresponding **Store** command completes. On **Discard**, the tentative record is removed and the previous committed record remains unchanged. Authentication uses only committed records. A state-changing phase returns **OK** only after the protocol-defined final acknowledgements are received; otherwise, it returns **FAIL**. If the real TOPRF execution or protocol phase would output $\perp$, the corresponding game also outputs $\perp$.

Game G_0 : real execution. G_0 is the stopped real-world execution of UpSPA with $\mathcal{A}$ controlling the corrupted users, login servers, storage providers, and network scheduling allowed by the model. Honest users use the actual values $F_K(x)$, honest ciphertexts contain the real protocol plaintexts, and all honest parties execute the protocol as written.

Game G_1 : replace TOPRF values. For every honestly generated full TOPRF key, an independent truly random function $R : \mathcal{X} \rightarrow \mathcal{Y}$ is sampled and associated with that key. Every use of the corresponding $F_K(x)$ by an honest user is replaced consistently by the associated $R(x)$. Repeated evaluations under the same full TOPRF key use the same associated random function, whereas independently generated full TOPRF keys use independent random functions. In particular, registration, authentication, secret update, and the reconstruction part of password update reuse the current $R(\text{pwd})$, while a fresh full TOPRF key generated during password update is associated with an independent $R(\text{pwd}_{\text{new}})$. The new association becomes current only if the password update is committed.

The TOPRF messages exchanged with corrupted storage providers are generated exactly as in G_0 . Their partial evaluations are processed using the same protocol-defined TOPRF reconstruction algorithm. If fewer than t_{sp} valid contributions remain or reconstruction outputs $\perp$, then G_1 produces the same $\perp$ and the corresponding phase follows the same failure behavior. A missing or invalid corrupted-provider response does not by itself cause authentication to fail when at least t_{sp} valid re-

sponses remain.

Remark. Setup and the creation of cid_{new} during password update do not invoke an interactive TOPRF session, since the user generates the corresponding full TOPRF key and evaluates it locally. These local evaluations are nevertheless included in G_1 and are replaced by the random-function value associated with the corresponding full TOPRF key.

Lemma 4.3.3 (Transition G_0 to G_1). If TOPRF satisfies receiver-side PRF security against the stated corruption threshold, then $\Delta_0 \leq \text{Adv}_{\text{TOPRF}}^{\text{prf}}(\mathcal{B}_{\text{prf}})$.

Proof. The advantage term denotes the standard hybrid over all honestly generated full TOPRF keys in the execution. Order these keys by their generation time and replace the corresponding PRF by an independent random function one key at a time. Adjacent hybrids differ for one independently generated full TOPRF key K^* . The reduction $\mathcal{B}_{\text{prf}}$ embeds the challenge shares for K^* as the shares held by corrupted providers and generates the remaining key material itself.

At the beginning of the TOPRF security experiment, the challenger samples one hidden bit. All local and interactive evaluations under K^* are then answered consistently: one challenge world returns the real $F_{K^*}(\cdot)$ values, while the other returns the values of one fixed random function associated with K^* . Thus, repeated evaluations of the same input receive the same value throughout the experiment. Setup and the local evaluation used to create cid_{new} are handled in the same way as the interactive evaluations.

The reduction relays the partial evaluations supplied by corrupted providers and applies the protocol-defined reconstruction algorithm without changing its outcome. In particular, it does not replace a failed reconstruction by a random-function value. Therefore, malformed, missing, or insufficient contributions produce the same $\perp$ and phase failure in both adjacent hybrids, while authentication continues whenever at least t_{sp} valid contributions remain.

If the challenge bit selects the real function, the view is the earlier adjacent hybrid; if it selects the random function, the view is the later one. The standard hybrid over the polynomially many honestly generated full TOPRF keys proves the

lemma. □

Game G_2 : replace identification-ciphertext plaintexts. Starting from G_1 , order the calls in $\mathcal{E}_{\text{id}}$. For every call $E_r^{\text{id}} = (k_r, m_r^{\text{id}})$, replace $c_r \leftarrow \text{Enc}_{k_r}(m_r^{\text{id}})$ by $\tilde{m}_r^{\text{id}} \xleftarrow{\$} \{0, 1\}^{|m_r^{\text{id}}|}, \tilde{c}_r \leftarrow \text{Enc}_{k_r}(\tilde{m}_r^{\text{id}})$. The game records $\mathcal{M}_{\text{id}}[\tilde{c}_r] = m_r^{\text{id}}$. Thus, setup records and tentative password-update records carry random same-length physical plaintexts, while future decryption, signature, timestamp, **Store**, and **Discard** behavior is determined by their unchanged logical records. When the ciphertext is a proposed cid_{new} , its logical association remains tentative until **Store** completes. On **Discard**, the tentative association is removed and the previous committed cid and TOPRF state remain current.

Remark. Each identification ciphertext is encrypted under the key prescribed by its protocol instance. The setup ciphertext uses the current $R(\text{pwd})$, whereas a password-update ciphertext uses the independent $R(\text{pwd}_{\text{new}})$ associated with the newly generated TOPRF key. The hybrid replaces only the plaintext and does not identify or reuse these two encryption keys.

Lemma 4.3.4 (Transition G_1 to G_2). Under IND – CPA security of authenticated encryption, $\Delta_1 \leq q_{\text{id}} \text{Adv}_{\text{AE}}^{\text{ind-cpa}}(\mathcal{B}_{\text{cpa}, \text{id}})$.

Proof. For $0 \leq r \leq q_{\text{id}}$, define $H_{2,r}$ as the execution in which the first r identification-encryption calls encrypt random same-length strings and the remaining calls encrypt their logical protocol plaintexts. Then $H_{2,0} = G_1$, $H_{2,q_{\text{id}}} = G_2$. If $\mathcal{A}$ distinguishes the endpoints with advantage ϵ , an adjacent pair is distinguished with advantage at least ϵ/q_{id} .

The reduction $\mathcal{B}_{\text{cpa}, \text{id}}$ guesses the target call r^* and assigns the IND-CPA challenger's hidden key to the opaque identification-encryption value used by that call. It does not receive that key and does not compare it with any protocol key. For the target call, it submits $(m_{r^*}^{\text{id}}, \tilde{m}_{r^*}^{\text{id}})$ as its equal-length challenge pair. Earlier and later calls are generated according to H_{2,r^*-1} or H_{2,r^*} , using locally sampled keys or the encryption oracle whenever the target opaque key is used.

The phase-specific simulator behavior is essential here. The reduction stores the

target challenge ciphertext in $\mathcal{M}_{\text{id}}$ and thereafter resolves every honestly generated identification ciphertext through that table. It therefore never needs a decryption oracle for the challenge key. Outside **BadCtxt**, an unknown adversarial ciphertext cannot be accepted under that key; an inconsistent provider response produces the same abort in both adjacent hybrids. Tentative password-update state is promoted only on **Store** and removed on **Discard**, independently of the challenge bit.

If the challenge bit selects the logical plaintext, the view is the earlier adjacent hybrid; if it selects the random plaintext, the view is the later one. Summing over the q_{id} calls proves the bound. $\square$

Game G_3 : replace account-ciphertext plaintexts. Starting from G_2 , order the calls in $\mathcal{E}_{\text{sp}}$. For every call $E_r^{\text{sp}} = (k_r, m_r^{\text{sp}})$, replace the encryption of m_r^{sp} by the encryption of $\tilde{m}_r^{\text{sp}} \xleftarrow{\$} \{0, 1\}^{|m_r^{\text{sp}}|}$, $\tilde{c}_r \leftarrow \text{Enc}_{k_r}(\tilde{m}_r^{\text{sp}})$. The game records $\mathcal{M}_{\text{sp}}[\tilde{c}_r] = m_r^{\text{sp}}$. Registration and secret update therefore expose random same-length account ciphertexts, while the logical $\mathcal{R}^{\text{ls}_j}$, counter, verifier, tentative-state, and finalization behavior is preserved in the simulator tables. Registration and secret-update records become current only after **Store**; **Discard** removes the tentative record and retains the previous committed value. Authentication ignores tentative records and applies the protocol-defined counter rule to the valid committed records.

Lemma 4.3.5 (Transition G_2 to G_3). Under IND – CPA security of authenticated encryption, $\Delta_2 \leq q_{\text{sp}} \text{Adv}_{\text{AE}}^{\text{ind-cpa}}(\mathcal{B}_{\text{cpa,sp}})$.

Proof. For $0 \leq r \leq q_{\text{sp}}$, let $H_{3,r}$ be the execution in which the first r account-encryption calls encrypt random same-length strings and the remaining calls encrypt their logical account plaintexts. Then $H_{3,0} = G_2$, $H_{3,q_{\text{sp}}} = G_3$. The reduction $\mathcal{B}_{\text{cpa,sp}}$ guesses the target account encryption and installs the IND-CPA challenger’s hidden key as the corresponding simulated K_0 . Because one K_0 can protect multiple login-server records and successive secret updates, the reduction uses its encryption oracle for every nonchallenge encryption under that same hidden key. At the target call it submits the logical account plaintext and a random same-length string as the challenge pair.

As in Lemma 4.3.4, the reduction records the challenge ciphertext in $\mathcal{M}_{\text{sp}}$. Registration keeps it tentative until the login-server decision; authentication returns only the current committed record; and secret update records the incremented logical counter and promotes it only after **Store**. Thus, all later decryptions, verifier computations, counter comparisons, and aborts are answered from the logical table without a decryption oracle. Outside **BadCtxt**, no unknown adversarial ciphertext is accepted under the hidden key.

The two challenge bits yield the two adjacent hybrids. A hybrid over the q_{sp} calls proves the lemma. $\square$

Game G_4 : ideal execution with $\mathcal{F}_{\text{UpSPA}}$ and Sim. In G_4 , replace the honest-party code by the simulator described in Section 4.3.2 interacting with $\mathcal{F}_{\text{UpSPA}}$. The simulator uses the same logical state and ciphertext tables already used in the preceding games. In Setup, it samples the same key material and produces a identification ciphertext of the same length and distribution established in G_1 and G_2 . In Registration and Secret Update, it maintains the account records tentatively and sends the same **Store/Discard** commands after the corresponding login-server decision. In Authentication, it uses only committed records, applies the same TOPRF reconstruction and counter-selection rules, and produces the same verifier and success or failure result. In Password Update, it retains ssk , $\mathcal{R}^{sp}$, and $\mathbf{K}_0$, generates fresh TOPRF key material, uses the independent $R(\text{pwd}_{\text{new}})$, produces the protocol-defined signatures, and commits the new identification record only after successful finalization. For corrupted users, it uses the values chosen by $\mathcal{A}$ and invokes the corresponding ideal phase only when those values pass the same honest-party checks. Tentative records and **Store/Discard** remain local simulator mechanics and are not added to $\mathcal{F}_{\text{UpSPA}}$.

Lemma 4.3.6 (Transition G_3 to G_4). Conditioned on the stopped-game convention, $\Delta_3 = |p_3 - p_4| = 0$.

Proof. We compare the two games phase by phase.

- **Setup.** In G_3 , the TOPRF key and shares, signing key pair, $\mathcal{R}^{sp}$, and $\mathbf{K}_0$

are sampled exactly as in the simulator. By G_1 , the password-dependent identification key is an opaque random-function value; by G_2 , cid encrypts a random same-length string. These are exactly the values generated by the setup simulation. Provider failures and the completed setup result are relayed identically.

- **Registration.** By G_1 , the corrupted providers see the same TOPRF messages and reconstruction behavior, and the logical identification state is recovered from $\mathcal{M}_{\text{id}}$. The simulator samples $\mathcal{R}^{\text{ls}_j}$, computes the provider-specific identifiers and vInfo_j honestly, and, by G_3 , sends a same-length dummy c_j . Both games first create tentative provider records, use the same login-server decision, and send the same **Store** or **Discard**.
- **Authentication.** Both games use the same TOPRF messages and reconstruction behavior from G_1 , resolve the current identification and account ciphertexts through the same logical tables, and ignore tentative entries. The same $\mathcal{R}^{\text{ls}_j}$ yields the same vInfo'_j . A corrupted login server receives that value in both games; an honest login server exposes only the ideal success or failure.
- **Secret update.** Both games sample a fresh $\mathcal{R}^{\text{ls}_j}_{\text{new}}$, increment the same logical counter, compute $\text{vInfo}_{j,\text{new}}$ honestly, and create a dummy account ciphertext as specified in G_3 . The record remains tentative until the same login-server decision causes **Store** or **Discard**.
- **Password update.** Both games reconstruct the current logical identification using the G_1 TOPRF replacement, retain ssk , $\mathcal{R}^{\text{sp}}$, and K_0 , and generate a fresh TOPRF key and shares using **TOPRFKeyGen**. The new key receives an independent opaque identification value, and G_2 makes cid_{new} an encryption of a random same-length string. Both games sign exactly $\text{Uid}_i \parallel \text{cid}_{\text{new}} \parallel \text{k}_{i,\text{new}} \parallel \text{time} \parallel i$, keep the new identification tuple tentative, and make it current only after **Store**. On **Discard**, both retain the previous identification key, shares, ciphertext, and timestamp.

- **Corrupted users.** Both games use the values chosen by $\mathcal{A}$ and execute the same honest provider and login-server checks. Outside **BadSig**, **BadHash**, and **BadCtxt**, a newly accepted signature, ambiguous hash binding, fresh valid adversarial ciphertext, or cross-key acceptance cannot occur. Therefore, **Sim** invokes an ideal phase exactly when the corresponding real honest parties accept it.

After the preceding replacements, every remaining value visible to $\mathcal{A}$ is either sampled by **Sim** from the same distribution as in G_3 , chosen by $\mathcal{A}$, or determined by the same public validity and finalization rules. The two distributions are therefore identical. $\square$

4.3.5 Completion of the Theorem

Let

$$\text{Adv}_{\text{UpSPA}, \mathcal{F}_{\text{UpSPA}}}^{\text{real/ideal}}(\mathcal{A}) = \left| \Pr[\text{REAL}_{\text{UpSPA}, \mathcal{A}}(1^\lambda, \text{aux}) = 1] - \Pr[\text{IDEAL}_{\mathcal{F}_{\text{UpSPA}}, \text{Sim}}(1^\lambda, \text{aux}) = 1] \right|.$$

The unstopped real execution differs from G_0 by at most $\epsilon_{\text{bad}}(\lambda)$, and G_4 differs from the unstopped ideal execution by at most the same amount. Applying Lemmas 4.3.3–4.3.6 and the triangle inequality gives

$$\begin{aligned} \text{Adv}_{\text{UpSPA}, \mathcal{F}_{\text{UpSPA}}}^{\text{real/ideal}}(\mathcal{A}) &\leq 2 \cdot \epsilon_{\text{bad}}(\lambda) + \text{Adv}_{\text{TOPRF}}^{\text{prf}}(\mathcal{B}_{\text{prf}}) \\ &\quad + q_{\text{id}} \cdot \text{Adv}_{\text{AE}}^{\text{ind-cpa}}(\mathcal{B}_{\text{cpa}, \text{id}}) + q_{\text{sp}} \cdot \text{Adv}_{\text{AE}}^{\text{ind-cpa}}(\mathcal{B}_{\text{cpa}, \text{sp}}). \end{aligned}$$

All query counts are polynomial. Receiver-side PRF security of TOPRF, IND – CPA privacy, ciphertext integrity and key binding of authenticated encryption, EUF – CMA security of **Sign**, and collision resistance of **H** make every term on the right negligible. This proves Theorem 4.3.1. $\square$

Password-security scope. Because **pwd** is human memorable, the theorem does not claim that an adversary’s prior probability of guessing it is negligible. It states that the corrupted providers’ protocol view supplies no additional offline verification predicate beyond the leakage and online attempts permitted by $\mathcal{F}_{\text{UpSPA}}$.

Chapter 5

ROUPSPA: ROBUST UPDATABLE SINGLE PASSWORD AUTHENTICATION

5.1 System Design & Threat Model

RoUpSPA inherits the parties, cryptographic core, user-visible functionality, and trust boundaries of UpSPA. The password-protected identification record, the login-server-specific account records, the TOPRF, authenticated encryption, domain-bound verifiers, and the five calls Π_{Setup} , Π_{Register} , $\Pi_{\text{Authenticate}}$, $\Pi_{\text{SecretUpdate}}$, and $\Pi_{\text{PwdUpdate}}$ remain as defined in Section 4.1 [34]. This section states only the changes introduced by partial provider availability. The exact phase messages, ordering, acceptance conditions, and completion rules appear only in Section 5.2.

5.1.1 System Model of RoUpSPA

Inherited model. Unless stated below, the parties, authenticated-domain model, trusted-client boundary, public provider configuration, login-server interface, privacy scope, and cryptographic assumptions are those of UpSPA [34]. A login server still stores only its high-entropy verifier and does not evaluate the TOPRF or communicate with storage providers. The resilience metadata reveals neither `pwd` nor a password hash, TOPRF output, identification plaintext, account plaintext, or decryption key.

Cryptographic threshold and availability quorum. The threshold t_{sp} retains its UpSPA meaning: it is the number of valid provider contributions required for TOPRF evaluation and the boundary below which provider corruption does not reveal the password-protected identification. The new parameter q_{sp} determines when a replicated write or synchronization has enough matching provider participation.

The parameters satisfy

$$1 \leq t_{sp} \leq q_{sp} \leq n_{sp}, \quad t_{sp} > \frac{n_{sp}}{2}, \quad 2 \cdot q_{sp} - n_{sp} > t_{sp} - 1.$$

For any $Q_1, Q_2 \subseteq [n_{sp}]$ with $|Q_1|, |Q_2| \geq q_{sp}$,

$$|Q_1 \cap Q_2| \geq 2 \cdot q_{sp} - n_{sp} > t_{sp} - 1.$$

Hence, two successful quorums contain an honest provider in their intersection. This condition preserves reachability of a completed record; it does not by itself authenticate a record or guarantee that a quorum is available.

Changed record fields. RoUpSPA adds freshness and synchronization information to the UpSPA record families. The identification plaintext contains the identification counter ctr_{id} , and account records carry the account counter ctr_{ac} . Providers use the shared account-family identifier $\text{SUid}_{l,j} \leftarrow H(\mathcal{R}^{sp} \parallel \text{ls}_j)$, rather than a provider-specific identifier. Because this identifier is shared for synchronization, it is treated only as a routing value. Record replacement is authorized by the user signature and constrained by the corresponding counter and local record state. The exact signed strings are fixed in Section 5.2.

Replica state. Freshness is tracked separately for the identification family and for each account family. A provider is active for a family only when its local record belongs to the latest completed write known for that family or when it has subsequently synchronized. A provider that missed the relevant write, restarted, or returned after an outage is stale for that family until synchronization succeeds. Stale status concerns availability and record selection; it does not reveal protected plaintext and does not alter the logical account state.

Completion and synchronization. Unlike fully available UpSPA, a completed RoUpSPA state change may be supported by a quorum rather than by all providers. Setup remains the initialization exception because every provider must receive its own long-term TOPRF share. Authentication continues to use the cryptographic threshold and does not become a write-quorum operation. Synchronization is

storage-layer maintenance: it brings a stale replica to an already completed logical state and is not a sixth call to $\mathcal{F}_{\text{UpSPA}}$. Section 5.2 is the sole source for the exact phase order, terminal messages, and provider acceptance rules.

Additional availability assumptions. RoUpSPA assumes that state-changing executions for the same identification or account family are serialized. Its liveness claims are conditional on obtaining the required valid threshold or quorum and, for phases involving an honest login server, on the bounded availability stated by the construction. Message suppression, a missing threshold, a missing quorum, or permanent loss of unresolved client state may prevent progress. These outcomes are denial of service or incomplete execution, not successful unauthorized transitions.

5.1.2 System Design of RoUpSPA

Inherited cryptographic structure. The separation between the compact password-protected identification state and the per-login-server account state is unchanged from UpSPA [34]. RoUpSPA does not introduce a new password-authentication primitive; it adds a resilient record lifecycle around the same protected values. Consequently, the password-security, account-secret confidentiality, and domain-separation arguments remain those of UpSPA, while the new design obligations concern replica freshness, quorum completion, and synchronization.

Shared addressing and explicit authorization. A common account-family identifier allows providers to refer to corresponding replicas during synchronization. Since that identifier may be visible within the storage layer, possession of it is not sufficient authorization. The user signature binds each accepted replacement to its intended user, record family, ciphertext, and counter. This is the principal authorization difference from the fully available construction.

Freshness and replica selection. The identification and account counters order valid records within their respective families. Active/stale status prevents a locally well-formed but outdated replica from being treated as current solely because it

remains decryptable. During synchronization, signatures and counters identify the unique freshest valid record, while quorum intersection ensures that a completed record remains reachable through an honest provider.

Cost of resilience. The persistent storage asymptotics remain those of UpSPA: one identification record per user and one account record per registered login server at each provider, giving $\mathcal{O}(1 + n_{ls})$ storage per user at one provider. RoUpSPA adds signatures, counters, replica-status metadata, quorum collection, and occasional synchronization. The exact round structure and per-phase communication cost are specified only in Section 5.2 and evaluated in the implementation chapter.

5.1.3 Threat Model

Adversary and corruption bounds. We consider a static malicious adversary that may control the network and the internal behavior of corrupted parties. It corrupts at most $t_{sp} - 1$ storage providers, at most $n_{ls} - 1$ login servers, and at most $n_u - 1$ users:

$$|\mathcal{C}_{SP}| \leq t_{sp} - 1, \quad |\mathcal{C}_{LS}| \leq n_{ls} - 1, \quad |\mathcal{C}_U| \leq n_u - 1.$$

A corrupted party may send malformed, stale, conflicting, or replayed values, withhold responses, or stop participating. Such behavior may cause denial of service. The security target is an honest user and, for an accepting login-server statement, an honest target $\mathcal{LS}_j$.

Trusted client and channels. The client is trusted while it handles `pwd` and reconstructed long-term secrets. Compromise of that endpoint is outside the guarantee. Honest-party communication uses authenticated and confidential channels. The adversary may schedule, delay, duplicate, and suppress messages subject to the bounded honest-login-server assumption of the construction. Canonicalization of $\mathbf{ls}_j$ and ordinary account-ownership checks remain deployment responsibilities.

Inherited cryptographic assumptions. The proof uses the same TOPRF, authenticated-encryption, signature, and hash assumptions as UpSPA. Under these

assumptions, fewer than t_{sp} providers do not obtain an offline password-verification predicate, and an adversary cannot create an accepted replacement for an honest user or make an older valid record supersede the latest completed one except with negligible probability. The shared account identifier is a lookup value and grants no authority by itself.

Additional leakage. The resilience layer reveals storage-side metadata that is absent from the fully available postcondition: a contacted provider may learn the record family, its public counter, whether its replica is stale, and whether synchronization is requested. The model does not hide provider-contact patterns, availability, quorum membership, update frequency, or the number of registered account families. It continues to hide the password, TOPRF output, and protected plaintexts within the inherited corruption bounds.

Safety and liveness boundary. The theorem protects completed phases. It does not force a corrupted provider or login server to answer, guarantee that a valid threshold or quorum exists, or provide automatic repair after permanent loss of unresolved client state. Setup may fail without all providers, authentication may fail without the required active threshold sets, and a write or synchronization may fail without q_{sp} valid responses. These are availability failures rather than successful unauthorized transitions.

Scope exclusions. The model excludes compromise of the trusted client, corruption of t_{sp} or more providers, forgotten-password recovery, permanent membership replacement, proactive refresh against mobile corruption, general Byzantine consensus, and general crash-recovery transactions spanning the login server and providers. These mechanisms may be added operationally but are not consequences of the present security theorem.

5.1.4 Design Challenges of RoUpSPA

Challenge 1: separating cryptographic security from availability. The threshold t_{sp} protects password-dependent computation, whereas q_{sp} expresses sufficient participation for replicated state. Keeping these roles separate avoids interpreting provider availability as a cryptographic assumption or treating threshold reconstruction as evidence of write completion.

Challenge 2: authorizing a shared record namespace. Synchronization requires a common account-family handle, but a shared handle cannot serve as an authorization secret. RoUpSPA therefore treats the handle as routing metadata and relies on the user signature, record binding, and counter rules for authorized replacement.

Challenge 3: distinguishing valid from current state. An older ciphertext may remain authentic and decryptable. Counters and family-specific active/stale status are therefore needed in addition to ciphertext validity. They let the system reject rollback without changing the protected plaintext structure inherited from UpSPA.

Challenge 4: repairing replicas without exposing secrets. A returning provider must identify the freshest completed record without learning `pwd`, a TOPRF output, another provider's TOPRF share, or decrypted account material. Synchronization consequently operates only on signed ciphertexts, counters, and public record metadata.

Challenge 5: limiting the resilience claim. RoUpSPA provides safety for completed quorum-backed state and conditional recovery for stale providers; it is not a general Byzantine transaction or consensus service. Missing responses, unresolved executions, and permanent client loss may prevent progress. Stating this boundary separately from the construction avoids repeating phase mechanics and prevents a liveness assumption from being presented as a cryptographic guarantee.

5.2 Construction

This section is the sole authoritative description of the RoUpSPA protocol. It fixes the exact phase messages, ordering, acceptance conditions, and completion rules. Sections 5.1.1–5.1.4 state only the differences from UpSPA and do not define a second protocol flow.

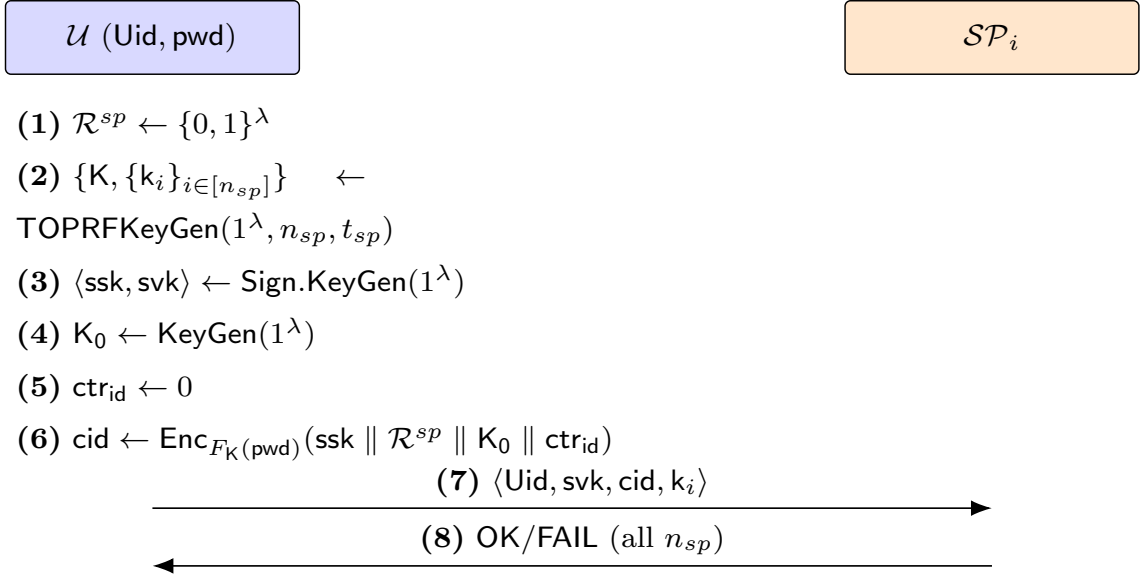
The construction uses direct storage-provider writes. The cryptographic threshold t_{sp} is used for TOPRF evaluation and ordinary reads, whereas q_{sp} is used for durable replicated writes and synchronization. The parameters satisfy

$$n_{sp} \geq q_{sp} \geq t_{sp} > \frac{n_{sp}}{2}, \quad 2 \cdot q_{sp} - n_{sp} > t_{sp} - 1.$$

Hence, every two successful q_{sp} quorums intersect in an honest provider. The account-family identifier is shared by all providers: $\text{SUid}_{l,j} \leftarrow \text{H}(\mathcal{R}^{sp} \parallel \text{ls}_j)$. It is a routing value rather than an authorization token. Account and identification replacements are authorized by signatures over the complete record and its counter.

An honest provider keeps one current identification record under Uid and one current account record under each $(\text{Uid}, \text{SUid}_{l,j})$. Registration accepts only account counter zero; Secret Update and Password Update accept only the exact next counter. The provider atomically installs the signed record before returning a durable acknowledgement. Exact retransmission is idempotent, while a different record at the same counter, a skipped counter, or a lower counter is rejected. Every read and synchronization also rejects distinct valid candidates tied at the largest counter instead of choosing between them.

A provider is active for a record family only if it acknowledged the latest completed write for that family or later synchronized it. A provider that missed the write, restarted, or returned after an outage is stale for the affected family and does not contribute to an ordinary read or write set until synchronization succeeds. State-changing executions for one identification or account family are serialized. Once any provider accepts a direct write, the user retains and retries the exact signed tuple until the required q_{sp} acknowledgements are obtained; no Authentication, later update, or Synchronization for that family begins while the write remains unresolved.

Figure 5.1: The RoUpSPA setup phase Π_{Setup} .

Setup. The one-time setup in Figure 5.1 is unchanged except for the identification counter used by the resilient variant. The user samples $\mathcal{R}^{sp}$, generates one TOPRF key and its n_{sp} shares, generates $\langle \text{ssk}, \text{svk} \rangle$, samples K_0 , and computes

$$\text{cid} \leftarrow \text{Enc}_{F_K(\text{pwd})}(\text{ssk} \parallel \mathcal{R}^{sp} \parallel K_0 \parallel \text{ctr}_{\text{id}}), \quad \text{ctr}_{\text{id}} = 0.$$

For each i , the user sends $\langle \text{Uid}, \text{svk}, \text{cid}, k_i \rangle$ to $\mathcal{SP}_i$. The provider accepts only if no setup record exists under Uid , stores its own TOPRF share and the common identification record, marks the identification active, and returns **OK**. Setup completes only after all n_{sp} providers acknowledge because a missing long-term TOPRF share cannot be obtained by ciphertext synchronization. The TOPRF key and shares remain unchanged during later Password Updates.

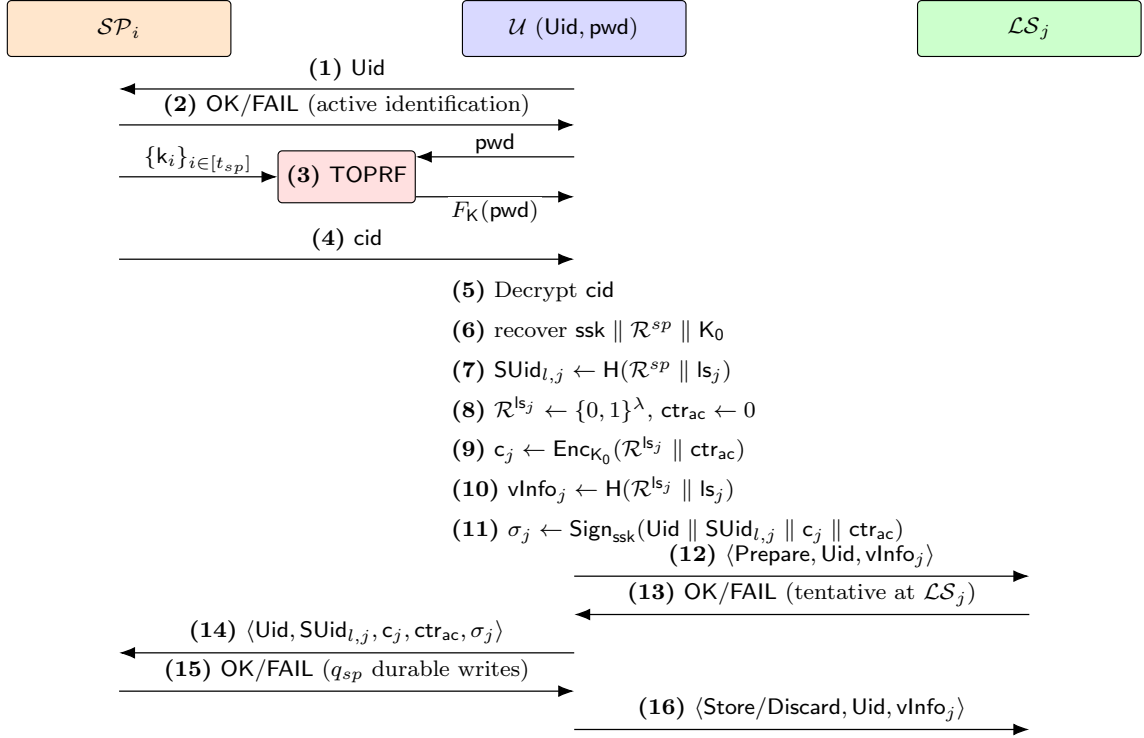
Storage-provider synchronization and repair. Synchronization is an auxiliary storage-layer procedure. It performs no TOPRF evaluation, creates no ciphertext, and changes no login-server state. A stale provider obtains at least q_{sp} responses for the affected family and validates the public record evidence.

For a counter-positive identification candidate $\langle \text{Uid}, \text{sig}_i, \text{cid}_{\text{new}}, \text{ctr}'_{\text{id}} \rangle$, the recovering provider checks $\text{Sign.Verify}_{\text{svk}}(\text{sig}_i, \text{Uid} \parallel \text{cid}_{\text{new}} \parallel \text{ctr}'_{\text{id}}) = 1$. The counter-zero Setup record is the unique initialization exception. The provider retains its own k_i ;

synchronization never copies or rotates a TOPRF share. For an account candidate $\langle \text{Uid}, \text{SUid}_{l,j}, c_j, \text{ctr}_{ac}, \sigma_j \rangle$, it checks $\text{Sign.Verify}_{\text{svk}}(\sigma_j, \text{Uid} \parallel \text{SUid}_{l,j} \parallel c_j \parallel \text{ctr}_{ac}) = 1$. Malformed records, invalid signatures, lower counters, and conflicting records at the same maximum counter are rejected. Among the remaining candidates, the provider installs the unique valid record with the largest counter and becomes active. If fewer than q_{sp} valid responses are available, it keeps its local state and remains stale. Synchronization is invoked only when no direct write for the same family is unresolved.

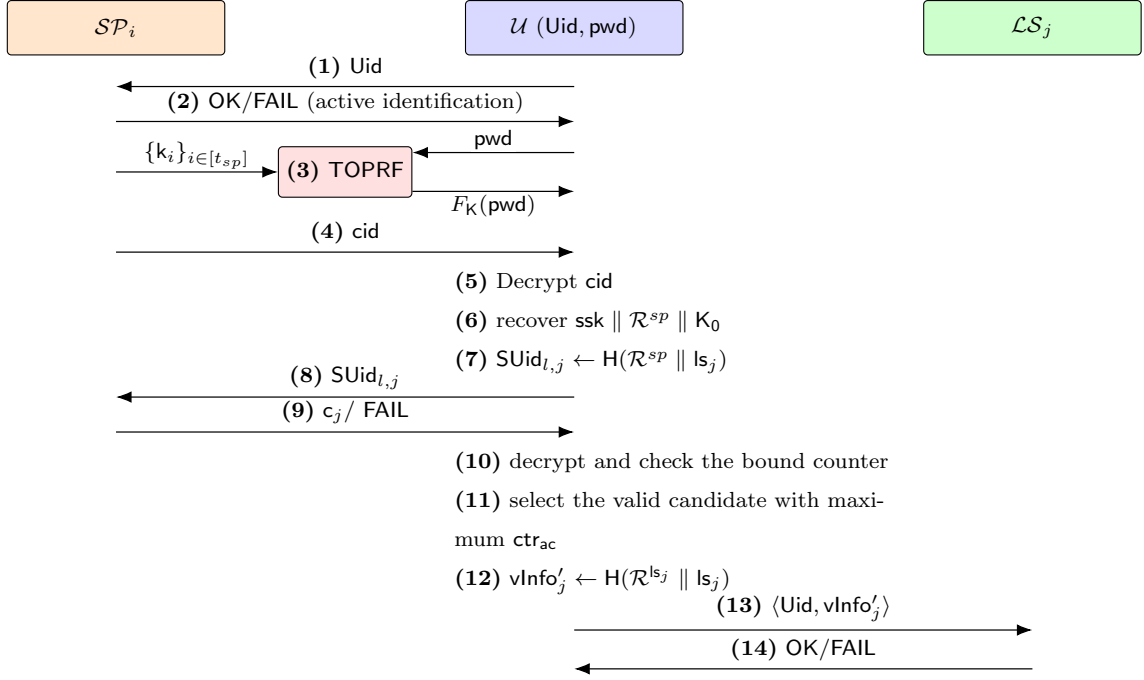
Registration. Registration in Figure 5.2 first opens the current identification record. The user obtains at least t_{sp} valid active identification responses whose svk values agree, evaluates the TOPRF on pwd , selects the record with the largest identification counter, and decrypts $\text{ssk} \parallel \mathcal{R}^{sp} \parallel K_0 \parallel \text{ctr}_{id} \leftarrow \text{Dec}_{F_K(\text{pwd})}(\text{cid})$. Failure of the threshold, counter, or authenticated-decryption checks returns FAIL. The user derives $\text{SUid}_{l,j}$, samples $\mathcal{R}^{ls_j}$, sets $\text{ctr}_{ac} = 0$, and computes $c_j \leftarrow \text{Enc}_{K_0}(\mathcal{R}^{ls_j} \parallel \text{ctr}_{ac})$, $\text{vInfo}_j \leftarrow H(\mathcal{R}^{ls_j} \parallel ls_j)$, $\sigma_j \leftarrow \text{Sign}_{\text{ssk}}(\text{Uid} \parallel \text{SUid}_{l,j} \parallel c_j \parallel \text{ctr}_{ac})$. Before contacting any provider, the user sends $\langle \text{Prepare}, \text{Uid}, \text{vInfo}_j \rangle$ to $\mathcal{LS}_j$. The login server accepts only if no committed or tentative account exists under Uid , records that exact tentative tuple, and returns OK. An exact replay returns the same result. A rejection or a Prepare timeout causes the user to abort before provider state is changed. After Prepare succeeds, the user sends the signed account tuple directly to all reachable providers. An honest provider obtains svk through Uid , verifies the complete signature, requires $\text{ctr}_{ac} = 0$ and no existing account under $(\text{Uid}, \text{SUid}_{l,j})$, atomically installs the record as current, and returns a durable acknowledgement. Registration reaches its provider boundary when q_{sp} providers acknowledge the same tuple. If this quorum is not obtained, the user sends $\langle \text{Discard}, \text{Uid}, \text{vInfo}_j \rangle$ to $\mathcal{LS}_j$ and the phase ends with FAIL. After the provider quorum, the user sends $\langle \text{Store}, \text{Uid}, \text{vInfo}_j \rangle$ to $\mathcal{LS}_j$. The matching Store message makes the tentative login-server record current. Sending Store or Discard is the last step of Registration; no further login-server response is part of the phase. Providers that did not acknowledge the completed write remain stale for this account until synchronization.

Authentication. Authentication in Figure 5.3 is read-only and uses t_{sp} rather than q_{sp} . The user obtains at least t_{sp} valid active identification responses with one

Figure 5.2: The RoUpSPA registration phase Π_{Register} .

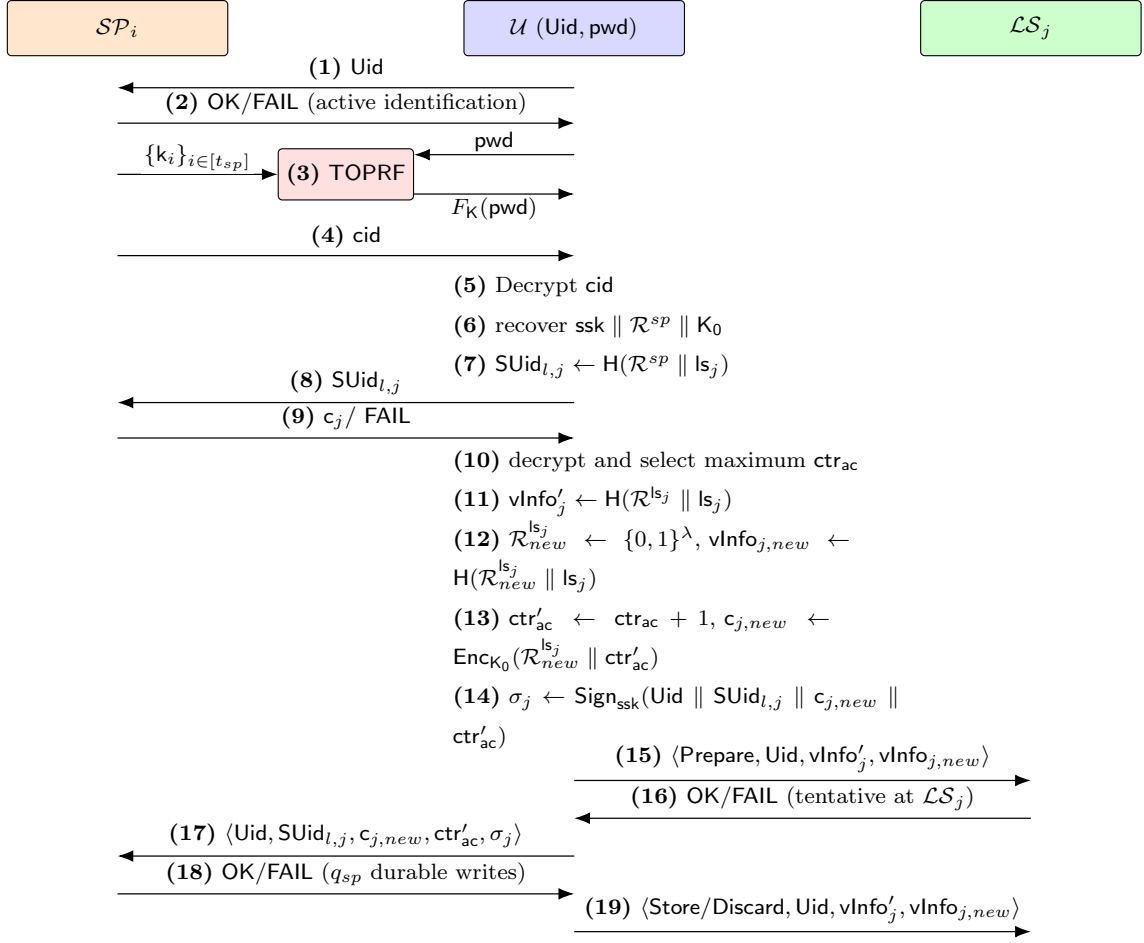
common svk , evaluates the TOPRF on pwd , selects the identification record with the largest counter, and decrypts cid .

The user derives $\text{SUid}_{l,j}$ and obtains at least t_{sp} valid active account responses. For each candidate, it decrypts $\mathcal{R}^{\text{ls}_j} \parallel \text{ctr}_{\text{ac}} \leftarrow \text{Dec}_{K_0}(c_j)$, checks the external and authenticated counters, and selects the candidate with the largest ctr_{ac} . It then sends $\langle \text{Uid}, H(\mathcal{R}^{\text{ls}_j} \parallel \text{ls}_j) \rangle$ to $\mathcal{LS}_j$. Authentication succeeds exactly when the login server accepts. A stale provider, an invalid record, or an insufficient active threshold yields FAIL and causes no repair or write-back.

Figure 5.3: The RoUpSPA authentication phase $\Pi_{\text{Authenticate}}$.

Secret update. Secret Update in Figure 5.4 first recovers the current identification and account records exactly as in Authentication. Let the selected account plaintext be $\mathcal{R}^{\text{ls}_j} \parallel \text{ctr}_{\text{ac}}$. The user computes $\text{vlnfo}'_j \leftarrow H(\mathcal{R}^{\text{ls}_j} \parallel \text{ls}_j)$, $\mathcal{R}^{\text{ls}_j}_{\text{new}} \leftarrow \{0, 1\}^\lambda$, $\text{vlnfo}_{j,\text{new}} \leftarrow H(\mathcal{R}^{\text{ls}_j}_{\text{new}} \parallel \text{ls}_j)$, $\text{ctr}'_{\text{ac}} \leftarrow \text{ctr}_{\text{ac}} + 1$, $c_{j,\text{new}} \leftarrow \text{Enc}_{K_0}(\mathcal{R}^{\text{ls}_j}_{\text{new}} \parallel \text{ctr}'_{\text{ac}})$, $\sigma_j \leftarrow \text{Sign}_{\text{ssk}}(\text{Uid} \parallel \text{SUid}_{l,j} \parallel c_{j,\text{new}} \parallel \text{ctr}'_{\text{ac}})$. The user first sends $\langle \text{Prepare}, \text{Uid}, \text{vlnfo}'_j, \text{vlnfo}_{j,\text{new}} \rangle$ to $\mathcal{LS}_j$. The login server accepts only if the current committed verifier equals vlnfo'_j and no other update is tentative. It records the exact tuple, keeps vlnfo'_j as the active verifier, and returns OK. A rejection or Prepare timeout changes no provider state.

After Prepare succeeds, the user sends the signed replacement directly to all reachable providers. An honest provider verifies the record under svk , requires that it is active for the account family and that $\text{ctr}'_{\text{ac}} = \text{ctr}_{\text{ac}} + 1$, atomically replaces its current account record, and returns a durable acknowledgement. If fewer than q_{sp} matching acknowledgements are obtained, the user sends $\langle \text{Discard}, \text{Uid}, \text{vlnfo}'_j, \text{vlnfo}_{j,\text{new}} \rangle$ to $\mathcal{LS}_j$ and the phase ends with FAIL.

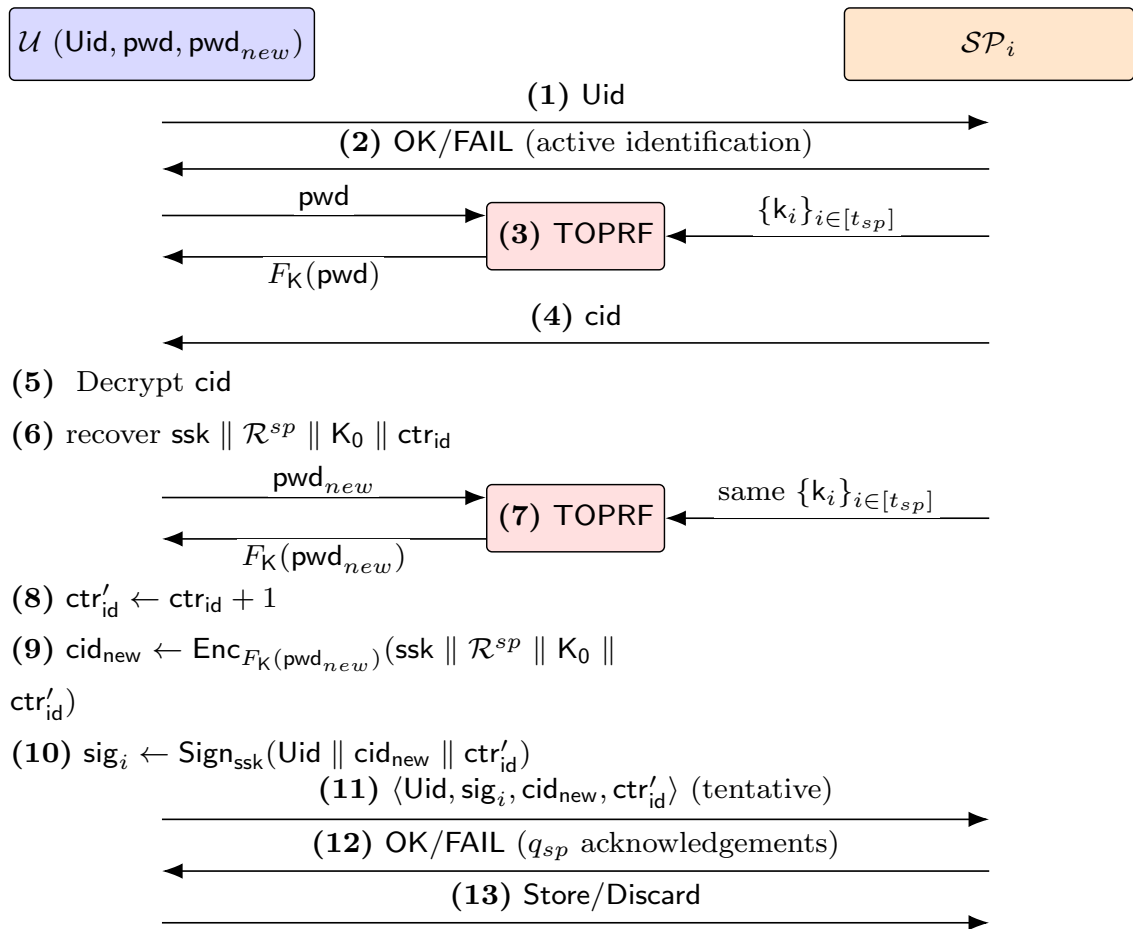
Figure 5.4: The RoUpSPA secret-update phase $\Pi_{\text{SecretUpdate}}$.

After the provider quorum, the user sends $\langle \text{Store}, \text{Uid}, \text{vlnfo}'_j, \text{vlnfo}_{j,\text{new}} \rangle$ to $\mathcal{LS}_j$. The matching Store message replaces the tentative verifier by $\text{vlnfo}_{j,\text{new}}$. Sending Store or Discard is the last step of Secret Update; no further login-server response is part of the phase. Providers that did not acknowledge the completed write remain stale until synchronization.

Password update. Password Update in Figure 5.5 changes only the protection of the identification record. The TOPRF full key and provider shares remain unchanged. The user first obtains at least t_{sp} valid active identification responses, evaluates the TOPRF on pwd , selects the identification record with the largest counter, and decrypts $\text{ssk} \parallel \mathcal{R}^{sp} \parallel K_0 \parallel \text{ctr}_{\text{id}} \leftarrow \text{Dec}_{F_K(\text{pwd})}(\text{cid})$. It then evaluates the same TOPRF key on pwd_{new} and computes $\text{ctr}'_{\text{id}} \leftarrow \text{ctr}_{\text{id}} + 1$, $\text{cid}_{\text{new}} \leftarrow \text{Enc}_{F_K(\text{pwd}_{\text{new}})}(\text{ssk} \parallel$

$\mathcal{R}^{sp} \parallel K_0 \parallel \text{ctr}'_{\text{id}}), \text{sig}_i \leftarrow \text{Sign}_{\text{ssk}}(\text{Uid} \parallel \text{cid}_{\text{new}} \parallel \text{ctr}'_{\text{id}}).$

The user sends the signed identification record to all reachable providers as tentative. An honest provider verifies the signature, requires that it is active for the identification family and that the new counter is exactly its current counter plus one, stores the tentative identification record while retaining its current identification record and its own k_i , and returns an acknowledgement. After q_{sp} matching acknowledgements, the user sends **Store**; otherwise, it sends **Discard**. Upon **Store**, an honest provider replaces the current identification record by the matching tentative record while retaining its own k_i . Upon **Discard**, it removes the matching tentative record and keeps the current identification record unchanged. The sent **Store** or **Discard** is the terminal step, and no additional acknowledgement follows. No login server is contacted. All account ciphertexts, account counters, and login-server verifiers remain unchanged.

Figure 5.5: The RoUpSPA password-update phase $\Pi_{\text{PwdUpdate}}$.

Record binding and completion semantics. The exact signed account record is $\text{Uid} \parallel \text{SUid}_{l,j} \parallel \text{c}_j \parallel \text{ctr}_{\text{ac}}$, and Secret Update replaces c_j and ctr_{ac} by $\text{c}_{j,\text{new}}$ and ctr_{ac}' . The exact signed identification replacement is $\text{Uid} \parallel \text{cid}_{\text{new}} \parallel \text{ctr}'_{\text{id}}$. All fields use the unambiguous encoding fixed in the preliminaries. The user identifier selects the verification key, the shared account identifier selects the record family, the ciphertext binds the protected state, and the counter binds freshness. No identifier or counter without the matching user signature authorizes a write.

The only tentative state is maintained by the login server. For Registration, $\langle \text{Store}, \text{Uid}, \text{vInfo}_j \rangle$ makes the matching tentative record current, whereas $\langle \text{Discard}, \text{Uid}, \text{vInfo}_j \rangle$ removes it. For Secret Update, the matching Store tuple replaces the tentative verifier by $\text{vInfo}_{j,\text{new}}$, whereas the matching Discard tuple removes the tentative update. Store or Discard is the terminal message; the login server sends no additional completion response.

The successful paths are

$$\begin{aligned} \Pi_{\text{Register}} &: \text{Prepare}_{\text{LS}} \longrightarrow q_{sp} \text{ direct account writes } \longrightarrow \text{Store}_{\text{LS}}, \\ \Pi_{\text{SecretUpdate}} &: \text{Prepare}_{\text{LS}} \longrightarrow q_{sp} \text{ direct account replacements } \longrightarrow \text{Store}_{\text{LS}}, \\ \Pi_{\text{PwdUpdate}} &: q_{sp} \text{ direct identification replacements.} \end{aligned}$$

A provider acknowledgement is counted only after the signed record is durable. Failure before an accepting login-server Prepare changes no provider state. If the provider quorum is not obtained, the user sends the matching Discard to the login server and the phase ends with **FAIL**. After the provider quorum, the user sends the matching Store and the phase ends successfully. No additional login-server commit message or acknowledgement follows Store or Discard.

Setup requires all n_{sp} providers. Authentication requires t_{sp} valid active responses for each required record family. Registration, Secret Update, Password Update, and Synchronization use q_{sp} at the single provider boundary described above. The construction therefore uses one provider-write boundary and treats the matching login-server Store or Discard as the terminal step of Registration and Secret Update.

5.3 Security Analysis

5.3.1 Security Model and Notation

Definition 5.3.1 (Security of RoUpSPA). We use the ideal/real-world model of UpSPA. In the real world, a non-uniform PPT adversary $\mathcal{A}$ interacts with RoUpSPA and controls the parties allowed by the corruption bounds. In the ideal world, a PPT simulator Sim interacts with $\mathcal{F}_{\text{UpSPA}}$ and produces the view of the corrupted parties. The protocol is secure if the real and ideal views and outputs are computationally indistinguishable for every such adversary.

The adversary is static and malicious and corrupts at most $t_{sp} - 1$ storage providers, $n_{ls} - 1$ login servers, and $n_u - 1$ users. The proof uses the same TOPRF, authenticated-encryption, signature, hash, password-security, and cross-site assumptions as Theorem 4.3.1. We do not treat the whole RoUpSPA execution as one inherited game. Instead, the TOPRF and authenticated-encryption reductions are instantiated below for the actual RoUpSPA calls, while the quorum, counter, tentative-state, active/stale, and synchronization arguments are proved directly for the resilient execution.

Ideal state and local resilience state. The functionality $\mathcal{F}_{\text{UpSPA}}$ contains only Π_{Setup} , Π_{Register} , $\Pi_{\text{Authenticate}}$, $\Pi_{\text{SecretUpdate}}$, and $\Pi_{\text{PwdUpdate}}$ and their logical outcomes. It does not contain provider availability, acknowledgement sets, active/stale flags, the tentative login-server entries used by Registration and Secret Update, the tentative root record used by Password Update, retries, recovery, or synchronization. These values are maintained locally by Sim . Synchronization changes only a real provider replica and never creates a new ideal call.

This separation keeps the cryptographic state and the resilience bookkeeping apart. A provider may be unavailable or stale without changing the logical state held by $\mathcal{F}_{\text{UpSPA}}$. Similarly, a successful synchronization installs a record that already belongs to a completed phase; it does not create another Registration, Secret Update, or Password Update in the ideal world.

Completed phases and execution discipline. The proof adopts the phase order, acceptance predicates, serialization rule, and completion points of Section 5.2. It does not define another protocol execution. *Sim* reports an ideal success exactly when the corresponding real phase reaches the construction's success point; an incomplete, rejected, or discarded execution creates no ideal success. All pending, acknowledgement, retry, and active/stale information remains local to *Sim* and is never added to $\mathcal{F}_{\text{UpSPA}}$.

Login-server availability. The proof uses the bounded honest-login-server assumption stated in Section 5.1.1 and instantiated by the construction. A corrupted login server may reject or deny service. Such behavior affects progress but does not allow *Sim* to report an ideal success before the real completion condition is met.

Counters, signatures, and verification direction. The proof uses the exact identification and account counters, signed strings, and verification directions fixed in Section 5.2. The two counters remain distinct and are advanced only by their corresponding record families. User signatures are checked by honest providers on the writes and synchronization candidates for which the construction requires them. Ordinary reads use authenticated decryption and the construction's counter and active/stale checks; the proof does not add a provider signature or a new verification step.

Simulator state. In addition to the cryptographic tables used to simulate the TOPRF and ciphertexts, *Sim* stores the following protocol state. For every simulated provider and record family, it stores the current identification or account tuple, its counter and user signature, and its active/stale status. For Password Update, it also stores at most one matching tentative identification tuple. For a serialized account write, it records the exact signed tuple and the providers that returned the actual matching acknowledgements. For a serialized password write, it records the exact tentative identification tuple and its matching acknowledgement set. For an honest simulated login server, it stores the current verifier and at most one exact tentative

Registration or Secret Update tuple. No provider-generated signature, final provider receipt, operation identifier, or additional epoch is maintained.

Proof-only call sets. Let q_K be the number of full TOPRF keys honestly generated during Setup. RoUpSPA keeps the same full key and provider shares across Password Update, so the old- and new-password evaluations are two inputs to one keyed function rather than evaluations under two newly generated keys.

Let $\mathcal{T}$ be the ordered set of TOPRF sessions in which an honest user is the receiver, and let $q_T = |\mathcal{T}|$. These sessions occur during Registration, Authentication, Secret Update, and the two TOPRF executions of Password Update. Synchronization performs no TOPRF evaluation.

Let $\mathcal{E}_{id}$ and $\mathcal{E}_{sp}$ be the ordered sets of honest identification and account encryption calls, and define $q_{id} = |\mathcal{E}_{id}|$, $q_{sp} = |\mathcal{E}_{sp}|$. Replication of one ciphertext to several providers does not create several encryption calls. Synchronization only copies an existing ciphertext and therefore creates no member of either set.

5.3.2 Main Security Theorem

Theorem 5.3.2 (Security of RoUpSPA). *Assume that TOPRF satisfies receiver-side PRF security against coalitions of fewer than t_{sp} providers and receiver-input obliviousness against malicious senders. Assume that authenticated encryption is IND – CPA-secure, provides ciphertext integrity, and satisfies the key-binding property required by the construction; Sign is EUF – CMA-secure; and H is collision resistant. Assume further that at most $t_{sp} - 1$ providers are corrupted, $t_{sp} > \frac{n_{sp}}{2}$, $2 \cdot q_{sp} - n_{sp} > t_{sp} - 1$, and that honest executions follow the serialization, terminal Store-or-Discard, and bounded honest-login-server conditions stated above. Then, for completed user-visible phases, RoUpSPA securely realizes $\mathcal{F}_{UpSPA}$ according to Definition 5.3.1. More precisely, for every non-uniform PPT adversary $\mathcal{A}$ corrupting at most $t_{sp} - 1$ providers, $n_{ls} - 1$ login servers, and $n_u - 1$ users, there exists a PPT simulator Sim such that, for every auxiliary input aux,*

$$\{\text{REAL}_{\text{RoUpSPA}, \mathcal{A}}(1^\lambda, \text{aux})\}_{\lambda \in \mathbb{N}} \stackrel{c}{\approx} \{\text{IDEAL}_{\mathcal{F}_{UpSPA}, \text{Sim}}(1^\lambda, \text{aux})\}_{\lambda \in \mathbb{N}}.$$

The theorem is a safety statement. It does not force a corrupted provider or login server to answer, guarantee that q_{sp} or t_{sp} valid responses exist, or provide automatic recovery after permanent loss of an unresolved client state.

Proof overview. The hybrid sequence replaces the TOPRF outputs and the plaintexts of honest identification and account ciphertexts while preserving the public execution fixed by Section 5.2. After these replacements, the remaining proof obligations concern only authorization, counter consistency, active/stale replica selection, quorum intersection, and synchronization.

5.3.3 Simulator Overview

Sim runs $\mathcal{A}$ internally and executes the honest-party algorithms of Section 5.2. Public fields, message lengths, scheduling, response sets, timeouts, and accept/reject outcomes are reproduced without redefining the phase flows. Values supplied by corrupted parties are forwarded unchanged, subject to the checks performed by the simulated honest parties.

For hidden honest values, **Sim** uses the same logical TOPRF and ciphertext tables as in the UpSPA proof. In addition, it stores only the resilience bookkeeping listed above: the current record for each family, any protocol-permitted tentative state, the relevant acknowledgement set, and active/stale status. The corresponding ideal call is made only at the completion point fixed by the construction. Before that point, the state remains local to **Sim**; rejection, abort, or insufficient participation produces the same real-world outcome and no ideal success.

For a corrupted user, **Sim** does not replace the values chosen by $\mathcal{A}$. It applies the construction's honest-provider and honest-login-server checks and invokes the ideal phase only when those checks and the real completion condition are satisfied. For a corrupted provider or login server, **Sim** exposes exactly the view generated by the corresponding real interaction and preserves the adversary's ability to reject, remain silent, or return malformed data.

Synchronization is simulated as storage-layer maintenance over records that already belong to the real execution. **Sim** applies the construction's validity, counter,

conflict, quorum, and active/stale rules, but it makes no ideal call and creates no additional TOPRF or encryption query. Thus, the simulator overview specifies only proof bookkeeping; all protocol mechanics remain exclusively in Section 5.2.

5.3.4 Bad Events

The signature, hash, and authenticated-encryption failures are analyzed before the hybrid sequence. They are not hybrid games and do not change the protocol state when moving from one game to the next.

Signature failure. The event **BadSig** occurs when an honest provider accepts during a write or Synchronization an account or counter-positive identification record under an honest $\mathbf{svk}$, although the exact signed message was not produced by the honest user. The relevant messages are $\text{Uid} \parallel \text{SUid}_{l,j} \parallel \mathbf{c}_j \parallel \mathbf{ctr}_{\text{ac}}$ and $\text{Uid} \parallel \text{cid}_{\text{new}} \parallel \mathbf{ctr}'_{\text{id}}$. Changing the user identifier, account family, ciphertext, or counter creates a new signed message. Exact retransmission of an honestly signed message is not a forgery and is handled by the idempotency and counter rules. Store and Discard are not additional signed records in the present construction. No provider signature is considered, and no user-side read verification is part of **BadSig**.

Hash failure. The event **BadHash** occurs if the adversary finds a collision that changes the binding of $\mathcal{R}^{sp} \parallel \mathbf{ls}_j$ to an account-family identifier or changes the binding of $\mathcal{R}^{\mathbf{ls}_j} \parallel \mathbf{ls}_j$ to a login-server verifier. Outside this event, one shared account identifier refers to the intended account family and one accepted verifier refers to the intended account secret and canonical login server.

Ciphertext failure. The event **BadCtxt** occurs if an honest party accepts a fresh adversarial identification or account ciphertext under an honest logical key, or if a ciphertext honestly created under one logical key is accepted under another key in a way that changes a phase result. This event covers the authenticated-decryption and key-binding checks used by the user when reading records. Because the user does not verify signatures on read responses, ciphertext integrity and the bound plaintext

counter are the checks that exclude a modified returned ciphertext. Replaying an honestly generated ciphertext is not itself a bad event and is handled by the record counter and active/stale rules.

Let q_{dec} be the number of adversarial ciphertext candidates processed under honest keys, q_{kb} the number of cross-key acceptance attempts, and q_{vk} the number of honest verification keys. Define

$$\begin{aligned} \epsilon_{\text{bad}}^{\text{RoUpSPA}}(\lambda) &= q_{\text{dec}} \cdot \text{Adv}_{\text{AE}}^{\text{int-ctxt}}(\mathcal{B}_{\text{ctxt}}) + q_{\text{kb}} \cdot \text{Adv}_{\text{AE}}^{\text{kb}}(\mathcal{B}_{\text{kb}}) \\ &\quad + q_{\text{vk}} \cdot \text{Adv}_{\text{Sign}}^{\text{euf-cma}}(\mathcal{B}_{\text{sig}}) + \text{Adv}_{\text{H}}^{\text{cr}}(\mathcal{B}_{\text{hash}}). \end{aligned}$$

Corollary 5.3.3 (Bad-event bound). *In either the real or ideal RoUpSPA execution,*

$$\Pr[\text{BadSig} \vee \text{BadHash} \vee \text{BadCtxt}] \leq \epsilon_{\text{bad}}^{\text{RoUpSPA}}(\lambda).$$

Proof. An accepted new signed record under an honest verification key gives an EUF – CMA forgery. A collision changing the shared identifier or verifier binding gives a collision for H . Acceptance of a fresh modified ciphertext or acceptance under the wrong logical key gives the corresponding ciphertext integrity or key-binding adversary. A union bound over the polynomially many attempts gives the stated probability. $\square$

In the hybrid sequence below, we condition on the complement of these events. The final advantage bound adds their probability on the real and ideal sides.

5.3.5 Hybrid Games

For each game G_i , let $p_i = \Pr[\mathcal{A} \text{ outputs } 1 \text{ in } G_i]$. Every game preserves the complete public execution and all accept, reject, and completion outcomes defined by Section 5.2. Only the hidden TOPRF and ciphertext values are changed.

G_0 : real RoUpSPA execution. Game G_0 is the real RoUpSPA execution conditioned on $\neg(\text{BadSig} \vee \text{BadHash} \vee \text{BadCtxt})$. Honest parties execute Section 5.2 exactly; no protocol step is restated in the game definition.

G_1 : replace full TOPRF evaluations. For each full TOPRF key honestly generated in Setup, the challenger samples one consistent random function R_K . Every honest receiver output that would be $F_K(x)$ is replaced by $R_K(x)$. The same function is used whenever the same full key is evaluated again. In particular, Password Update uses $R_K(\text{pwd})$ and $R_K(\text{pwd}_{\text{new}})$ for two inputs of the same function; it does not receive values from two independently sampled functions. Provider shares, public transcripts, quorum membership, and every later protocol decision remain as in G_0 .

A distinguisher between G_0 and G_1 yields a receiver-side PRF adversary for one honestly generated full TOPRF key. Guessing the challenged key gives $|p_0 - p_1| \leq q_K \cdot \text{Adv}_{\text{TOPRF}}^{\text{prf}}(\mathcal{B}_{\text{prf}})$.

G_2 : simulate honest-receiver TOPRF transcripts. Game G_2 keeps the random-function outputs of G_1 but replaces each honest-receiver TOPRF interaction by the receiver-input-oblivious simulation. The simulated transcript has the same public shape and produces the same tabled random-function output. The adversary still chooses the behavior of corrupted providers, and all threshold failures and response omissions are reproduced. Only the dependence of the transcript on the honest receiver input is removed.

A distinguisher between G_1 and G_2 yields a receiver-input-obliviousness adversary for one session. A standard session hybrid gives $|p_1 - p_2| \leq q_T \cdot \text{Adv}_{\text{TOPRF}}^{\text{rio}}(\mathcal{B}_{\text{rio}})$.

G_3 : replace honest identification plaintexts. In Game G_3 , each honest encryption producing cid or cid_{new} encrypts an independent same-length random string instead of $\text{ssk} \parallel \mathcal{R}^{sp} \parallel K_0 \parallel \text{ctr}_{\text{id}}$ or its updated form. The challenger keeps the actual logical identification plaintext in its private table and uses that table for later protocol computation and simulated decryption. All public values and protocol outcomes that depend on the ciphertext are generated consistently with the ciphertext appearing in that game.

Replacing the identification encryptions one at a time gives $|p_2 - p_3| \leq q_{\text{id}} \cdot \text{Adv}_{\text{AE}}^{\text{ind-cpa}}(\mathcal{B}_{\text{cpa},\text{id}})$. Replication of one identification ciphertext to several providers

does not create additional hybrids, and Synchronization only copies a ciphertext already created in an earlier call.

G_4 : replace honest account plaintexts. Game G_4 applies the same change to each honest encryption producing c_j or $c_{j,new}$. The encrypted account plaintext $\mathcal{R}^{ls_j} \parallel \text{ctr}_{ac}$ is replaced by an independent same-length random string, while the logical account secret and counter remain in the simulator table. All public values and protocol outcomes that depend on the account ciphertext are generated consistently with the ciphertext and logical state of this game.

A call-by-call hybrid gives $|p_3 - p_4| \leq q_{ac} \cdot \text{Adv}_{\text{AE}}^{\text{ind-cpa}}(\mathcal{B}_{\text{cpa},ac})$. Again, replication and Synchronization do not add encryption calls.

G_5 : ideal execution. Game G_5 is the execution of $\mathcal{F}_{\text{UpSPA}}$ with Sim . The simulator uses the random-function and logical-plaintext tables already present in G_4 and keeps all resilience state locally. The transition from G_4 to G_5 changes no cryptographic distribution. Its correctness depends only on the completed state, counter, active/stale, and quorum properties proved below.

Lemma 5.3.4 (RoUpSPA cryptographic hybrid bound). *Conditioned on the complement of BadSig, BadHash, and BadCtxt,*

$$\begin{aligned} |p_0 - p_4| &\leq q_K \cdot \text{Adv}_{\text{TOPRF}}^{\text{prf}}(\mathcal{B}_{\text{prf}}) + q_T \cdot \text{Adv}_{\text{TOPRF}}^{\text{rio}}(\mathcal{B}_{\text{rio}}) \\ &\quad + q_{\text{id}} \cdot \text{Adv}_{\text{AE}}^{\text{ind-cpa}}(\mathcal{B}_{\text{cpa},\text{id}}) + q_{ac} \cdot \text{Adv}_{\text{AE}}^{\text{ind-cpa}}(\mathcal{B}_{\text{cpa},ac}). \end{aligned}$$

Proof. The bound follows by the triangle inequality from the four game transitions above. The primitive reductions are the same types used in the UpSPA proof, but their query sets and consistency conditions here are exactly those of RoUpSPA. In particular, the unchanged TOPRF key is represented by one random function across both Password Update inputs, and every public state transition remains the deterministic consequence of the same response sets and logical state fixed by the construction. $\square$

5.3.6 Synchronization and Counter Consistency

The remaining statements are deterministic protocol properties. They do not introduce additional cryptographic games.

Lemma 5.3.5 (Authorized writes and terminal decisions). *For an honest user and outside **BadSig**, an honest provider accepts only a record signed for the exact user, record family, ciphertext, and permitted counter. It accepts at most one distinct record for a fixed family and counter, and exact retransmission is idempotent. Registration and Secret Update install the accepted account record before acknowledgment. Password Update keeps the accepted identification record tentative and changes the current identification only on the matching Store; matching Discard removes only that tentative record. No honest Authentication or Synchronization begins before an earlier serialized write for the same family reaches its terminal Store or Discard decision.*

Proof. For account writes, an honest provider verifies the user signature, Registration requires counter zero and account nonexistence, and Secret Update requires the exact current counter plus one. For Password Update, the provider verifies the user signature, requires the exact next identification counter, and keeps at most one matching tentative identification tuple. A distinct accepted tuple under an honest **svkat** the same counter would cause **BadSig**; a skipped or lower counter is rejected by the local rule.

The Store or Discard command in Password Update applies only to the matching tentative identification. Store promotes it and Discard removes it without changing the previous identification. For Registration and Secret Update, Store or Discard applies only to the exact tentative login-server tuple. Serialization prevents an honest read, later write, or Synchronization from being started between the proposal and its terminal decision. A discarded partial account write is not turned into ideal success; replicas containing only that failed write are excluded as stale and may cause an availability failure. $\square$

Lemma 5.3.6 (Authentication from active threshold responses). *Assume fewer than t_{sp} providers are corrupted. If an honest user completes Authentication with at*

least t_{sp} valid active identification responses and at least t_{sp} valid active account responses, then outside the bad events the selected largest-counter records are the latest completed identification and account states. If the target login server is honest, its current verifier matches the selected account record. If either active threshold is unavailable, Authentication fails or aborts.

Proof. Each t_{sp} response set contains an honest provider. An honest active provider returns its current record for the relevant family. It never returns a tentative Password Update identification, and a replica written only by a discarded or failed account execution is treated as stale. A corrupted provider may return an older honestly generated record, but that record has a smaller counter. A modified or fabricated returned ciphertext that passes authenticated decryption and the internal/external counter check causes **BadCtxt**. The honest user does not rely on a returned signature check.

By Lemma 5.3.5, a later serialized write cannot be observed as a completed state before its Store decision. Therefore, the largest acceptable counter among active responses identifies the current completed identification or account state. For Registration and Secret Update, the successful terminal Store makes the exact tentative verifier current at an honest $\mathcal{LS}_j$. Hence, its verifier equals $H(\mathcal{R}^{ls_j} \parallel ls_j)$ for the selected account record. The argument uses an active t_{sp} threshold, not a q_{sp} read quorum. $\square$

Theorem 5.3.7 (Quorum consistency and recovery correctness). *Assume a fixed corruption set of at most $t_{sp} - 1$ providers and $2 \cdot q_{sp} - n_{sp} > t_{sp} - 1$. Consider the identification family indexed by Uid or an account family indexed by $(\text{Uid}, \text{SUid}_{l,j})$. Let rec^* be the latest completed record before a Synchronization begins. For an account record, completion means that q_{sp} providers acknowledged the matching direct write and the user sent the matching login-server Store. For a Password Update identification, completion means that q_{sp} providers acknowledged the matching tentative identification and the user sent the matching provider Store. If Synchronization obtains q_{sp} valid current-record responses and selects the unique valid record with the largest counter, it returns exactly rec^* .*

Proof. Let W be the set of q_{sp} providers that hold rec^* as their current record after the terminal Store, and let R be the later Synchronization response quorum. Then $|W \cap R| \geq 2q_{sp} - n_{sp} > t_{sp} - 1$, so $W \cap R$ contains an honest provider. That provider returns rec^* and never decreases its counter. Serialization excludes a later unresolved write from the considered Synchronization.

A corrupted provider cannot make an honest recovering provider accept a different higher-counter current record without an honest user signature. If the exact signed message was not produced by the honest user, **BadSig** occurs. If the ciphertext or its key binding is modified, **BadCtxt** occurs. An honestly signed later record belongs to a later serialized phase and cannot be a current synchronization candidate before its terminal Store. Therefore, the response quorum contains rec^* and contains no acceptable record with a larger completed counter. The unique-largest-counter rule returns rec^* . $\square$

Corollary 5.3.8 (No rollback after a completed phase). *Under Theorem 5.3.7, successful Synchronization cannot install a identification or account record older than the latest completed record for that family.*

Proof. The theorem returns exactly the latest completed record, whose counter is not smaller than any earlier completed counter. $\square$

Corollary 5.3.9 (No divergent completed records). *Two distinct records with the same record key and counter cannot both become completed through quorums of size q_{sp} .*

Proof. The two acknowledgement quorums intersect in an honest provider. For direct account writes, that provider cannot accept and acknowledge two different records at one counter. For tentative identification writes, it cannot retain and acknowledge two different tentative records at one counter. Thus, two distinct records at that counter cannot both reach the Store condition of a completed phase. $\square$

Safety and liveness boundary. The deterministic lemmas establish safety only when the participation and completion conditions of Section 5.2 are met. The adversary may prevent those conditions by withholding or corrupting responses, which

yields denial of service or an incomplete execution rather than ideal success. Quorum and counter reasoning adds no computational advantage term.

5.3.7 Ideal-World Equivalence

After G_4 , honest TOPRF outputs are represented by consistent random-function values and honest identification and account plaintexts by same-length random strings. The remaining view contains only public protocol fields, adversarially chosen values, scheduling, response sets, counters, replica status, and accept/reject outcomes. **Sim** reproduces this view by executing the authoritative construction and keeping the resilience state outside $\mathcal{F}_{\text{UpSPA}}$. User-visible ideal transitions occur at the same completion points as their real counterparts, while Synchronization creates no ideal call and is governed by Theorem 5.3.7.

Lemma 5.3.10 (Terminal-hybrid equivalence). *Conditioned on the complement of the bad events and on completed user-visible phases, G_4 and the ideal execution G_5 have identical distributions. Hence, $|p_4 - p_5| = 0$.*

Proof. The random-function and logical-plaintext tables give every hidden honest value in G_4 the same distribution as the value maintained by **Sim** in G_5 . Public messages and deterministic checks are reproduced exactly. Lemma 5.3.5 establishes authorized state transitions, Lemma 5.3.6 establishes current-state reads, and Theorem 5.3.7 establishes synchronization correctness. Therefore, **Sim** invokes each ideal transition at the completion point fixed by the construction and reproduces the remaining resilience behavior locally. $\square$

5.3.8 Final Advantage Bound

Let

$$\begin{aligned} \text{Adv}_{\text{RoUpSPA}, \mathcal{F}_{\text{UpSPA}}}^{\text{real/ideal}}(\mathcal{A}) = & \left| \Pr[\text{REAL}_{\text{RoUpSPA}, \mathcal{A}}(1^\lambda, \text{aux}) = 1] \right. \\ & \left. - \Pr[\text{IDEAL}_{\mathcal{F}_{\text{UpSPA}}, \text{Sim}}(1^\lambda, \text{aux}) = 1] \right|. \end{aligned}$$

Combining Corollary 5.3.3, Lemma 5.3.4, and Lemma 5.3.10 gives

$$\begin{aligned} \text{Adv}_{\text{RoUpSPA}, \mathcal{F}_{\text{UpSPA}}}^{\text{real/ideal}}(\mathcal{A}) &\leq 2 \cdot \epsilon_{\text{bad}}^{\text{RoUpSPA}}(\lambda) + q_K \cdot \text{Adv}_{\text{TOPRF}}^{\text{prf}}(\mathcal{B}_{\text{prf}}) \\ &\quad + q_T \cdot \text{Adv}_{\text{TOPRF}}^{\text{rio}}(\mathcal{B}_{\text{rio}}) + q_{\text{id}} \cdot \text{Adv}_{\text{AE}}^{\text{ind-cpa}}(\mathcal{B}_{\text{cpa}, \text{id}}) \\ &\quad + q_{\text{ac}} \cdot \text{Adv}_{\text{AE}}^{\text{ind-cpa}}(\mathcal{B}_{\text{cpa}, \text{sp}}). \end{aligned}$$

All query counts are polynomial. The stated primitive assumptions make every computational term negligible. Quorum intersection, exact-counter acceptance, active/stale bookkeeping, terminal Store/Discard processing, and Synchronization consistency are deterministic and add no advantage term. Therefore, $\text{Adv}_{\text{RoUpSPA}, \mathcal{F}_{\text{UpSPA}}}^{\text{real/ideal}}(\mathcal{A}) \leq \text{neg}(\lambda)$, which proves Theorem 5.3.2.

Password-security scope. As for UpSPA, the theorem does not make a human password uniformly random. It states that fewer than t_{sp} corrupted providers, together with the permitted corrupted users and login servers, obtain no additional offline verification predicate beyond the leakage and online outcomes represented by $\mathcal{F}_{\text{UpSPA}}$. The resilience changes affect availability, tentative/current replica state, and synchronization; they do not change the underlying password-security argument.

5.4 Alternative State-Synchronization Mechanisms

The previous sections define the quorum-based RoUpSPA construction and analyze its security under the selected resilience model. Quorum synchronization is our main design choice because it directly addresses storage-provider unavailability without introducing a global public component. However, resilience can be achieved through different synchronization mechanisms, and each mechanism shifts the cost, assumptions, and attack surface in a different way. Therefore, studying alternative designs is useful not only as a design exercise, but also as a way to measure the efficiency–security tradeoff of resilient state recovery.

In this section, we discuss two alternative synchronization directions: a public-bulletin-board-based design and a blockchain-based design. The $\mathcal{PBB}$ -based approach provides an append-only recovery layer that unavailable providers can later read to catch up. We consider both an end-to-end authenticated untrusted $\mathcal{PBB}$ and

a DKG-based authorized $\mathcal{PBB}$, since they represent different points in the trade-off between cryptographic trust, admission control, and denial-of-service resistance. We then discuss a blockchain-based variant, where the ledger plays the role of a public ordering and availability anchor. These alternatives are not used as the main construction, but they clarify which costs and assumptions are avoided by the quorum-based design and which benefits may be obtained by using a stronger public synchronization layer.

5.4.1 PBB-based Synchronization

First candidate solution is to use a public bulletin board, denoted by $\mathcal{PBB}$, as an append-only synchronization and recovery layer. After a successful registration or update, the user posts the required related materials to the $\mathcal{PBB}$. A storage provider that missed the update or registration can later read the corresponding $\mathcal{PBB}$ entry or entries, verify that the entry is authorized, and update or write its local state without requiring interaction with the user. This follows the standard role of append-only bulletin boards in verifiable systems: once a message is posted, it should remain visible and should not be silently removed or changed [32].

Importantly, the $\mathcal{PBB}$ is not used to store plaintext secrets and is not used as a password verifier. Its role is limited to availability, synchronization, and recovery. Password-related information posted to the board is encrypted and authenticated. Thus, the $\mathcal{PBB}$ may help unavailable providers catch up, but it does not learn the user's password, login-server-specific secrets, or password-protected state.

However, the PBB design adds a strong availability assumption. During registration and update, the user must be able to post to the board. During provider recovery, the provider must be able to read the board. Therefore, if the $\mathcal{PBB}$ is unavailable, censored, overloaded, or partitioned, the system can lose exactly the resilience property that the board was introduced to provide. This is the main reason we do not choose PBB as the primary solution.

PBB variant 1: End-to-end authenticated untrusted PBB

In the first and cryptographically cleaner variant, the $\mathcal{PBB}$ is treated as an untrusted append-only log. It may accept arbitrary messages, including malformed, duplicate, stale, or adversarially generated records. Authenticity is enforced end-to-end by the user. During setup, the user generates a signing key pair $\langle \text{ssk}, \text{svk} \rangle$. Any password-related record, such as the identification-secret ciphertext cid or the login-server-secret ciphertext c_j , is signed by user $\mathcal{U}$ under ssk before being posted to the $\mathcal{PBB}$. Before applying a write or update obtained from the $\mathcal{PBB}$, a storage provider verifies the record using svk .

The storage provider must also check freshness. The counter must increase, the phase must be correct, and, when required, the previous-state record must match the provider's current state or be reconstructable from a valid chain of board entries. Therefore, even if the $\mathcal{PBB}$ is malicious or accepts fake records, honest providers ignore messages that do not carry a valid user signature and a valid counter.

This variant gives the strongest security model for PBB because the $\mathcal{PBB}$ is not trusted for authenticity or authorization. Its compromise does not allow the adversary to forge an accepted user update. The main drawback is denial of service. The board may be flooded with invalid messages. Providers may need to scan many records, verify many signatures, and discard many invalid entries. Therefore, flooding is not a successful forgery attack, but it is an efficiency and availability attack.

PBB variant 2: DKG-based authorized PBB

In the second variant, the $\mathcal{PBB}$ is implemented as an authorization-enforcing service, for example by a committee of board servers. During registration, the user and the $\mathcal{PBB}$ establish an account-specific write-authorization mechanism. This may be obtained through a distributed key generation procedure or a threshold authorization mechanism. DKG is a standard tool for threshold cryptographic systems, but DKG itself carries nontrivial assumptions and subtle security require-

ments. For example, Gennaro, Jarecki, Krawczyk, and Rabin showed that a widely used Pedersen-style DKG protocol does not guarantee uniformly random generated keys, and they proposed stronger protocols satisfying DKG security requirements [24].

In this DKG-based PBB model, the board rejects unauthorized writes before storing them. Thus, an adversary cannot easily fill the $\mathcal{PBB}$ with arbitrary junk records. This makes the DKG-based variant useful as an efficiency and availability optimization. Storage providers do not need to scan a large number of invalid board records, and the $\mathcal{PBB}$ can enforce one accepted update per user, phase, and counter.

However, this improvement comes with a stronger assumption. The $\mathcal{PBB}$ is no longer merely an untrusted append-only log. It becomes part of the write authorization path. If storage providers rely only on the board's admission decision and do not verify user signatures, then security depends on the correctness and availability of the $\mathcal{PBB}$ committee. A malicious or unavailable board may censor valid updates or, if the authorization mechanism is broken, allow invalid ones. For this reason, the conservative version combines both approaches: the board may use DKG-based admission control to reduce flooding, but providers still verify the user's signature before accepting a record.

The PBB approach is attractive because it gives automatic catch-up. A provider that missed registration or update can read the board later and repair itself without user interaction. This is stronger than lazy repair in the quorum design. However, PBB also introduces a new global component. The board must be available during registration, update, and recovery. If the board is unavailable, then every user depending on it is affected. This is different from quorum, where only enough storage providers need to be available.

PBB also adds inefficiency. Every registration and update needs a board write. The board grows over time. Providers must fetch and verify board entries. In the untrusted variant, invalid-message flooding can increase scanning and verification cost. In the DKG-authorized variant, the board committee must run authorization

protocols and maintain availability. Therefore, PBB improves autonomous recovery but pays with global infrastructure, extra storage, extra verification, and a stronger liveness dependency.

PBB can also create security damage if it is used as the main solution without care. By security damage here we do not mean that plaintext secrets are exposed under the correct construction. Under the correct construction, the board sees only encrypted and authenticated material. The damage is additional attack surface and metadata leakage. The board can reveal that an account was updated, when it was updated, how often it was updated, and possibly which encrypted records belong to the same user unless identifiers are carefully hidden. If password-related ciphertexts are placed on a public board permanently, any future cryptographic weakness or implementation mistake becomes more damaging because the adversary has a durable public archive. If the board is authorization-enforcing, compromise or censorship of the board can also block valid updates. For these reasons, PBB is useful as an alternative and as an implementation comparison, but not as the main design.

5.4.2 Blockchain-based Synchronization

The second candidate solution is blockchain-based synchronization. A blockchain can be viewed as a public append-only ledger. In that sense, it is similar to a PBB. A user could publish update commitments, counters, signed records, or pointers to encrypted recovery bundles on-chain. Providers that missed registration or update could later query the chain and catch up. Bitcoin's original design already uses digital signatures and peer-to-peer consensus to avoid a trusted third party [54]. More generally, modern blockchain systems provide public replicated ledgers, but the exact consistency, liveness, cost, and finality properties depend on the concrete chain.

However, blockchain is not chosen as the main solution because it complicates

both proof and implementation. First, the proof must model blockchain finality. A transaction is not simply “written” when it is submitted. Depending on the chain, it may be pending, included, reorganized, or finalized. For example, Ethereum proof-of-stake finality depends on validators and checkpoint finalization, so the security proof must either wait for finality or explicitly tolerate reorganization ¹. Second, every on-chain operation has cost. On Ethereum, computation and transactions are paid using gas, and more complex smart-contract operations consume more gas ². Third, a public chain creates privacy pressure because on-chain actions are public and permanent by default, and patterns of activity may reveal sensitive information even when real-world names are not directly written on-chain ³.

A blockchain-based solution also creates a new security boundary. If the smart contract stores the latest counter, then the contract must be correct. If the contract verifies signatures, then the signature scheme and contract implementation must be correct. If the contract stores ciphertexts or pointers, then the system must specify what happens when off-chain payloads disappear. If the chain censors transactions or fees become too high, registration and update may be delayed. If a permissioned blockchain is used instead of a public one, then the trust assumption becomes similar to a BFT committee.

5.4.3 Implementation comparison.

We implement the quorum-based construction as the main resilient design and compare it with the PBB-based and blockchain-based variants only as alternative synchronization layers. For the PBB case, we evaluate the untrusted end-to-end-authenticated PBB and the DKG-based authorized PBB in terms of registration success under provider unavailability, update success, repair of missed providers, bandwidth, storage overhead, and signature-verification cost; however, we do not

¹<https://ethereum.org/developers/docs/consensus-mechanisms/pos/>

²<https://ethereum.org/developers/docs/gas/>

³<https://ethereum.org/privacy/>

model PBB flooding or denial-of-service in the implementation. For the blockchain-based variant, the goal is not to make blockchain the main construction, but to understand whether a ledger can be used with reasonable efficiency by storing only compact commitments or metadata on-chain, keeping the encrypted recovery material off-chain, and using the chain only as an ordering and availability anchor. This comparison allows us to separate the cost of the core UpSPA operations from the cost introduced by each synchronization mechanism. The blockchain-based version should use the following tricks.

1. **Store only commitments on-chain.** The chain should not store password-related ciphertexts unless absolutely necessary. It should store only a compact commitment such as $H(record)$, the counter, the phase, and possibly a pointer to an encrypted off-chain bundle.
2. **Keep encrypted recovery bundles off-chain.** The actual encrypted update material can remain with storage providers or an external object store. The blockchain only authenticates which bundle is the latest authorized one.
3. **Use Merkle batching.** Instead of one transaction per user update, many updates can be batched into a Merkle tree. The chain stores one Merkle root, and providers receive a Merkle proof for the user's update. This reduces on-chain cost.
4. **Use lazy chain checks.** A storage provider should not query the chain for every login if its local counter is fresh. It should query only when the user presents a higher counter, when local state is missing, or when there is a conflict.
5. **Use signatures outside the chain.** The user signs the update record off-chain. The provider verifies the signature locally. The chain is used as an availability and ordering anchor, not as the only verifier.

6. **Use layer-2 or permissioned-chain experiments separately.** A layer-2 or permissioned blockchain may reduce cost and latency, but it changes assumptions. Therefore, results must be reported separately: public chain security assumptions are not the same as permissioned-chain committee assumptions.
7. **Avoid putting stable user identifiers on-chain.** The on-chain commitment should not directly reveal the user identity, login-server identity, or storage-provider mapping. High-entropy account-specific identifiers and unlinkable commitments should be used.
8. **Define finality policy.** The implementation must define when a blockchain update is considered committed. For example, it may wait for finality on a proof-of-stake chain, or wait for a fixed confirmation depth on a probabilistic-finality chain. This finality policy must be reflected in the performance results.

Part III

Benchmarking and Results

Chapter 6

ENGINEERING THE SPA FAMILY: FROM SPA AND TSPA TO UpSPA

6.1 Introduction

This chapter reports the implementation and evaluation that support the formal constructions. The protocol definitions in Sections 4.2 and 5.2 are authoritative: this chapter does not redefine phase order, signed messages, record-completion conditions, or security semantics. It explains how those constructions and the comparison schemes are mapped to software, storage, serialization, and measurement targets.

The implementation is divided across three code bases: for the UpSPA–TSPA ¹ experiments, for the unified threshold-scheme comparison ², and for quorum, *PBB*, blockchain, and gas experiments ³. The descriptions below were checked against repository revisions 0a555127, 3e12c6ee, and ea54a0f8, respectively; a final archival release should preserve the exact data-producing revisions together with the raw result files.

For comparability, the timing harness measures completed successful paths and isolates cryptographic work, provider work, serialization, modeled transport, and backend overhead. Retry loops, permanent client loss, production crash recovery, and liveness-only exchanges are not imposed uniformly on comparison schemes that do not define them. Consequently, the latency results are implementation measurements, not an additional liveness theorem and not a replacement for the completion rules of Chapters 3 and 4.

¹https://github.com/rezasaadi/UpSpa_benchmark

²<https://github.com/rezasaadi/Threshold-Password-Authentication-Protocols>

³https://github.com/rezasaadi/UpSPA_resilient_version

The evidence is reported in three groups. The first compares UpSPA and TSPA and then compares UpSPA with PASTA-U and AugSSO. The second evaluates RoUpSPA and the quorum, signed- $\mathcal{PBB}$, DKG-authorized- $\mathcal{PBB}$, direct-chain, and Merkle-batched synchronization backends. The third reports receipt-level EVM gas, a separate Foundry snapshot, and the status of the TVM-compatible TRON implementation. Measurements from different groups are kept separate when their interfaces, cryptographic groups, network models, or resource units differ.

6.2 Implementation Scope and Design Goals

The implementation targets the cryptographic state machine and its benchmark services rather than a production password manager. It includes client and provider computation, login-server adapters, encoded records, persistence, controlled failure injection, and reproducible result generation. Browser secure-input handling, production TLS termination, account-recovery policy, multi-tenant rate limiting, monitoring, backup, and operational key management remain outside the measured boundary.

A strict separation is maintained between protocol fields and implementation metadata. Identifiers, ciphertexts, counters, signatures, and update messages are included only when they appear in the corresponding construction. Database row identifiers, timestamps, benchmark sample identifiers, local status flags, transport envelopes, and diagnostic phase tags are implementation-only metadata. They are not treated as new protocol fields and are not included in a protocol hash or signature unless the construction explicitly binds them.

Each run records the scheme, phase, n_{sp} , t_{sp} , target, network profile, warm-up count, sample count, and random seed. Resilient runs additionally record q_{sp} , the exercised unavailable-provider count, and the selected synchronization backend. The primary statistic for phase P , target X , and configuration c is

$$T_{X,p50}^P(c) = \text{median}_{r \in [R]} \left(t_{X,\text{end}}^{P,r} - t_{X,\text{start}}^{P,r} \right).$$

Ranges in summary tables are ranges of configuration medians, not ranges over all raw repetitions. Client, provider, and full targets are sampled independently and

their medians are not assumed to add.

The LAN and WAN series combine measured computation with fixed network profiles; they are controlled composed latencies rather than geographically deployed socket measurements. Ledger execution is reported separately in its native unit. EVM gas, TRON Energy, TRON Bandwidth, confirmation delay, and CPU time are never converted into one aggregate metric.

6.3 Shared Software Architecture and Component Decomposition

The shared architecture contains a client engine, login-server adapter, storage-provider service, threshold-cryptography module, state codec, synchronization adapter, and benchmark recorder. The client engine invokes the phase routine defined by the relevant protocol module; it does not embed a second copy of the construction. The login-server adapter stores and compares the verifier required by that routine. The provider service exposes typed record and TOPRF operations, while the threshold module performs response validation and reconstruction.

The state codec converts typed values into canonical byte strings before they are hashed, signed, encrypted, stored, or measured. The synchronization adapter is pluggable: quorum, signed- $\mathcal{PBB}$, DKG-authorized- $\mathcal{PBB}$, direct-chain, and Merkle-batch implementations export the same measurement-facing interface, but keep their native authorization, recovery, and resource costs separate. The benchmark controller starts services, supplies configurations, injects the selected failures, and collects outputs; it is not a protocol party.

Provider requests within one implemented stage are issued in parallel. For a participating set I , the composed stage cost is represented as

$$T_{\text{stage}}^P(I) = T_{\text{client,pre}}^P + \max_{i \in I} \{L_i + T_{\text{SP}_i}^P\} + T_{\text{client,post}}^P,$$

where L_i is the configured transport contribution. Successive stages remain sequential. This stage-based model is used only to compose the implemented benchmark schedule; it does not redefine the formal phase.

The recorder stores raw intervals, encoded byte counts, configuration metadata, and backend-native receipts before aggregation. Public-state runners also retain

transaction identifiers, block metadata, confirmation state, artifact hashes, and the unmodified receipt so that every summary row remains traceable to its source.

6.4 Cryptographic Primitives and Parameter Choices

The WPES UpSPA–TSPA implementation uses Ristretto255 over Curve25519 and BLAKE3. Its manifest specifies `curve25519-dalek 4`, `blake3 1`, `chacha20poly1305 0.10`, `ed25519-dalek 2`, `rand_chacha 0.3`, `rand_core 0.6`, `aes 0.8`, `ctr 0.9`, and `criterion 0.5`. Thus, update signatures are instantiated with Ed25519 through `ed25519-dalek`. Because that repository does not contain a lock file at the checked revision, the chapter reports the manifest constraints rather than unsupported patch versions.

The unified comparison uses Rust 1.93.0 through its toolchain file, has a language floor of Rust 1.75, and is built with `--locked --release`. Its UpSPA and quorum crates use the same Ristretto255, BLAKE3, XChaCha20–Poly1305, Ed25519, and ChaCha-based RNG families. The PASTA-U crate additionally uses BLS12-381, while AugSSO uses BLS12-381 and X25519. The resilient repository retains locked dependency sets for its quorum, *PBB*, and blockchain timing targets.

The cryptographic interface follows the syntax fixed in the protocol chapters. In particular, this chapter does not add an authenticated-data field, a root counter, an account signature, or another signed tag to UpSPA when the construction does not contain that field. The Rust benchmark nevertheless passes the byte strings `Uid || “|cipherid”` and `Uid || “|ciphersp”` through the AEAD associated-data interface. These strings are implementation-side context labels; they are not new protocol records or signed fields, and the formal claims in Chapter 3 do not rely on them. Backend-specific EIP-712 domains, Merkle leaf prefixes, database tags, and benchmark labels are likewise reported as backend or codec metadata rather than as fields of the core protocol.

For XChaCha20–Poly1305, the implementation container is

$$\text{CipherBlob} = (\text{nonce}_{24}, \text{ct}, \text{tag}_{16}).$$

The encryption routine fills the 24-byte nonce through the supplied `RngCore` in-

terface and stores the nonce and 16-byte authentication tag beside the ciphertext. Benchmark fixtures use a deterministically seeded `ChaCha20Rng` so that configurations are reproducible; this benchmark choice is not a deployment nonce policy. A deployment must use an appropriate cryptographic random source and must not repeat a nonce under one key. The container fields are implementation metadata for the authenticated-encryption algorithm and do not change the protocol plaintext.

Text values are passed as UTF-8 byte strings. The benchmark expects the trusted client to supply the canonical login-server label `lsj` fixed by Chapters 3 and 4; it does not implement a second domain-canonicalization policy. Fixed-width values use fixed byte lengths, and provider indices and counters are encoded in little-endian form by the Rust implementation. Context prefixes separate hash and encryption uses. A production transport encoding should length-prefix every variable-length field while preserving the field order of the authoritative construction.

Threshold sharing and TOPRF operations use the implementations associated with the selected benchmark. Their security definitions are given in the preliminaries; here the implementation records only the concrete group, threshold, response count, and time spent in evaluation and reconstruction. The receipt-level EVM registry additionally uses EIP-712 authorization and supports ERC-1271 controllers, while the current TVM registry authorizes direct writes through caller binding. These platform-specific choices are not pooled with the Rust cryptographic timings.

6.5 Implementing UpSPA and Threshold Variants

The TSPA module implements the registration and authentication baseline of [35]. Its software state records provider-response arrays, encoded encrypted shares, OPRF and reconstruction timings, and the login-server verification result. The module is kept separate from UpSPA so that its message sizes, provider work, and measured phase boundaries come from its own executable target rather than from a reconstruction of UpSPA components.

The UpSPA module invokes the five routines defined in Section 4.2. Its soft-

ware decomposition mirrors the two record families without repeating their protocol flow. A provider's committed root tuple is $\text{UpSPARoot}_i = (\text{Uid}, \text{svk}, \text{cid}, \text{k}_i)$, and its account tuple is $\text{UpSPAAccount}_{i,j} = (\text{SUid}_{i,j}, \text{c}_j)$. The account counter remains inside the encrypted account plaintext, and the password-update authorization tuple is handled by the password-update routine rather than being added to every persistent account record. In particular, UpSPA secret update is not described as a signed account write, and no root counter is added to the fully available construction.

The current benchmark snapshot measures one Ed25519 signature per provider over the new root ciphertext, new TOPRF share, timestamp field, and provider index; it uses a fixed timestamp and does not include Uid in the timed byte string. Section 4.2 is authoritative and additionally binds Uid and requires the freshness check stated there. Therefore, the reported value is a signature-cost benchmark rather than a byte-for-byte interoperability claim. A conforming deployment must sign the exact message of Section 3.2 and supply a live freshness value. This correction changes the encoded message length, not the number or type of measured signature operations, so no chart is altered.

The unified threshold-scheme repository also exposes the lightweight `pwdupd_v2` target, which retains the existing TOPRF shares and uses one signature. It is the implementation variant used in the unified comparison and is not substituted for the fully available password-update construction of Chapter 3. Accordingly, its rows are reported as executable comparison data, while protocol correctness and authorization continue to refer to the exact construction selected by the corresponding chapter.

Provider work is exposed as an aggregate SP target and as part of the composed full target. Requests inside one provider stage are parallel, and the full target executes the implemented stages in repository order. For schemes A and B measured at the same phase, target, configuration, and network profile, relative results use

$$\text{speedup}_{A/B}^P = \frac{T_{B,p50}^P}{T_{A,p50}^P}, \quad \text{reduction}_{A/B}^P = 100 \cdot \left(1 - \frac{T_{A,p50}^P}{T_{B,p50}^P} \right).$$

No ratio is formed across different targets or parameter grids.

The PASTA-U and AugSSO modules retain their own token-generation and up-

date interfaces. Their reported values cover the complete successful benchmark phase used by their repository. The harness does not claim that these systems and UpSPA have identical relying-party, recovery, or liveness semantics; it normalizes hardware, sampling, thresholds, and network profiles to compare executable cost.

6.6 Update-Enabled and Resilient Variants

The update handlers call the formal UpSPA and RoUpSPA routines and expose only implementation concerns: record encoding, persistence, provider parallelism, byte counts, and timing boundaries. Secret update and password update remain separate handlers because they touch different record families; this chapter does not restate their cryptographic steps.

The RoUpSPA module uses the changed fields defined in Section 5.2. Providers use the shared account identifier $\text{SUID}_{l,j}$, external account and root counters where prescribed, and user signatures only on the records signed by the construction. The module tracks the provider set contributing to each measured execution and invokes the selected synchronization adapter for recovery experiments.

The resilient comparison intentionally relaxes nonuniform liveness machinery. The timed samples cover completed successful executions and controlled recovery cases. Retry duration, unbounded delay, permanent client failure, provider replacement, and production crash repair are excluded unless a dedicated backend target measures them. This matches the scope of the comparison schemes, many of which do not define the same liveness interface, while leaving the security and completion claims of Chapter 4 unchanged.

For phase P , configuration c , and network profile N , the isolated robust increment is reported only for paired runs:

$$\begin{aligned}\Delta_{Q,p50}^P(c) &= T_{\text{RoUpSPA,full-no-net,p50}}^P(c) - T_{\text{UpSPA,full-no-net,p50}}^P(n_{sp}, t_{sp}), \\ \Delta_{Q,N,p50}^P(c) &= T_{\text{RoUpSPA,full-net,p50}}^P(c, N) - T_{\text{UpSPA,full,p50}}^P(n_{sp}, t_{sp}, N).\end{aligned}$$

The subtraction is not used when phase, n_{sp} , t_{sp} , or network profile differs.

The signed- $\mathcal{PBB}$ adapter records append, read, scanned-entry, signature-verification, and installation costs. The DKG-authorized board adds its admission

cost. Direct-chain mode records submission and confirmation; Merkle mode separately records root publication, proof generation, proof size, off-chain verification, and optional transactional verification. Encrypted recovery bundles remain off chain, and public records contain only opaque tags, commitments, counters, phases, and bundle hashes.

6.7 State Management, Record Formats, and Persistence

The benchmark state is divided into protocol records and measurement data. For fully available UpSPA, a provider stores the root tuple $(\text{Uid}, \text{svk}, \text{cid}, \text{k}_i)$ and the account tuple $(\text{SUid}_{l,i,j}, \text{c}_j)$, while the login-server adapter stores $(\text{Uid}, \text{vInfo}_j)$. The account counter is inside the encrypted account plaintext. No external root counter, generic phase field, or account-update signature is attributed to the Chapter 3 record unless it appears in Section 4.2.

The RoUpSPA structs use the shared account identifier, counters, signatures, and replica information required by Section 5.2. Backend bookkeeping—for example, the selected live-set bitmap, sample identifier, timing label, encoded length, transaction identifier, or encrypted-bundle pointer—is maintained separately and is not treated as a cryptographic protocol field.

The Rust provider targets use in-memory maps. Their lookup and replacement measure local map operations, not disk latency, write-ahead logging, replicated database transactions, or crash recovery. An update to one map entry occurs inside one local function call, but this must not be read as a durable atomicity claim across the client, login server, and providers. Benchmark summaries are written as UTF-8, whitespace-separated `.dat` rows; passwords, decrypted root plaintexts, and account plaintexts are not written to those result files.

An encoded XChaCha blob stores its nonce, encrypted body, and tag. Public-state experiments additionally retain the canonical off-chain record, encrypted recovery bundle, bundle-location metadata, batch identifier, leaf encoding, and Merkle proof required to verify the published commitment. EVM and TVM state remain platform specific, and Merkle batching reduces public per-record storage without

removing the off-chain bundle or its availability requirement.

6.8 Service Interfaces, Deployment Model, and Reproducibility

The supplied Rust artifacts expose library calls and command-line benchmark runners; they do not implement REST, HTTP, gRPC, JSON, or Protocol Buffers. Client, login-server, provider, and synchronization adapters exchange typed Rust values and canonical byte arrays inside the benchmark process. A future deployment may wrap these calls in authenticated network services, but no production endpoint, status-code map, transport authentication policy, or retry guarantee is measured here.

The benchmark controller resolves the parameter grid, creates deterministic fixtures, selects the unavailable providers, executes warm-up and measured iterations, and writes the raw row. It is measurement infrastructure rather than a protocol party. The non-ledger output is UTF-8 and whitespace separated; each row records the scheme, target, operation, network profile, RNG flag, n_{sp} , t_{sp} , q_{sp} where applicable, unavailable-provider count, sample counts, and the reported distribution statistics.

For phase P , encoded communication is

$$C_{\text{net}}^P = \sum_{m \in \mathcal{M}^P} |\text{encode}(m)|.$$

The LAN/WAN runner combines these byte counts with the configured stage model, propagation, jitter, bandwidth, and calibrated provider processing. It does not replace the formal phase schedule. EVM gas, TRON Energy, TRON Bandwidth, SUN fees, CPU time, and confirmation latency are summarized independently.

A reproducible archive should retain the repository revision, complete command, Rust toolchain, locked dependency file where present, release-build settings, hardware and operating system, network parameters, seed policy, raw result file, and plotting script. Ledger rows additionally retain contract address, chain identifier, transaction and block metadata, status, confirmation or solidification state, and the untouched receipt.

6.9 Testing, Benchmarking, and Validation

6.9.1 Correctness and Measurement Validation

A sample is timed only after the implementation output satisfies the relevant construction or comparison-scheme check. For UpSPA and RoUpSPA, the reference behavior is taken from Sections 4.2 and 5.2; Chapter 5 does not add an acceptance condition. Wrong passwords, insufficient threshold responses, invalid ciphertexts or signatures, mismatched identifiers, and stale or unavailable records are retained as failed diagnostics rather than successful samples.

The harness performs 50 warm-up iterations and records 200 measured samples for the Rust latency series. Random-value generation is outside the timed region. Plotted points and latency-table entries are medians. Client, SP, and full targets are independent measurements, so their $p50$ values are not algebraically added or rejected when small sampling inversions occur.

6.9.2 Common Benchmark Environment and Network Profiles

The WPES and unified Rust experiments use a 64-bit Windows laptop with an 11th Gen Intel Core i7-1185G7 CPU at 3.00 GHz, 16 GB RAM, and an NVIDIA GeForce RTX 2060 GPU. The GPU is reported as hardware metadata but is not used for the measured protocol operations. The UpSPA–TSPA and threshold-comparison sources are maintained in the repositories cited above. The resilient/PBB/blockchain implementation is kept in the separate third code base.

The LAN model uses a 0.5 ms RTT, 0.05 ms jitter, and 1000 Mbps bandwidth. The WAN model uses a 60 ms RTT, 5 ms jitter, 50 Mbps bandwidth, and 64 bytes of message overhead. The resulting values are composed end-to-end estimates, not measurements from geographically deployed VPSs.

The stage labels in the WPES table describe the simplified successful-path schedule used by that benchmark repository. They exclude retry, crash-recovery, and liveness-only exchanges that were not implemented uniformly across the comparison schemes. They must therefore be read as *modeled benchmark stages*, not as formal protocol-round counts and not as a replacement for the message diagrams in

Chapters 3 and 4.

6.9.3 UpSPA and TSPA comparison

The deployment grid is

$$(n_{sp}, t_{sp}) \in \{(3, 2), (6, 2), (6, 3), (6, 6), (10, 2), (10, 4), (10, 8), (10, 10)\}.$$

Table 6.1 reports ranges of configuration medians. Its last column gives the modeled-stage count defined above.

Phase	Scheme	Client	SP	LAN (full)	WAN (full)	Modeled stages
Setup	UpSPA	0.028–0.038	–	0.571–0.621	63.90–66.77	1
Registration	TSPA	0.058–0.184	$< 10^{-3}$	0.598–0.766	63.98–66.97	1
Registration	UpSPA	0.086–0.222	0.039–0.192	1.182–1.368	126.28–132.75	2
Authentication	TSPA	0.056–0.197	0.039–0.195	0.608–0.792	62.29–66.65	1
Authentication	UpSPA	0.083–0.218	0.039–0.192	1.168–1.292	125.26–128.93	2
Secret update	UpSPA	0.081–0.208	0.039–0.193	1.704–1.869	188.03–195.00	3
Password update	UpSPA	0.140–0.337	0.099–0.365	1.276–1.537	125.89–134.21	2

Table 6.1: UpSPA and TSPA deployment-grid median-latency ranges (ms); the last column reports the simplified successful-path stage model.

Local computation remains below one millisecond over the deployment grid. The larger WAN differences follow the implemented stage count: one modeled stage is about 63–67 ms, two stages are about 125–134 ms, and three stages are about 188–195 ms. Password update is therefore below secret update on WAN even though its local computation is higher.

Client registration and authentication at fixed $n_{sp} = 30$. Figure 6.1 isolates threshold sensitivity. UpSPA registration grows from 0.071 to 0.564 ms and remains below TSPA registration; authentication converges at $t_{sp} = 30$, where the medians are 0.574 and 0.583 ms.

Client setup and updates at fixed $n_{sp} = 30$. Figure 6.2 shows setup increasing from 0.032 to 0.128 ms, secret update from 0.071 to 0.577 ms, and password update from 0.374 to 0.945 ms. The isolated middle-sweep reversals do not coincide with a code-path change and are treated as ordinary median variation.

LAN registration and authentication at fixed $n_{sp} = 30$. Figure 6.3 exposes the additional implemented network stage: UpSPA registration spans 1.206–1.769 ms and authentication 1.198–1.783 ms, whereas the corresponding TSPA curves remain lower over this fixed-provider sweep.

LAN setup and updates at fixed $n_{sp} = 30$. Figure 6.4 separates local work from stage count. Setup remains 0.665–0.760 ms, secret update 1.821–2.285 ms, and password update 1.576–2.241 ms. The shorter benchmark schedule keeps password update below secret update over most of this sweep.

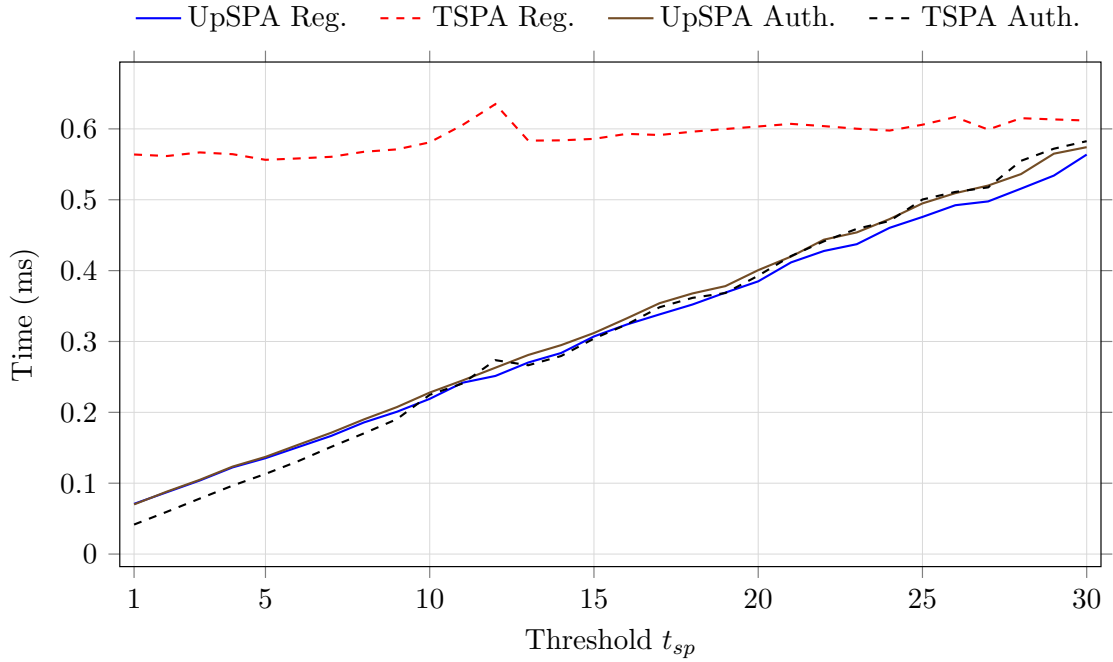
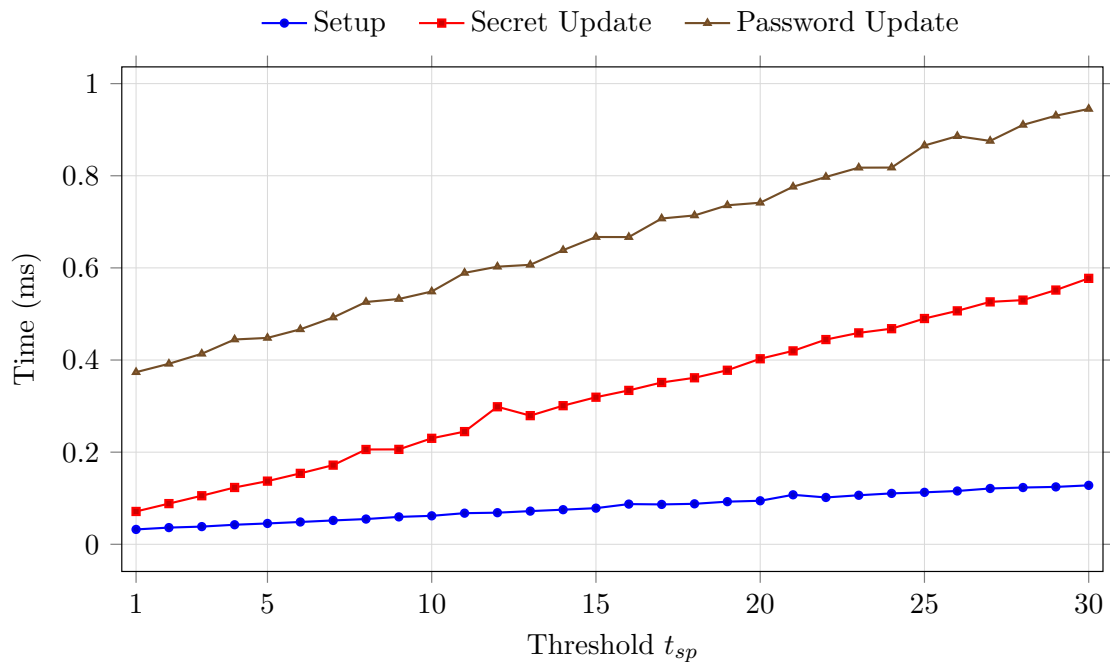


Figure 6.1: Client registration and authentication, $n_{sp} = 30$.

Client registration and authentication with $t_{sp} = 0.6 \cdot n_{sp}$. Figure 6.5 shows the joint effect of increasing provider count and threshold. From $n_{sp} = 10$ to 100, UpSPA registration rises from 0.163 to 1.051 ms, compared with 0.188 to 2.247 ms

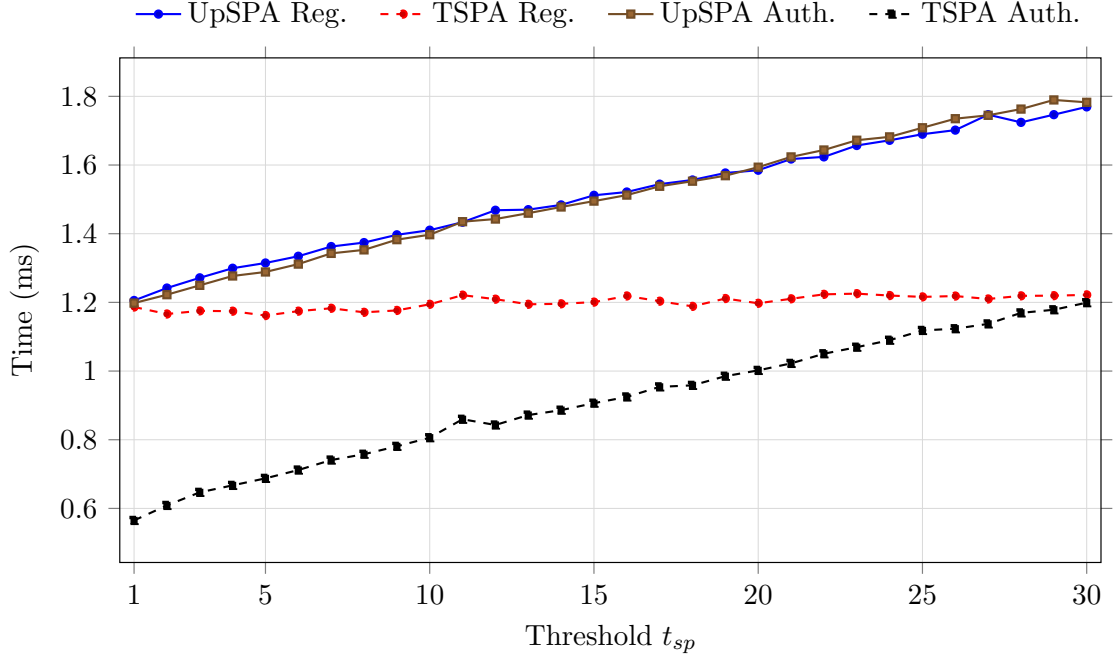
Figure 6.2: Client setup and updates, $n_{sp} = 30$.

for TSPA. At $n_{sp} = 100$, authentication is 1.046 ms for UpSPA and 1.178 ms for TSPA.

Client setup and updates with $t_{sp} = 0.6 \cdot n_{sp}$. Figure 6.6 reports setup reaching 0.681 ms, secret update 1.067 ms, and password update 2.763 ms at $n_{sp} = 100$. Password update has the steepest local curve but remains independent of the number of login-server accounts.

LAN registration and authentication with $t_{sp} = 0.6 \cdot n_{sp}$. Figure 6.7 shows the registration curves crossing between $n_{sp} = 60$ and 70: the growing TSPA computation eventually outweighs UpSPA's additional modeled stage. Authentication narrows but does not cross within the measured range.

LAN setup and updates with $t_{sp} = 0.6 \cdot n_{sp}$. Figure 6.8 shows password update below secret update at $n_{sp} = 30$ (1.953 versus 2.060 ms) but above it at $n_{sp} = 40$ (2.222 versus 2.195 ms). At this scale, additional local/provider work becomes visible despite the shorter benchmark schedule.

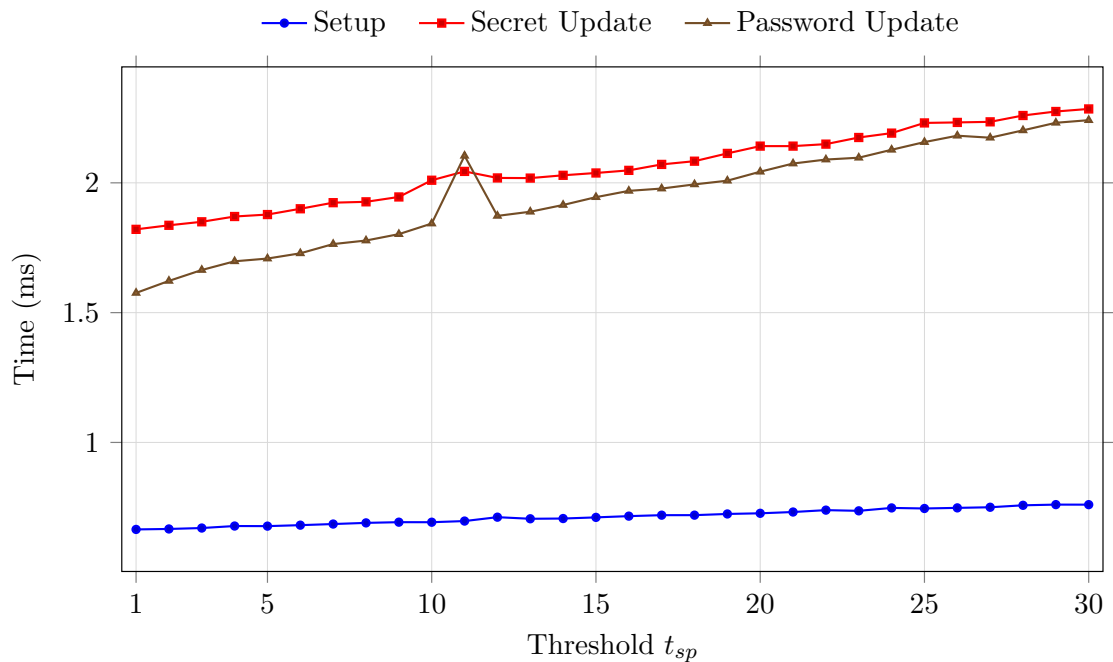
Figure 6.3: LAN registration and authentication, $n_{sp} = 30$.

WAN comparison retained in the appendix

Appendix B retains only the two direct WAN comparisons between UpSPA and TSPA: the fixed- $n_{sp} = 10$ sweep and the proportional $t_{sp} = 0.6n_{sp}$ sweep. They confirm that the common RTT and the modeled stage count dominate the WAN totals, so the same curves are not repeated in the main chapter.

6.9.4 Comparison of UpSPA, PASTA-U and AugSSO

This subsection first compares the three schemes on the deployment grid in Figures 6.9–6.11, and then uses the fixed- n_{sp} and proportional-threshold sweeps to distinguish sensitivity to t_{sp} from joint scaling in n_{sp} and t_{sp} . For a fair same-platform comparison, the PASTA-U results are obtained from our modern Rust implementation rather than from the original paper’s cross-platform measurements; the validity of this baseline is examined in Appendix A through a comparison with a paper-style Shoup-RSA Rust profile and the published token-generation and password-update results.

Figure 6.4: LAN setup and updates, $n_{sp} = 30$.

Registration. Figure 6.9 shows that UpSPA and PASTA-U have the lowest client costs, 0.085–0.170 ms and 0.068–0.099 ms, respectively, whereas AugSSO remains at 0.389–0.443 ms. On the SP side, PASTA-U performs only a negligible store operation, UpSPA grows mainly with the threshold, and AugSSO pays per-provider public-key and record-generation costs. The LAN values remain below 1.60 ms for all three schemes. In WAN, AugSSO is fastest at 64.59–66.98 ms because registration requires approximately one RTT, while UpSPA and PASTA-U remain near two RTTs. The small SP decrease of AugSSO at (7, 7) and the minor WAN reversals are below the benchmark/jitter scale and do not indicate a protocol transition.

Authentication. Figure 6.10 separates the computational costs more clearly. UpSPA remains below 0.17 ms on the client and 0.15 ms on the SP side. The client and SP costs of PASTA-U and especially AugSSO increase with t_{sp} because more threshold responses, interpolations, and token or public-key operations are processed; the LAN curves preserve this ordering. In WAN, PASTA-U is fastest at 65.55–71.02 ms because its authentication is close to one RTT. UpSPA remains near

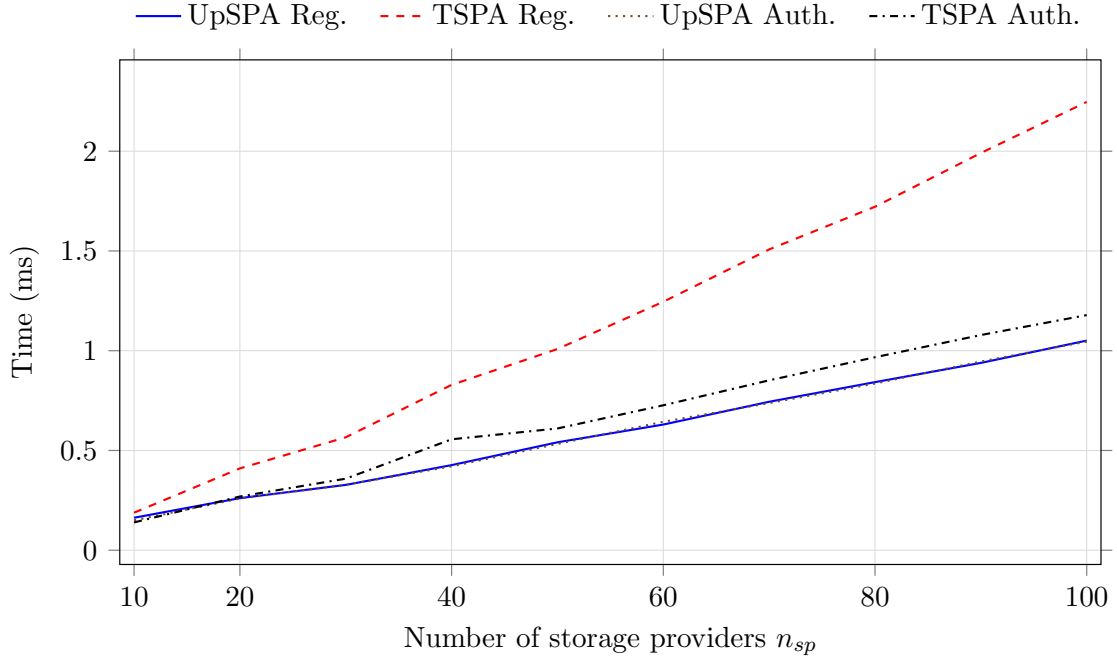
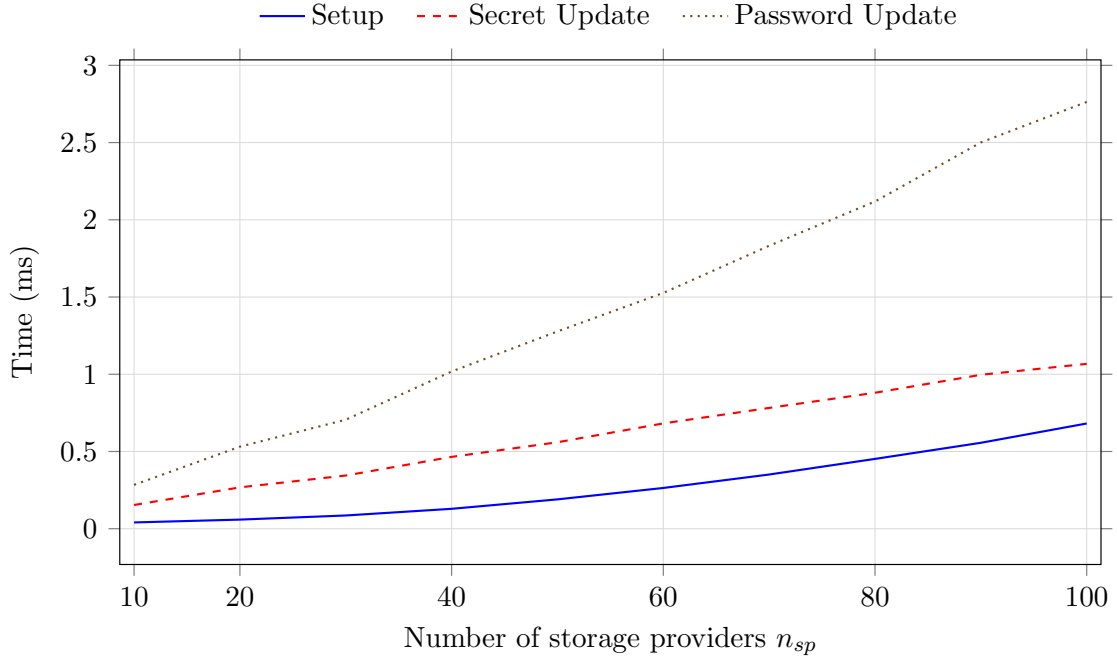


Figure 6.5: Client registration and authentication, $t_{sp} = 0.6n_{sp}$.

two RTTs at 125.25–127.37 ms, while **AugSSO** adds its larger two-stage client computation and reaches 135.69–160.97 ms. The small non-monotonic points at fixed low threshold are within the configured jitter and are not algorithmic anomalies.

Password update. Figure 6.11 shows the largest computational separation. UpSPA remains below 0.27 ms on the client and 0.28 ms on the SP side. AugSSO has the largest client cost (15.09–38.73 ms), whereas PASTA-U has the largest calculated SP cost (10.02–23.09 ms) because its update repeats threshold authorization and prepares provider-specific transition material. The LAN curves therefore follow the computational ordering. In WAN, UpSPA is fastest at 125.88–132.03 ms, AugSSO requires 143.44–174.51 ms, and PASTA-U reaches 228.43–239.83 ms because of its longer update schedule. The isolated decreases at (5, 2) or (7, 2) are ordinary $p50$ variation; the overall scaling remains increasing.

Proportional-threshold scaling: registration. Figure 6.12 increases n_{sp} while maintaining $t_{sp} = 0.6n_{sp}$. Client cost grows from 0.150 ms to 1.071 ms for UpSPA and from 0.074 ms to 0.698 ms for PASTA-U, whereas AugSSO grows only from 0.482 ms

Figure 6.6: Client setup and updates, $t_{sp} = 0.6n_{sp}$.

to 0.636 ms because its fixed hash-to-group and key-generation work remains large relative to additional share generation. Consequently, AugSSO overtakes UpSPA between $n_{sp} = 40$ and 50 and PASTA-U between 90 and 100. The LAN curves show the same crossover behavior. In WAN, the one-round AugSSO path remains dominant at 67.642–71.666 ms, compared with approximately 128–141 ms for the other schemes. Its initial client decrease from $n_{sp} = 10$ to 20 and the nearly flat PASTA-U points at 20 and 30 are small-size $p50$ /cache effects, not asymptotic improvements.

Proportional-threshold scaling: authentication. Figure 6.13 converts threshold-dependent work into approximately linear growth. On the client, UpSPA increases from 0.149 ms to 1.075 ms, PASTA-U from 4.071 ms to 20.523 ms, and AugSSO from 22.656 ms to 221.759 ms, reflecting progressively heavier work per threshold response. LAN is therefore computation-dominated and reaches 2.248 ms, 21.649 ms, and 224.879 ms, respectively, at $n_{sp} = 100$. In WAN, PASTA-U remains fastest (70.472–91.072 ms), UpSPA remains mostly RTT-dominated (128.397–132.612 ms), and AugSSO reaches 362.216 ms because its increasing local work

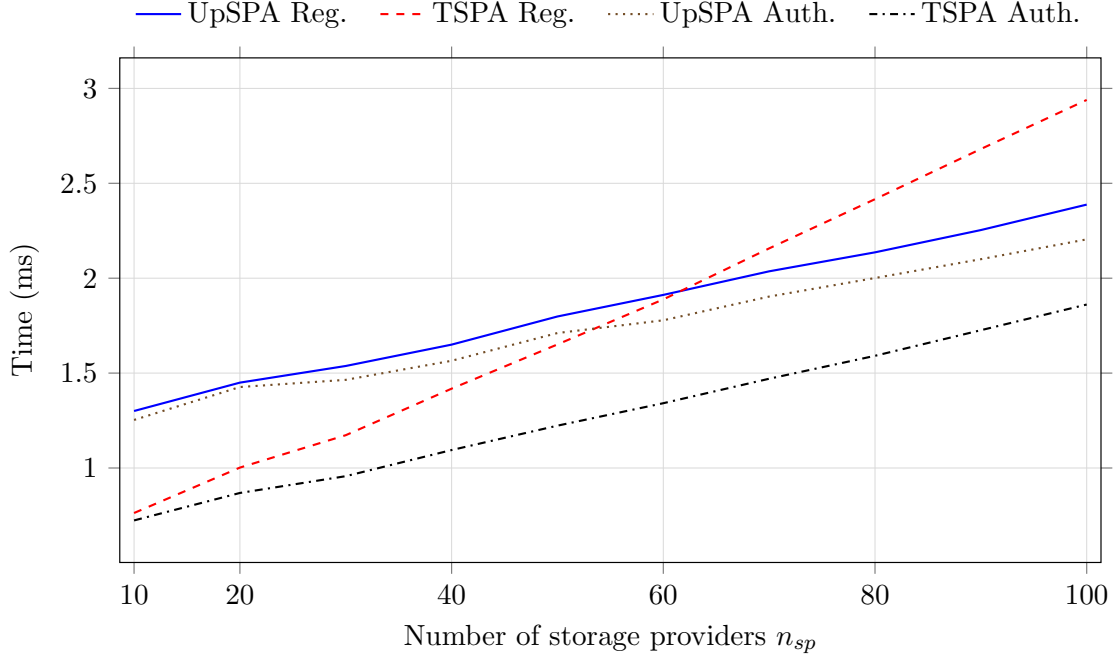
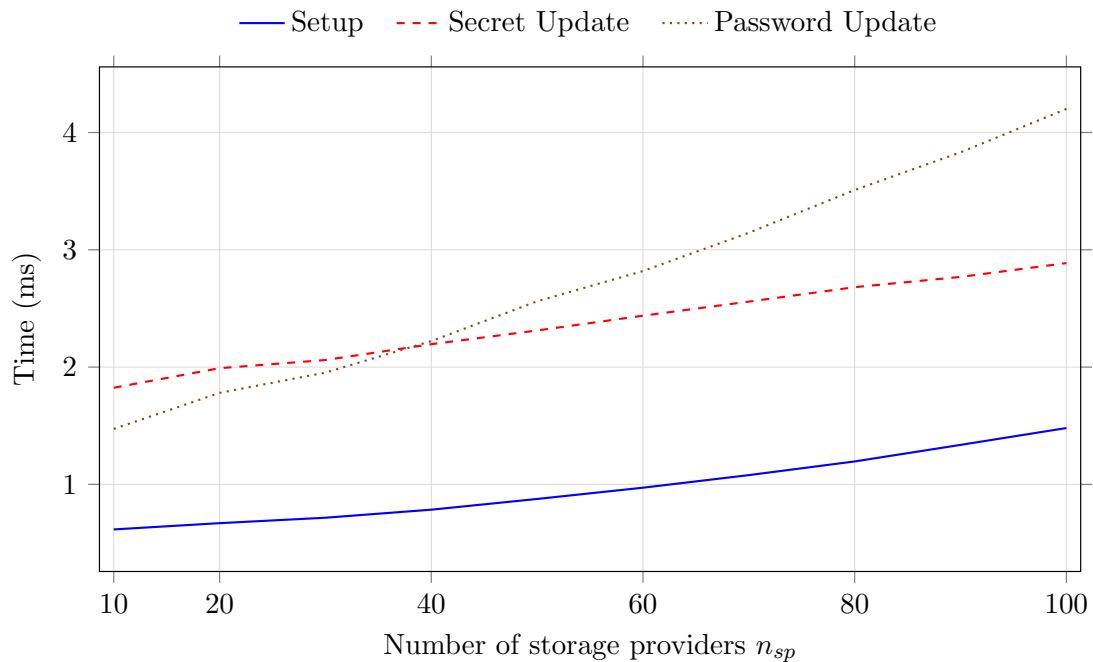


Figure 6.7: LAN registration and authentication, $t_{sp} = 0.6n_{sp}$.

is added to two exchanges. The small WAN decreases and the unusually small PASTA-U increment from $n_{sp} = 50$ to 60 are moderate median/jitter fluctuations; the scaling trend is unchanged.

Proportional-threshold scaling: password update. Figure 6.14 produces the largest absolute costs. Client latency grows from 0.275 ms to 2.762 ms for UpSPA, from 4.611 ms to 24.409 ms for PASTA-U, and from 47.246 ms to 457.130 ms for AugSSO. The steep AugSSO curve combines threshold old-password authentication with new-credential generation and update processing; LAN therefore reaches 4.399 ms, 43.227 ms, and 456.526 ms, respectively, at $n_{sp} = 100$. In WAN, UpSPA remains fastest at 132.107–144.754 ms. AugSSO is faster than PASTA-U at $n_{sp} = 10$ and 20, but the curves cross near $n_{sp} = 30$; thereafter AugSSO’s heavier threshold computation dominates, while the growing PASTA-U update body and final fan-out also expose a superlinear serialized-byte term. The cases where an AugSSO LAN median is slightly below the separately measured client median are not negative network costs: the experiments use independent samples, and medians do not add

Figure 6.8: LAN setup and updates, $t_{sp} = 0.6n_{sp}$.

compositionally.

6.9.5 Comparison of UpSPA and RoUpSPA

This subsection compares the fully available, non-resilient **UpSPA** execution with its robust variant, **RoUpSPA**, over five (n_{sp}, t_{sp}, q_{sp}) settings. The threshold t_{sp} controls cryptographic reconstruction, whereas q_{sp} controls availability and finalization of mutable records. The figures retain the complete local and composed-LAN views. WAN behavior is summarized after the figures because its totals are governed mainly by the fixed RTT and additional sequential stages.

Registration. Figure 6.15 shows that **RoUpSPA** increases local registration from 0.197 ms to 0.445 ms, compared with 0.084–0.147 ms for **UpSPA**, because it validates counters and records and completes a quorum write. The composed-LAN difference remains 2.115–2.285 ms across the five settings.

Secret update. Figure 6.16 follows the same decomposition. Local **RoUpSPA** cost rises from 0.200 ms to 0.460 ms, while the composed-LAN gap remains 2.120–

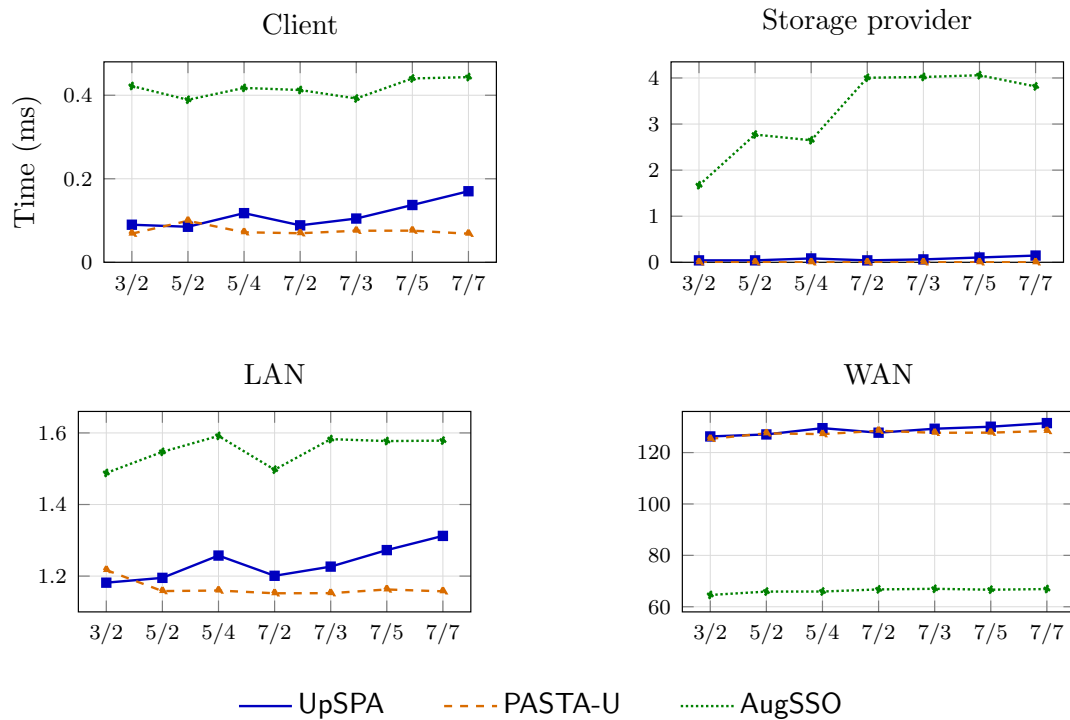


Figure 6.9: Registration latency across benchmark targets.

2.330 ms.

Password update. Figure 6.17 has the largest local values because the phase recovers the old state, performs two TOPRF interactions, constructs the new password-protected state, and commits a signed record to a quorum. Even so, RoUpSPA remains below 0.73 ms locally, its isolated quorum overhead is 0.058–0.147 ms, and the composed-LAN gap is 2.185–2.463 ms.

WAN interpretation. The resilient WAN totals are not plotted because the fixed 60 ms RTT and the additional sequential quorum stages dominate the observed differences. The resulting separation is therefore a property of the modeled communication schedule, not a direct measure of cryptographic or local quorum computation, and is not used to rank the two constructions.

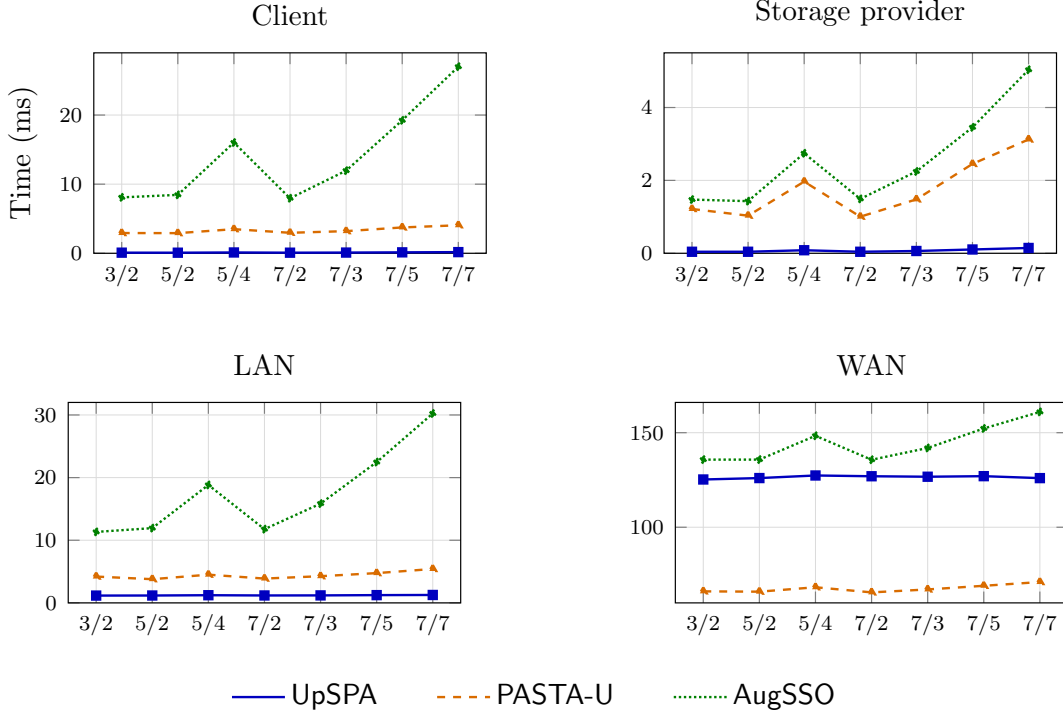


Figure 6.10: Authentication latency across benchmark targets.

6.9.6 Resilient Synchronization Backend Evaluation

The resilient suite compares quorum/ABD, signed- $\mathcal{PBB}$, DKG-authorized- $\mathcal{PBB}$, and blockchain-backed synchronization under common admissible tuples. The retained panels report complete local and composed-LAN execution. The value `full_no_net` denotes the complete local benchmark phase, whereas backend overhead isolates the synchronization-specific work.

For quorum/ABD, q_{sp} is the implemented quorum. For the public-state modes, the same parameter records the matched availability scenario and is not a board or ledger quorum. The retained plots use the minimum proportional policy $q_{sp} = \lceil 0.8n_{sp} \rceil$ over security-valid tuples. Table 6.2 reports the $n_{sp} = 10 \rightarrow 100$ endpoints for this policy.

Proportional registration scaling. Figure 6.18 shows similar local growth for quorum/ABD and signed- $\mathcal{PBB}$: their medians move from 0.436 and 0.381 ms at $n_{sp} = 10$ to 3.723 and 3.869 ms at $n_{sp} = 100$. The composed-LAN curves preserve

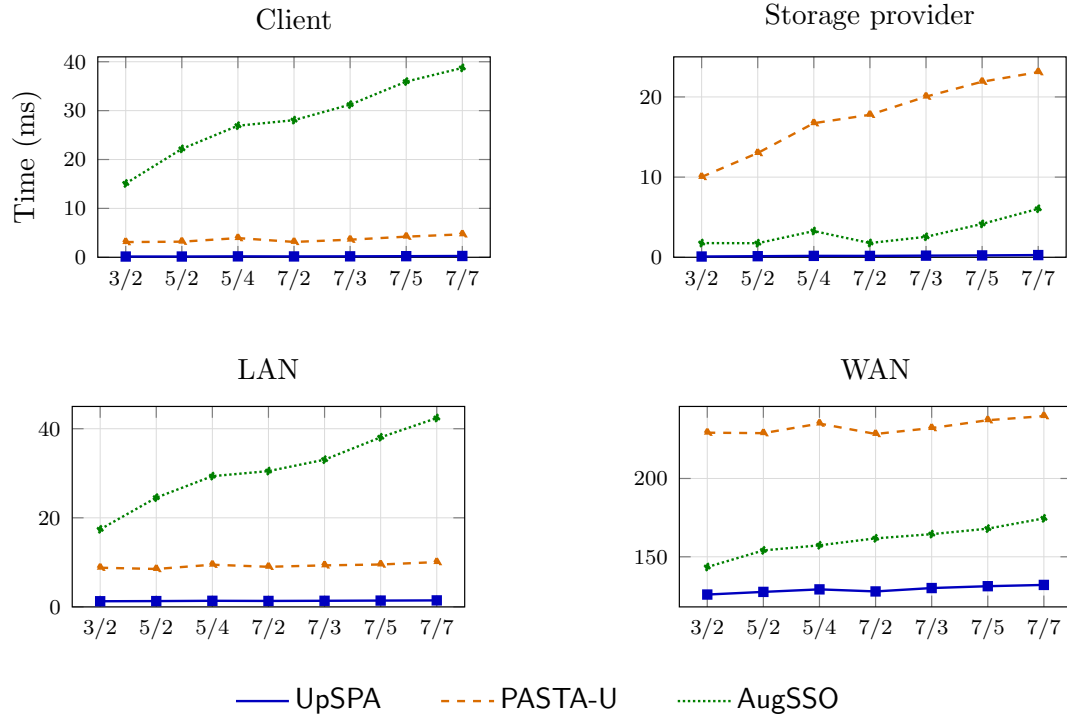
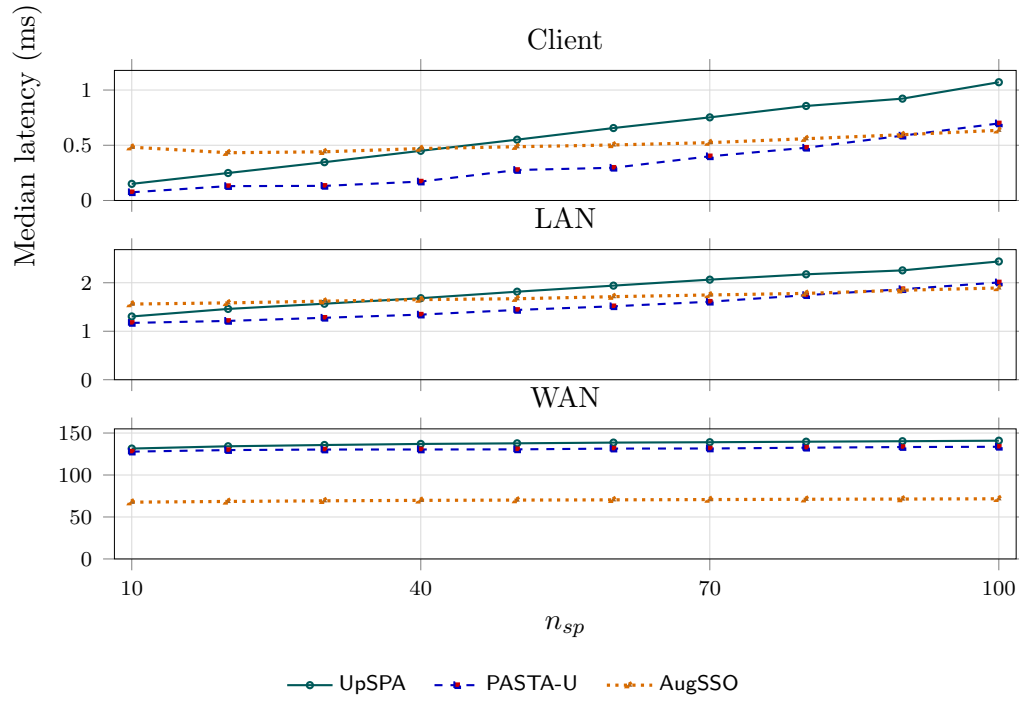
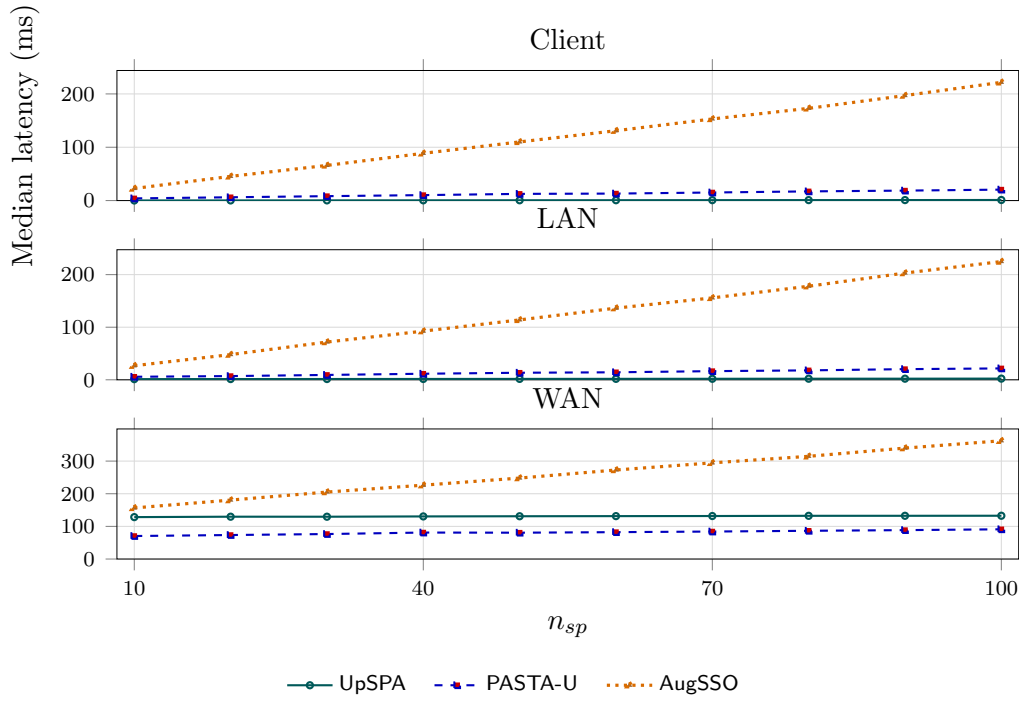


Figure 6.11: Password-update latency across benchmark targets.

the same close ordering.

Proportional secret-update scaling. Figure 6.19 separates the backends more clearly. At $n_{sp} = 100$, quorum/ABD reaches 11.241 ms locally, compared with 3.852 ms for signed- $\mathcal{PBB}$; the composed-LAN panel preserves this difference.

Proportional password-update scaling. Figure 6.20 shows the strongest local divergence: at $n_{sp} = 100$, quorum/ABD reaches 29.176 ms and signed- $\mathcal{PBB}$ 8.254 ms. The composed-LAN panel shows that the coordination cost remains visible after adding the common LAN model.

Figure 6.12: Registration latency, $t_{sp} = 0.6n_{sp}$.Figure 6.13: Authentication latency, $t_{sp} = 0.6n_{sp}$.

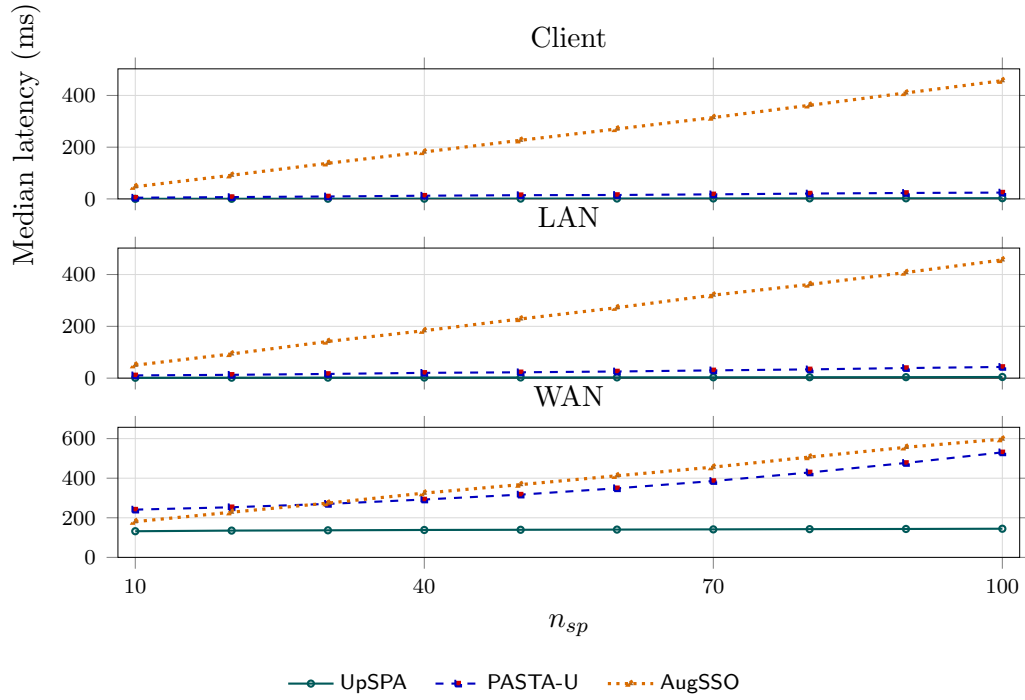
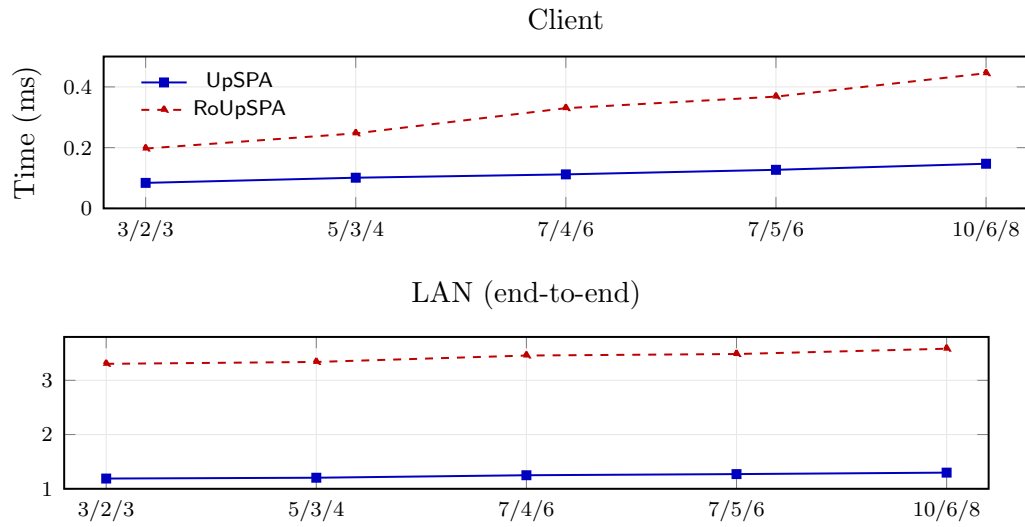
Figure 6.14: Password-update latency, $t_{sp} = 0.6n_{sp}$.

Figure 6.15: Registration latency: UpSPA and RoUpSPA.

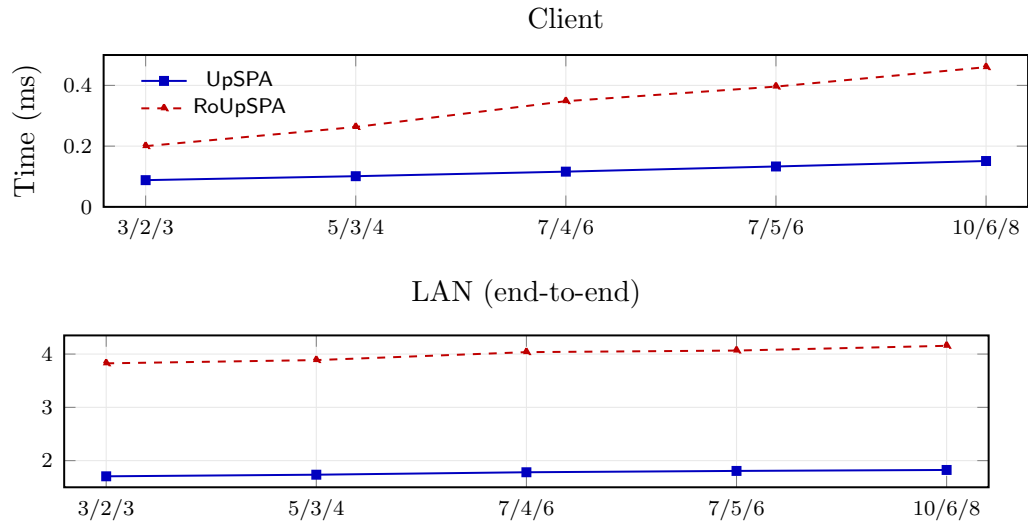


Figure 6.16: Secret-update latency: UpSPA and RoUpSPA.

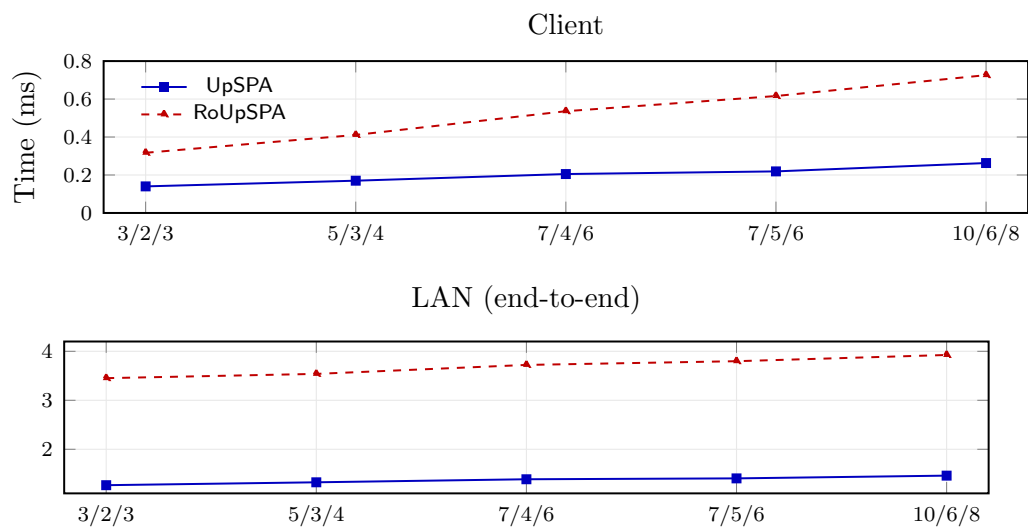


Figure 6.17: Password-update latency: UpSPA and RoUpSPA.

Phase	Backend	Complete local $p50$ (ms)
Registration	Quorum/ABD	$0.436 \rightarrow 3.723$
	Signed $\mathcal{PBB}$	$0.381 \rightarrow 3.869$
	DKG $\mathcal{PBB}$	$0.515 \rightarrow 4.750$
	Permissioned chain	$0.923 \rightarrow 6.737$
	Layer-2 chain	$0.923 \rightarrow 6.737$
	Public chain	$0.923 \rightarrow 6.737$
Secret update	Quorum/ABD	$0.447 \rightarrow 11.241$
	Signed $\mathcal{PBB}$	$0.412 \rightarrow 3.852$
	DKG $\mathcal{PBB}$	$0.550 \rightarrow 4.852$
	Permissioned chain	$0.811 \rightarrow 6.855$
	Layer-2 chain	$0.811 \rightarrow 6.855$
	Public chain	$0.811 \rightarrow 6.855$
Password update	Quorum/ABD	$1.018 \rightarrow 29.176$
	Signed $\mathcal{PBB}$	$0.904 \rightarrow 8.254$
	DKG $\mathcal{PBB}$	$0.894 \rightarrow 8.220$
	Permissioned chain	$1.014 \rightarrow 8.610$
	Layer-2 chain	$1.014 \rightarrow 8.610$
	Public chain	$1.014 \rightarrow 8.610$

Table 6.2: Local endpoint medians, $q_{sp} = \lceil 0.8n_{sp} \rceil$.

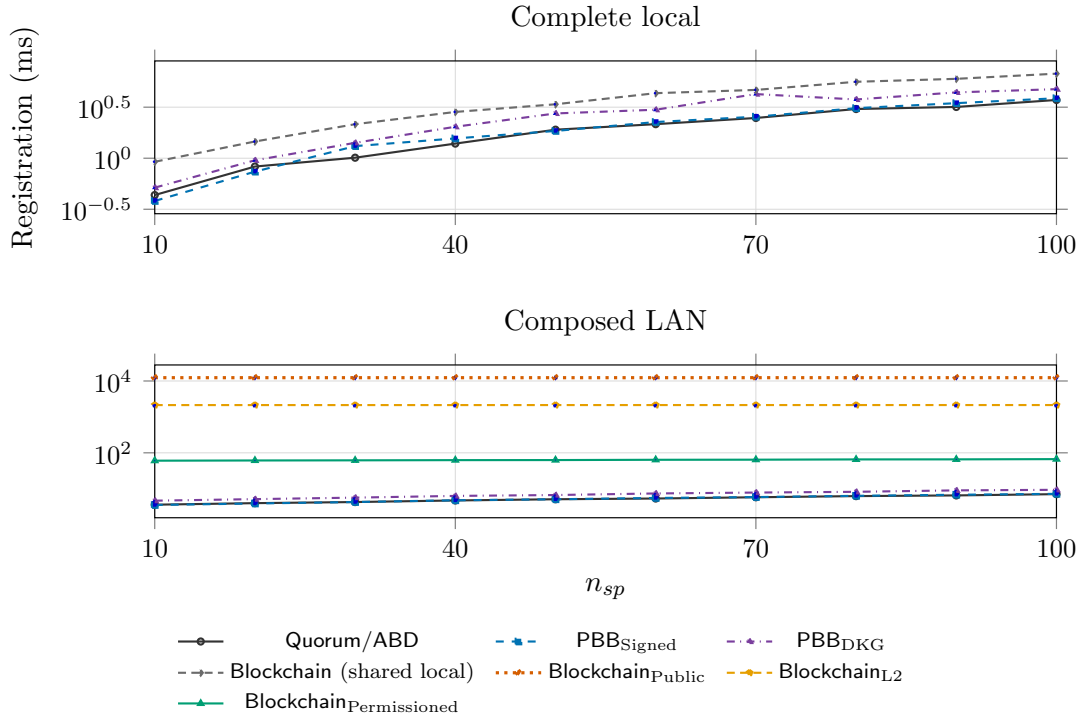
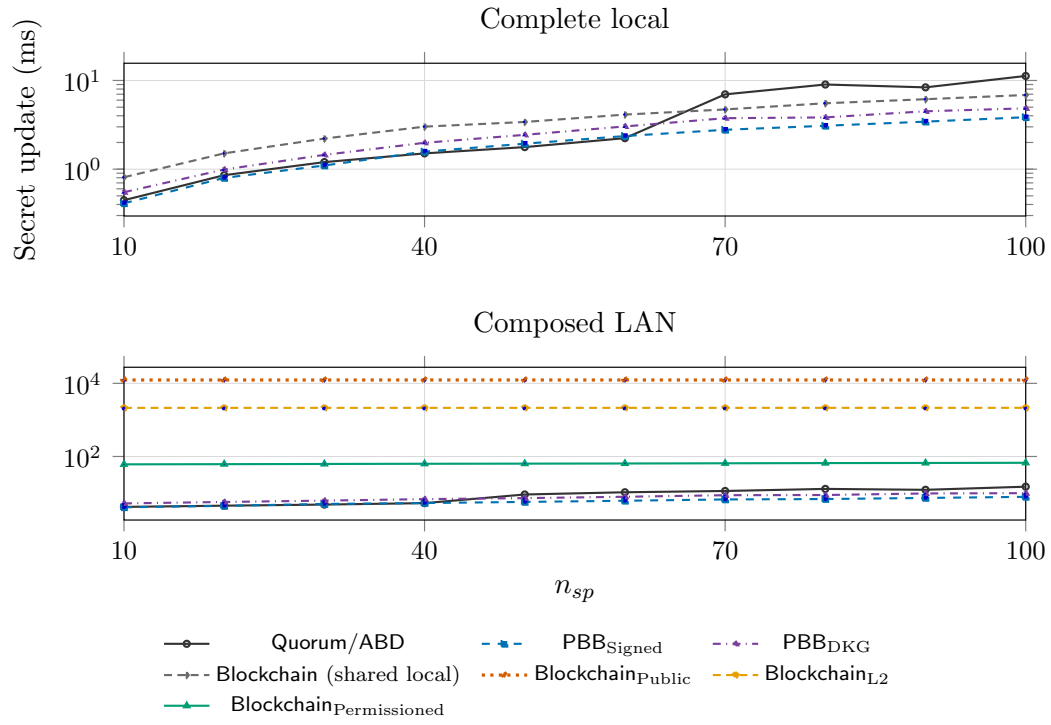
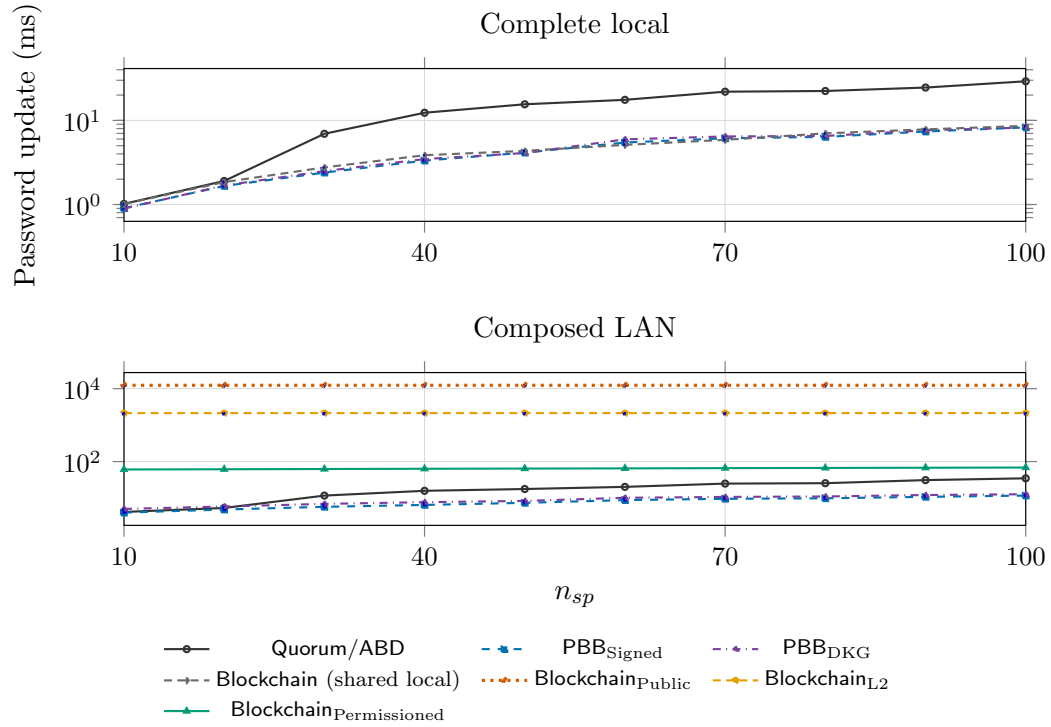
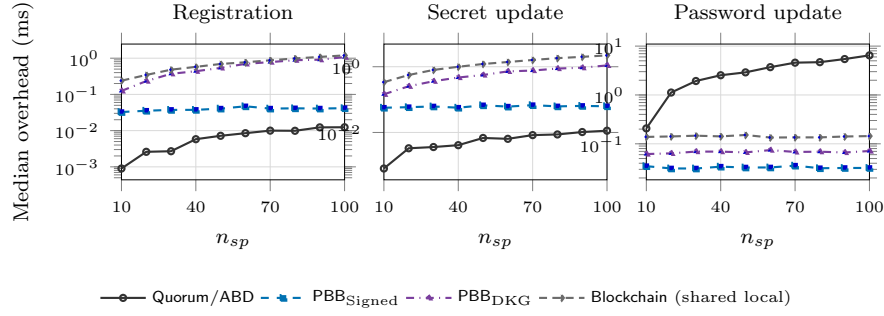
Figure 6.18: Registration latency, $q_{sp} = \lceil 0.8n_{sp} \rceil$.

Figure 6.21 isolates mechanism-specific overhead under the minimum proportional policy. Registration overhead grows from 0.0009 to 0.0124ms for quorum/ABD, 0.0326 to 0.0417ms for signed- $\mathcal{PBB}$, 0.1247 to 1.0712ms for DKG- $\mathcal{PBB}$, and 0.2392 to 1.1963ms for the shared blockchain-local path. Password update differs sharply: quorum/ABD grows from 0.2068 to 6.4609ms, whereas signed- $\mathcal{PBB}$ remains around 0.03–0.04ms. These values are not interchangeable primitive timings; each series sums the synchronization work specific to its backend.

Figure 6.19: Secret-update latency, $q_{sp} = \lceil 0.8n_{sp} \rceil$.Figure 6.20: Password-update latency, $q_{sp} = \lceil 0.8n_{sp} \rceil$.

Figure 6.21: Backend overhead, $q_{sp} = \lceil 0.8n_{sp} \rceil$.

The retained resilient plots therefore focus on absolute scaling under the minimum secure availability policy and on synchronization-specific overhead. WAN results are not plotted: the fixed RTT and backend-specific finality profiles dominate their totals, so they indicate the chosen communication model rather than local protocol cost.

6.9.7 Receipt-Level EVM Gas Results

The completed receipt run is identified as **qa-local-anvil-20260728b** on Anvil chain ID 31337 with Anvil 1.5.1. It is a local execution-cost and regression run, not a public-network fee, congestion, or finality experiment. It uses ten repetitions, batch sizes $B \in \{8, 32, 128\}$, and one confirmation. With only ten successful samples per operation, p_{95} is the observed maximum and should be treated as a diagnostic tail value; contract deployment has one observation and is reported as a point measurement. The manifest contains 91 receipt observations from blocks 2 through 92; all 91 have successful status. The archived local run maps deployer, publisher, relayer, and controller to the same Anvil EOA. The contract still executes the EIP-712 authorization path, so the receipts remain useful as local regression evidence, but a final public-network run should use distinct role accounts. ERC-1271 authorization is covered by the contract tests but is not a separate receipt series in this dataset; an ERC-1271 gas comparison must therefore be measured and reported as another group rather than inferred from the EOA rows. Table 6.3 reports the direct path

and deployment, and Table 6.4 reports batch publication and transactional proof verification.

Operation	N	Min gas	$p50$ gas	$p95$ gas	Max gas
Contract deployment	1	1 356 750	1 356 750	1 356 750	1 356 750
Direct registration	10	123 607	123 619	123 639	123 639
Direct secret update	10	51 157	51 173	51 189	51 189
Direct password update	10	51 145	51 169	51 189	51 189

Table 6.3: Receipt-level EVM gas for deployment and direct RoUpSPA commitments.

B	Depth	Publish min	Publish $p50$	Publish max	$p50/B$	Verify $p50$
8	3	93 625	93 637	93 637	11 704.625	28 260
32	5	93 625	93 643	93 661	2 926.344	30 790
128	7	93 649	93 661	93 661	731.727	33 350

Table 6.4: Receipt-level EVM gas for Merkle publication and proof verification.

Direct registration costs about 2.416 times one direct update because it installs the controller and first record, whereas the secret- and password-update medians differ by only four gas. Root publication changes by 24 gas between $B = 8$ and $B = 128$, approximately 0.026%, so its per-record cost falls from 11 704.625 to 731.727 gas. At $B = 128$, publication-only amortization is about 69.9 times below a direct secret update and 168.9 times below direct registration. When every leaf is also verified in a transaction, the combined $p50$ values are 39 964.625, 33 716.344, and 34 081.727 gas for $B = 8, 32, 128$. Thus $B = 32$ is the best of the three measured points for the publish-plus-on-chain-verification policy, whereas $B = 128$ is best when proofs are checked off chain and only publication is amortized.

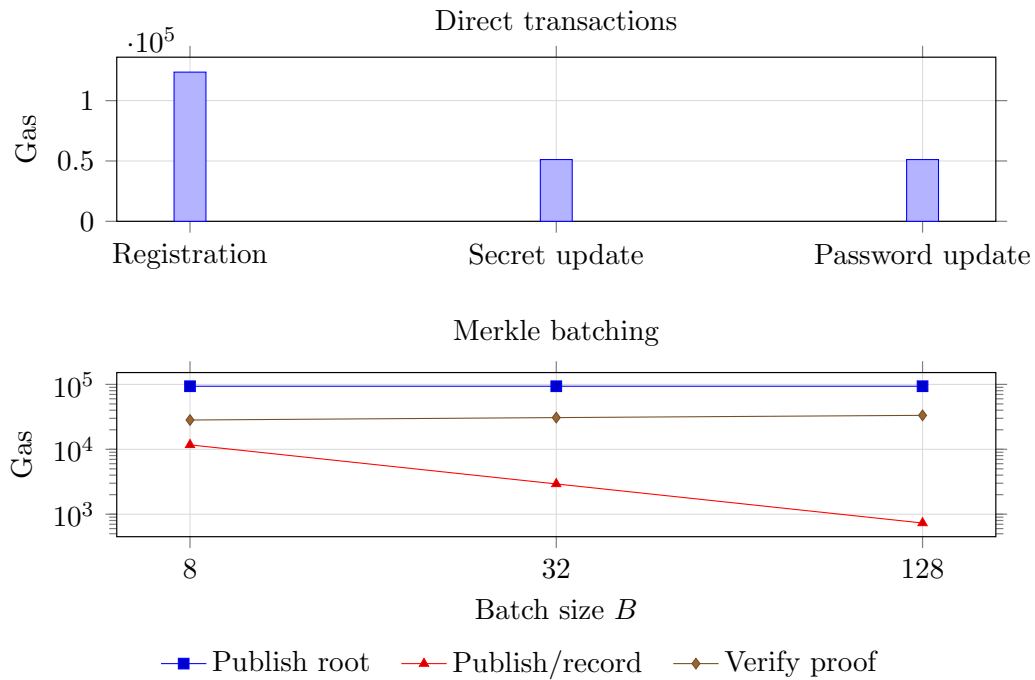


Figure 6.22: EVM gas for direct and Merkle operations.

Figure 6.22 has two distinct panels. The upper panel shows that initial direct registration is more expensive than either direct update because it installs the controller and first record. The lower panel shows that root publication is nearly batch-size independent, publication cost per record falls with B , and transactional proof verification grows with Merkle depth.

6.9.8 Foundry Snapshot Cross-Check

The older UpSPA chain-gas archive measures a smaller commitment anchor with Foundry’s snapshot boundary. Test setup is excluded and signatures are checked off chain by providers. Tables 6.5 and 6.6 reproduce every snapshot target rather than selecting only successful writes.

The snapshot supports the same qualitative interpretation without being merged with the receipt distribution. Registration is substantially more expensive than an overwrite (121 902 versus approximately 32 800 gas), root append is essentially unchanged for 1, 64, and 256 leaves, and proof verification grows from 13 730 gas

at depth 8 to 37 624 gas at depth 32. Rejected stale, duplicate, malformed, and unknown-batch paths are also explicit, which is useful for regression and denial-of-service analysis even though they are not normal successful-scenario costs.

Single-commitment-board Foundry target	Gas
Read latest after a valid write	6 240
Rejected duplicate counter	5 305
Rejected stale counter	5 668
Rejected invalid phase	9 421
Rejected zero bundle-pointer hash	9 247
Rejected zero record hash	9 169
Rejected zero tag	9 229
Valid registration commitment	121 902
Valid secret-update commitment	32 842
Valid password-update commitment	32 798

Table 6.5: Foundry gas for direct commitments.

6.9.9 TRON Implementation and Pending Native Measurements

The TVM-compatible contract and benchmark runner are complete. The matrix matches the EVM logical scenarios: deployment, direct registration, direct secret update, direct password update, root publication for each batch size, and transactional proof verification. Each row is designed to carry Energy, Bandwidth, SUN fees, submission/confirmation latency, transaction identifier, block metadata, success, and solidification state. The supplied archive, however, contains no completed live TRON receipt file. For that reason this chapter does not print a TRON chart, does not print synthetic Energy or Bandwidth values, and does not reuse EVM gas as a proxy. A later table and chart can be inserted without changing the schema once all normal rows satisfy `success = true` and `solidified = true`.

6.10 Discussion

Merkle-root-board Foundry target	Gas
Rejected proof for unknown batch	14 879
Verify proof of depth 8	13 730
Verify proof of depth 16	21 614
Verify proof of depth 32	37 624
Append root for 1 leaf	144 515
Append root for 64 leaves	144 538
Append root for 256 leaves	144 472
Read batch after write	6 708
Rejected invalid counter range	9 825
Rejected duplicate batch tag	5 511
Rejected duplicate Merkle root	7 658
Rejected empty batch	9 702
Rejected zero batch tag	9 449
Rejected zero Merkle root	9 455

Table 6.6: Foundry gas for Merkle roots.

The measurements separate local cryptography, provider work, implemented communication stages, and synchronization-backend cost. At deployment scale, local UpSPA and TSPA computation remains below one millisecond; the larger WAN gaps are therefore dominated by the successful-path stage schedules used by the benchmark. Fixed- n_{sp} and proportional sweeps are both necessary: threshold sensitivity is clearest at fixed provider count, whereas joint n_{sp}/t_{sp} growth reveals the registration crossover in which UpSPA overtakes TSPA on LAN.

Secret update and password update should not be compared as interchangeable operations. The former rotates one account secret; the latter changes the compact password-protected state once and is independent of n_{ls} . Password update performs more local work at large parameters but can remain lower on WAN when its implemented schedule saves a stage. This is a performance property of the benchmark path, while the formal update semantics remain those of Section 4.2.

The threshold-SSO comparison also reflects different role assignments and cryp-

tographic groups. PASTA-U benefits from a shorter WAN authentication path, AugSSO pays more for BLS12-381 threshold operations as t_{sp} grows, and UpSPA has the strongest measured password-update performance. These are complete successful-phase comparisons, not claims of identical relying-party, recovery, or liveness semantics.

The resilient-backend results show no universal winner. Quorum/ABD is competitive for registration and small tuples but grows more rapidly for large secret- and password-update configurations. Signed- $\mathcal{PBB}$ provides low local update and tail latency in many supplied profiles; DKG authorization adds admission work; and blockchain profiles are governed mainly by their configured confirmation delays. Policy ratios near one on composed WAN do not erase local backend overhead: the common network model compresses the relative difference.

The receipt-level EVM results show an expensive first direct write, nearly constant Merkle-root publication, and proof cost increasing with depth. The Foundry snapshot supports the same qualitative trends but is not pooled with the receipt distribution because its contract and timed boundary differ. TRON numbers remain absent until successful solidified receipts are available; EVM gas is not used as a proxy for Energy, Bandwidth, or SUN fees.

Finally, the LAN/WAN values are controlled compositions rather than production deployments, and the benchmark relaxes retry and liveness machinery for fair cross-scheme timing. The chapter therefore supports an engineering conclusion, not a new protocol theorem: UpSPA adds update functionality with small local cost; resilient designs move cost into quorum coordination, board processing, ledger confirmation, or proof handling; and optimization should target the backend-specific dominant component.

Part IV

EPILOG

Chapter 7

CONCLUSION, LIMITATIONS, AND FUTURE WORK

7.1 Conclusion

Passwords remain difficult to replace because they are familiar, inexpensive, and supported by almost every online service. Their usability, however, is tied to weak secrets, password reuse, and server-side verification material that may be tested offline after compromise. This thesis extends single-password authentication toward secure state evolution: rotating an account secret, changing the password, and tolerating unavailable storage providers without weakening protection against offline password guessing.

The fully available construction, UpSPA, addresses the update problem under the system and corruption assumptions stated in Sections 4.1.2 and 4.1.3. Its central design choice is the separation of the user’s state into a compact password-protected identification record and independent login-server-specific records. The threshold OPRF output protects only the identification layer, while a high-entropy key recovered from that layer protects the account records. This structure permits a secret update to replace the authentication material of one login server and a password update to refresh only the identification record. Password change is therefore independent of the number of registered login servers, rather than requiring all account records to be rewritten.

Threshold OPRF execution hides the password-derived value from insufficiently many storage providers; authenticated encryption protects stored state; signatures authorize identification-state updates; counters provide freshness; and context bindings prevent records from being moved between accounts or providers. The resulting state machine supports all five protocol phases without placing a password or stable password-derived verifier at a login server. The design challenges are summarized

in Section 4.1.4.

Security is formalized through ideal–real simulation for completed phases. A sequence of games replaces password-dependent OPRF outputs and encrypted records, while forgery, authenticated-encryption, and hash-collision events are handled separately. Under the stated static corruption bounds, the transcript and corrupted storage state provide no additional offline password test beyond the unavoidable online guessing probability.

The robust construction, **RoUpSPA**, extends **UpSPA** design to partial storage-provider unavailability. Its model is given in Sections 5.1.2 and 5.1.3. The main distinction is between threshold access and quorum durability. Authentication needs only enough valid responses to complete the threshold computation, whereas registration and updates need enough acknowledgments to make the new state durable. The condition $2 \cdot q_{sp} - n_{sp} > t_{sp} - 1$, together with $t_{sp} > n_{sp}/2$, ensures that completed quorums intersect in sufficient honest state and cannot support incompatible committed versions. These guarantees are stated in Theorem 5.3.7 and its rollback and divergent-commit consequences. Authentication remains outside the quorum-write path, so the stronger availability requirement is imposed only when persistent state changes.

The proof for **RoUpSPA**, summarized by Theorem 5.3.2, inherits the cryptographic core of **UpSPA** and adds version-consistency and quorum-intersection arguments. Section 5.4 also studies public logs and blockchain commitments. They may simplify late catch-up, but add global availability, metadata, flooding, cost, and finality assumptions, so they remain optional alternatives.

Chapter 6 connects the formal designs to an executable system and separates client, provider, login-server, persistence, and synchronization costs. The measurements show the intended trends: cost grows with participating providers, parallelism hides part of the network overhead, and password update remains independent of the number of login servers. The thesis therefore progresses from SPA and TSPA to updateability, resilience, and implementation while preserving protection against offline password testing.

7.2 Limitations

The adversary is **static**, with corrupted parties fixed according to Sections 4.1.3 and 5.1.3. Adaptive compromise, proactive refresh, secure erasure, and accumulation of provider state over time are outside the model. The phase-oriented proofs are not universally composable and do not cover arbitrary protocol composition or unrestricted concurrent interleaving.

The user **endpoint is trusted** while it handles the password and decrypted state. The protocols reduce the damage caused by compromised login servers and bounded sets of storage providers, but cannot protect a password entered on a device controlled by a keylogger or malicious browser component. Secure authenticated channels, correct origin identification, and reliable certificate validation are therefore assumed. Domain binding limits cross-context misuse, but does not replace browser and operating-system isolation or prevent every real-time session attack.

Cryptography also cannot guarantee availability against **denial of service**. The fully available version requires all storage providers during state-mutating phases, while RoUpSPA requires at least q_{sp} acknowledgments for a completed write and at least t_{sp} valid providers for threshold computation. Registration and secret update additionally assume that the **login server is available** apart from a bounded interruption; otherwise the phase aborts and must be retried. The protocol fails closed, but it cannot force unavailable or malicious parties to respond. Permanent loss beyond the selected thresholds may make an account unrecoverable even when the user still knows the password.

The consistency guarantees apply to the specified state machine, not to an **arbitrary distributed database**. Implementations must serialize conflicting updates, preserve counters and signed fields, and make retransmission idempotent. Provider replacement, threshold changes, migration, and catastrophic recovery are not part of the main protocol; public-log variants also require separate assumptions about censorship, flooding, finality, and off-chain availability.

Finally, the **implementation** is a research prototype. Its experiments isolate protocol costs under controlled hardware, network conditions, parameters, and li-

libraries; they are not a production security audit, large-scale deployment, or longitudinal usability study. Comparisons with threshold single-sign-on systems are useful at the level of phase cost, but the systems do not provide identical functionality. Production use would additionally require hardened setup and key provisioning, transport and database security, rate limits, monitoring, backups, signed errors, and extensive browser and failure testing.

7.3 Future Work

A stronger formal model should address adaptive corruption, proactive refresh, concurrent execution, and composability. Symbolic verification or model checking could complement the cryptographic proof by exploring message reordering, duplicate delivery, crashes, and overlapping updates.

A second direction is dynamic provider management. A deployable system should allow providers to join, leave, or be replaced, and should permit t_{sp} and q_{sp} to change without re-registering every account. This requires authenticated configuration versions, secure re-sharing of identification state, and rules preventing a removed provider from combining old information with newly corrupted parties. The same mechanism could support migration between administrative domains and recovery from long-term provider failure.

The browser deployment should authenticate setup, replace development keys and certificates, sign protocol errors, and move security-critical orchestration into an audited core. Database TLS, timeouts, rate limits, monitoring, backups, and retry-safe persistence should be tested across independent providers. A multi-site user study could then evaluate latency, form compatibility, recovery, and usability beyond the microbenchmarks of Chapter 6.

Performance work may further parallelize TOPRF and retrieval requests, contact providers in latency-aware stages, and batch signature verification. These optimizations must preserve the threshold and quorum rules and avoid concentrating trust in one administrative domain.

A post-quantum extension requires more than replacing the signature scheme.

The TOPRF, authenticated encryption, key derivation, update authorization, and proof model must be reconsidered together. Future work should study post-quantum or KEM-based constructions that retain password privacy, efficient password update, and threshold resilience at acceptable communication cost. The resulting design should be proved and benchmarked separately against classical and quantum adversaries.

Finally, the framework may support hybrid authentication and recovery. Passwords will remain relevant for many users, while passkeys and hardware-backed credentials are increasingly common. Distributed password protection could serve as a recovery or migration layer for device-bound credentials, so that neither loss of one device nor compromise of one service is sufficient for account takeover. This would preserve the central lesson of the thesis: a password is most useful when it is not itself the long-term verifier and when no single supporting party can test it offline.

Appendix A

PAS-TA-U BASELINE AND TTG VALIDATION

A.1 Purpose and Fairness of the Baseline

The main UpSPA–PAS-TA-U–AugSSO evaluation uses our modern Rust implementation of PAS-TA-U [56] as the PAS-TA-U baseline. This is the fairest baseline for the main comparison because all three schemes are executed on the same host, through the same Rust benchmarking framework, with the same parameter notation, sample policy, and network profiles. Substituting the numerical values reported by the original PAS-TA-U paper would instead mix protocol cost with Python overhead, different cryptographic libraries, different hardware, real OpenSSL/OpenSSH integration, and a different transport and timing boundary.

The modern implementation follows the PAS-TA-U token-generation and password-update flows, but deliberately uses a current cryptographic profile: a Ristretto255 TOPRF, BLS12-381 threshold token generation (TTG), BLAKE3, and XChaCha20–Poly1305. The published implementation uses a 2HashTDH-style TOPRF, Shoup threshold RSA with a 2048-bit modulus, SHA-256/SHA-512 in its concrete code path, and AES-128-OFB. To determine whether the modern baseline remains representative of PAS-TA-U rather than benefiting from an unexamined implementation mismatch, we additionally implement a paper-style Rust profile and evaluate three complementary experiments. The first isolates TTG partial evaluation, combination, and verification. The second measures complete token generation and password update over a common real-TCP environment. The third validates that the paper-style threshold-RSA output is accepted through the ordinary OpenSSH RSA verification path.

This design separates three questions. The modern-versus-paper-style Rust comparison measures the effect of the cryptographic profile under one implementa-

tion environment. The paper-style-versus-published comparison checks whether the reimplementation follows the scale and threshold trend of the original evaluation. The main UpSPA comparison continues to use the modern Rust profile because it is the only PAS-TA-U result obtained under the same platform and measurement discipline as UpSPA and AugSSO. Accordingly, this appendix validates the fairness of that baseline without treating cross-platform ratios as protocol-level speedups.

The original PAS-TA-U paper writes a configuration as (t, n) . To remain consistent with the rest of this paper, we report the same settings as (n_{sp}, t_{sp}) , mapping the number of PAS-TA-U identity servers to n_{sp} and its threshold to t_{sp} . This is only a parameter mapping and does not identify the roles or security interfaces of PAS-TA-U identity servers with those of UpSPA storage providers. The evaluated settings are $(n_{sp}, t_{sp}) \in \{(3, 2), (6, 3), (10, 5), (10, 10)\}$.

A.2 Measurement Methodology

The two Rust profiles were built in release mode and evaluated on the same physical host as the unified comparison: a 64-bit Windows machine with an 11th Gen Intel Core i7-1185G7 processor at 3.00 GHz and 16 GB RAM. WSL2/Linux was used for process isolation, socket shaping, and the OpenSSH-agent experiment. Each configuration was evaluated in three independent process-level runs. Every run performed 100 warm-up iterations followed by 100 measured successful executions. The tables report the arithmetic mean of the three per-process means, corresponding to 300 measured executions per entry. The raw rows additionally retain the minimum, $p50$, $p95$, maximum, and sample standard deviation. Arithmetic means are used here because the original PAS-TA-U tables report averages; consequently, these values are kept separate from the $p50$ series in the main comparison and are not pooled with Figures 6–14.

Setup, key generation, and cached Shoup-RSA parameter generation are outside the timed regions. Before timing each configuration, the runners check that exactly t_{sp} TTG shares produce a valid token, fewer than t_{sp} shares do not, a changed message invalidates the token, the correct password succeeds, a wrong password

fails, password update succeeds, the old password fails after the update, and the new password succeeds. Two invalid diagnostic runs—one initial SSH wall-clock attempt and one WAN run for which `tc` was not active—are preserved with the raw artifacts but excluded from aggregation and rerun under the intended settings.

The first experiment isolates TTG partial evaluation at one PAS-TA-U server, client-side combination of exactly t_{sp} partial tokens, and verification of one final token. TOPRF work, symmetric encryption, database access, networking, and setup are excluded. The second experiment uses one real TCP process per PAS-TA-U server, persistent length-prefixed connections, and parallel requests to exactly t_{sp} PAS-TA-U servers in each threshold round. Linux `tc netem` applies a 2 ms one-way delay for the LAN4 profile and a 40 ms one-way delay for the WAN80 profile, yielding approximately 4 ms and 80 ms RTT, respectively. These are client-observed wall-clock measurements over TCP, not RTT values added to local computation; they include serialization, socket I/O, protocol randomness, server scheduling, and the implemented acknowledgements. They do not include TLS. Token generation covers Request, parallel Respond, and Finalize, but excludes final application-server verification. Password update includes the old- and new-password TOPRF rounds, construction of all server-specific update ciphertexts, threshold generation and verification of the update token, delivery to all n_{sp} PAS-TA-U servers, record replacement, and acknowledgements. State reset is performed after, and outside, each timed update sample.

The third experiment validates that the paper-style Shoup-RSA result can be consumed by an ordinary OpenSSH RSA verification path. A minimal Unix-domain OpenSSH-agent interface advertises the generated RSA public key, accepts `rsa-sha2-256` requests, and threshold signs the exact SSH data. We separately measure the agent-side PAS-TA-U signing operation and the complete wall-clock latency of a fresh `ssh ... true` command against an isolated loopback `sshd`. This is a native OpenSSH-agent validation, not an exact reproduction of the authors' patched-`libcrypto` path. The published reference values were measured in Python on Ubuntu 20.04 using OpenSSL/OpenSSH, real protected sockets, a 2-core 2.2 GHz Intel i5-5200U, 8 GB RAM, and the same nominal 4/80 ms RTT profiles [56].

Table A.1: Isolated TTG costs for the two Rust PAS-TA-U profiles. Values are arithmetic means in milliseconds over three runs of 100 measured executions. The published PAS-TA-U evaluation reports complete token generation rather than isolated TTG operations and therefore has no row in this table.

(n_{sp}, t_{sp})	Implementation	TTG profile	PartEval/server	Combine/client	Verify
(3, 2)	Modern Rust	BLS12-381	0.461	0.474	2.184
(3, 2)	Paper-style Rust	Shoup RSA-2048	2.484	1.163	0.089
(6, 3)	Modern Rust	BLS12-381	0.441	0.699	2.159
(6, 3)	Paper-style Rust	Shoup RSA-2048	2.538	1.285	0.091
(10, 5)	Modern Rust	BLS12-381	0.461	1.195	2.210
(10, 5)	Paper-style Rust	Shoup RSA-2048	2.494	1.915	0.094
(10, 10)	Modern Rust	BLS12-381	0.443	2.411	2.172
(10, 10)	Paper-style Rust	Shoup RSA-2048	2.494	3.498	0.090

They are therefore treated as cross-implementation references.

A.3 Results

A.3.1 Isolated TTG Cost

Table A.1 exposes the expected difference between the two token profiles. One Shoup-RSA server-side partial evaluation remains nearly constant at 2.48–2.54 ms and is about 5.4–5.8 times the corresponding modern-profile cost. Combination grows with t_{sp} in both implementations: from 0.474 to 2.411 ms for the modern profile and from 1.163 to 3.498 ms for the paper-style profile. The ordering reverses for verification. Shoup RSA uses the small public exponent and verifies in approximately 0.09 ms, whereas the BLS12-381 profile performs pairing-based verification in approximately 2.16–2.21 ms. Thus, the primitive choice changes where the TTG cost is paid even though the surrounding PAS-TA-U flow is unchanged.

Table A.2: Complete token-generation latency in milliseconds. The two Rust columns are client-observed real-TCP measurements under Linux `tc`; the published column is the arithmetic mean reported for the original Python/OpenSSH implementation. Final application-server verification is excluded.

Network	(n_{sp}, t_{sp})	Modern Rust	Paper-style Rust	Published PAS-TA-U
LAN4	(3, 2)	5.948	19.559	22.8
LAN4	(6, 3)	6.451	19.698	25.1
LAN4	(10, 5)	7.157	23.322	28.2
LAN4	(10, 10)	9.696	38.075	32.5
WAN80	(3, 2)	86.277	100.490	109.5
WAN80	(6, 3)	86.025	101.755	112.4
WAN80	(10, 5)	85.141	104.152	115.8
WAN80	(10, 10)	90.192	115.375	119.9

A.3.2 Token Generation over Real TCP

Table A.2 shows that the cryptographic profile is clearly visible on LAN. The modern implementation requires 5.948–9.696 ms, whereas the paper-style profile requires 19.559–38.075 ms; the latter is 3.0–3.9 times the modern value under the common Rust and TCP environment. The paper-style values are 14–22% below the published value at the first three LAN configurations and 17% above it at (10, 10). Over WAN, the difference between the two Rust profiles contracts to a factor of 1.16–1.28 because one network exchange dominates local computation. The paper-style WAN values are within approximately 4–10% below the published values. This agreement in scale and threshold trend supports the paper-style implementation as a useful validation profile, while the remaining gaps are consistent with Rust versus Python, persistent plain-TCP versus per-request protected connections, different libraries and hardware, and the absence of the original OpenSSH integration from this experiment.

Table A.3: Complete password-update latency in milliseconds. Rust values include old- and new-password TOPRF evaluation, server-specific update construction, update-token generation and verification, delivery to all PAS-TA-U servers, record replacement, and acknowledgements.

Network	(n_{sp}, t_{sp})	Modern Rust	Paper-style Rust	Published PAS-TA-U
LAN4	(3, 2)	25.532	54.780	65.3
LAN4	(6, 3)	26.304	55.458	74.4
LAN4	(10, 5)	29.056	66.370	82.7
LAN4	(10, 10)	31.743	99.079	95.1
WAN80	(3, 2)	339.986	377.231	325.5
WAN80	(6, 3)	339.165	378.775	333.1
WAN80	(10, 5)	341.851	388.589	346.9
WAN80	(10, 10)	345.343	416.835	357.6

A.3.3 Password Update over Real TCP

The longer update path in Table A.3 exposes both n_{sp} -dependent payload construction and t_{sp} -dependent threshold work. On LAN, the modern profile is 2.1–3.1 times lower than the paper-style profile. The paper-style values are 16–25% below the published values for the first three configurations and 4% above the published (10, 10) point. In WAN, the multiple threshold exchanges and final fan-out dominate: the modern and paper-style ranges become 339.165–345.343 ms and 377.231–416.835 ms, respectively. The modern advantage consequently contracts to approximately 10–17%. The paper-style WAN values are 12–17% above the published measurements. These differences are small enough to preserve the same round- and threshold-driven behavior, but they are not used to claim reproduction: the Rust runner sends the final update to all n_{sp} PAS-TA-U servers in parallel and waits for all acknowledgements over persistent plain TCP, whereas the original Python implementation uses a different protected-connection schedule, language, application integration, and host.

Table A.4: OpenSSH-agent validation of the paper-style Rust profile, in milliseconds. “Sign service” measures the local agent-side PAS-TA-U threshold signing path; “SSH command” measures a complete fresh `ssh ... true` invocation. The published token-generation value is a reference for the signing phase and is not directly comparable with the complete SSH-command column.

Network	(n_{sp}, t_{sp})	Sign service	SSH command	Published token generation
LAN4	(3, 2)	20.027	283.133	22.8
LAN4	(6, 3)	25.834	295.059	25.1
LAN4	(10, 5)	39.992	316.739	28.2
LAN4	(10, 10)	75.808	352.161	32.5
WAN80	(3, 2)	22.283	1281.183	109.5
WAN80	(6, 3)	27.493	1283.739	112.4
WAN80	(10, 5)	41.661	1311.231	115.8
WAN80	(10, 10)	74.047	1370.666	119.9

A.3.4 OpenSSH Validation

Table A.4 confirms that the Shoup-RSA token produced by the paper-style profile is accepted through the standard OpenSSH RSA verification path. The agent-side signing time is nearly independent of the LAN/WAN profile because the agent and its threshold-signing fixture communicate locally through a Unix-domain interface; the `tc` delay applies to the SSH client-server connection. Its LAN value is close to the published token-generation value for (3, 2) and (6, 3), but grows more quickly at the two larger thresholds. The complete SSH command is substantially larger—283–352 ms on LAN and 1.28–1.37 s on WAN—because it includes process startup and the complete OpenSSH negotiation rather than only PAS-TA-U token generation. It is therefore reported as an integration check, not as an alternative estimate of the published token-generation table.

A.4 Relative Performance

The isolated measurements show that Shoup RSA moves cost toward server-side partial signing and client-side combination, while its verification remains substantially cheaper than pairing-based BLS verification. A Shoup-RSA partial evaluation costs approximately 2.48–2.54 ms, compared with 0.44–0.46 ms for the modern BLS profile, and is therefore about $5.4\text{--}5.8\times$ more expensive. Combination grows with t_{sp} in both profiles, but the paper-style cost remains larger throughout the evaluated range. The ordering reverses for verification: Shoup RSA requires only 0.089–0.094 ms, whereas BLS verification requires 2.159–2.210 ms because it performs pairing-based checks. The modern profile therefore does not dominate every isolated TTG operation.

Despite the verification result, the modern profile outperforms the paper-style profile in every complete-phase configuration. Across the four parameter settings, modern Rust is $3.05\text{--}3.93\times$ faster for token generation under LAN4 and $1.16\text{--}1.28\times$ faster under WAN80. For password update, the corresponding improvements are $2.11\text{--}3.12\times$ and $1.11\text{--}1.21\times$, respectively. The smaller WAN factors are expected: once one or more full network exchanges are included, RTT dominates the local difference between BLS and Shoup RSA.

Compared with the published implementation, the modern profile reduces token-generation latency by 70.2–74.6% under LAN4 and 21.2–26.5% under WAN80. It also reduces LAN4 password-update latency by 60.9–66.6%. The published and modern WAN80 password-update values are instead comparable: the modern profile is 4.5% slower at $(n_{sp}, t_{sp}) = (3, 2)$ and 1.8% slower at $(6, 3)$, but 1.5% faster at $(10, 5)$ and 3.4% faster at $(10, 10)$. These small WAN differences are within the range expected from the distinct language, hardware, socket/TLS treatment, and application integration and should not be interpreted as a protocol-level advantage.

The complete-phase advantage of the modern implementation therefore results from the combined TOPRF, TTG, encryption, serialization, and message-processing profile, rather than from every individual primitive. This distinction is important for the main comparison: using the original paper’s absolute values would penalize

PAS-TA-U for an older Python/OpenSSL/OpenSSH environment, while using the modern Rust implementation places PAS-TA-U, UpSPA, and AugSSO under one controlled platform. The paper-style profile and published values are retained here to expose this implementation sensitivity and to verify that the modern baseline preserves the PAS-TA-U phase structure and expected threshold scaling.

A.5 Fairness of the Main Comparison and Limitations

The experiments support the use of the modern Rust PAS-TA-U implementation as the fair baseline in the main UpSPA–PAS-TA-U–AugSSO comparison. It executes the published PAS-TA-U phase structure and is evaluated with the same host, implementation language, benchmark harness, parameter sets, and measurement discipline as the other two schemes. The paper-style Rust profile changes only the concrete cryptographic profile toward the original PAS-TA-U choices and reproduces the expected threshold-dependent behavior. Its complete-phase results are also of the same order as the published values, particularly for WAN token generation. These observations show that the main PAS-TA-U baseline is not a partial or artificially simplified implementation; its lower latency is principally explained by the modern cryptographic and software profile.

The appendix does not establish an exact reproduction of the published implementation. The remaining differences include the language and library stack, exact 2HashTDH group configuration, hardware and operating system, persistent plain-TCP transport in the common-network experiment, the absence of TLS there, and an OpenSSH-agent interface rather than the published patched-`libcrypto` path. Ratios between the two Rust profiles therefore characterize cryptographic-profile and implementation effects under a common environment. Comparisons with the published column remain cross-implementation validation only; no protocol-level speedup is claimed. The original paper’s non-threshold plain-SSH baseline is omitted because neither Rust profile implements that baseline.

Appendix B

LATENCY DIAGNOSTICS

This appendix retains only the WAN plots that directly compare UpSPA and TSPA. The client and LAN results are reported in the implementation chapter; the WAN views are supplementary because the fixed network delay dominates their absolute values. They remain useful for confirming that the same relative behavior persists under the common WAN model. All plotted values are medians from the benchmark configuration described in the implementation chapter.

Fixed provider count. At $n_{sp} = 10$, UpSPA remains about one modeled WAN round above TSPA in registration and authentication. Threshold variation changes local work but not this stage-dominated ordering.

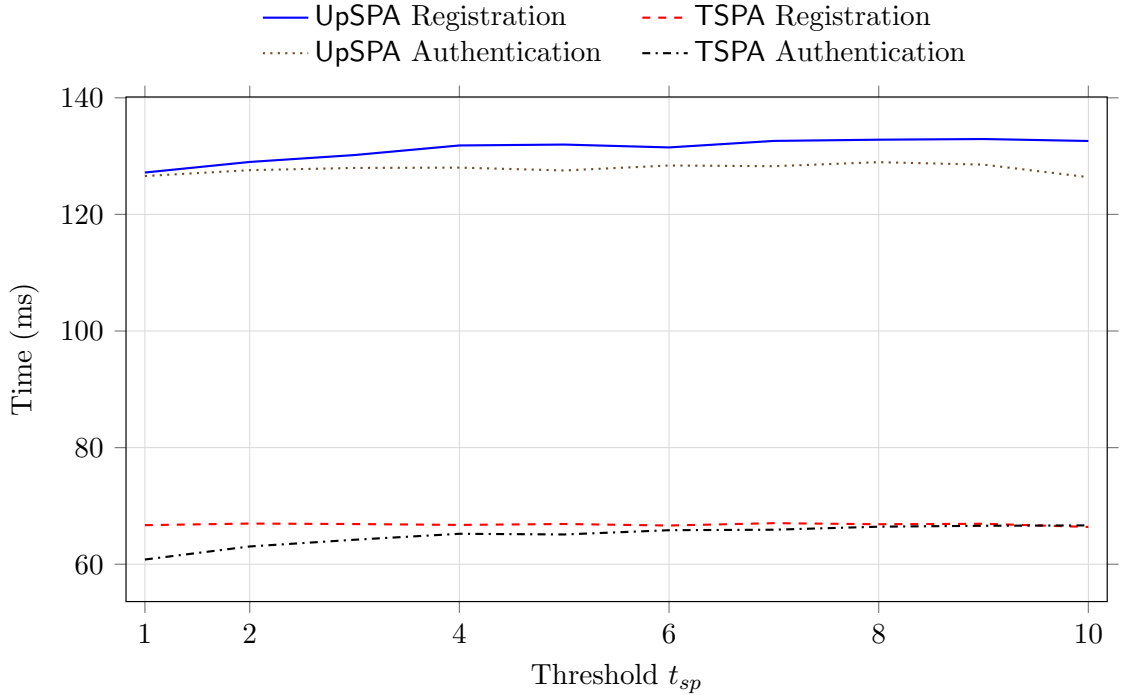


Figure B.1: WAN registration and authentication, $n_{sp} = 10$.

Proportional scaling. With $t_{sp} = 0.6n_{sp}$, both schemes grow moderately from $n_{sp} = 10$ to 100, while the approximately one-RTT separation persists. The comparison therefore confirms network-stage sensitivity rather than a new protocol ranking.

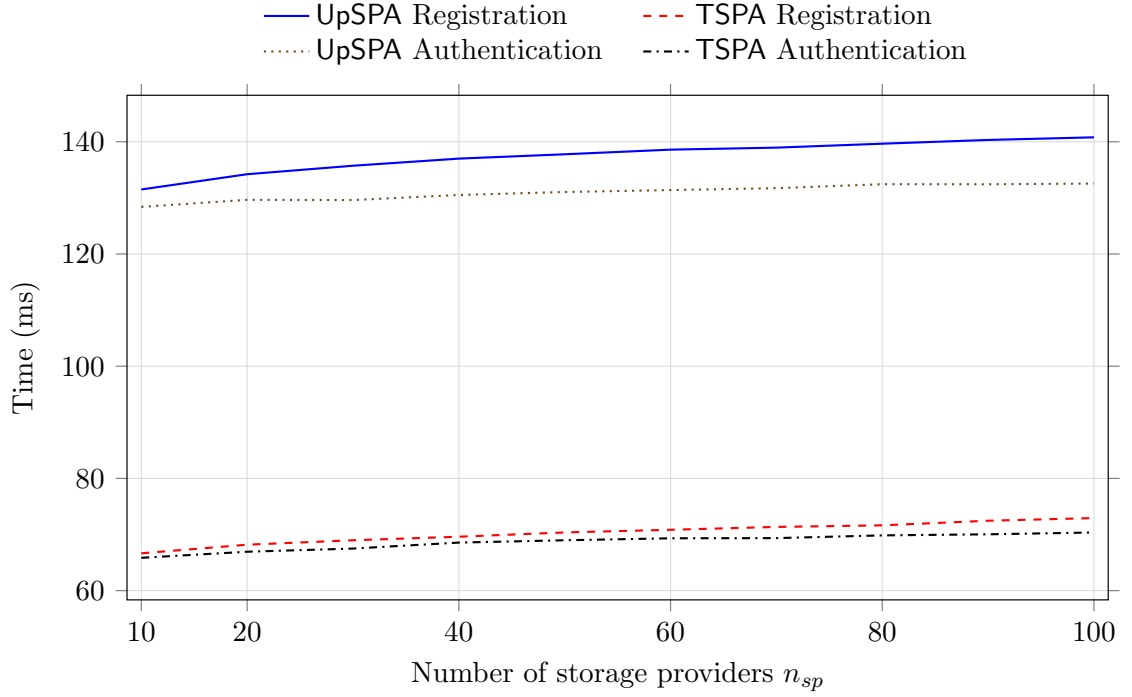


Figure B.2: WAN registration and authentication, $t_{sp} = 0.6n_{sp}$.

BIBLIOGRAPHY

- [1] Tolga Acar, Mira Belenkiy, and Alptekin Küpçü. Single password authentication. *Computer Networks*, 57(13):2597–2614, 2013.
- [2] Shashank Agrawal, Peihan Miao, Payman Mohassel, and Pratyay Mukherjee. Pasta: password-based threshold authentication. *ACM CCS*, 2018.
- [3] Hagit Attiya, Amotz Bar-Noy, and Danny Dolev. Sharing memory robustly in message-passing systems. *Journal of the ACM*, 42(1):124–142, January 1995.
- [4] Ali Bagherzandi, Stanislaw Jarecki, Nitesh Saxena, and Yanbin Lu. Password-protected secret sharing. In *ACM CCS*, 2011.
- [5] Carsten Baum, Tore Frederiksen, Julia Hesse, Anja Lehmann, and Avishay Yanai. Pesto: proactively secure distributed single sign-on, or how to trust a hacked server. In *EuroS&P*, 2020.
- [6] Donald Beaver. Foundations of secure interactive computing. In *Advances in Cryptology – CRYPTO ’91*, volume 576 of *Lecture Notes in Computer Science*, pages 377–391. Springer, 1992.
- [7] Mihir Bellare and Chanathip Namprempre. Authenticated encryption: Relations among notions and analysis of the generic composition paradigm. *Journal of Cryptology*, 21(4):469–491, 2008.
- [8] Mihir Bellare, David Pointcheval, and Phillip Rogaway. Authenticated key exchange secure against dictionary attacks. In *Advances in Cryptology – EUROCRYPT 2000*, volume 1807 of *Lecture Notes in Computer Science*, pages 139–155. Springer, 2000.
- [9] Mihir Bellare and Phillip Rogaway. The security of triple encryption and a framework for code-based game-playing proofs. In *Advances in Cryptology –*

- EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 409–426. Springer, 2006.
- [10] Steven M. Bellovin and Michael Merritt. Encrypted key exchange: Password-based protocols secure against dictionary attacks. In *Proceedings of the 1992 IEEE Computer Society Symposium on Research in Security and Privacy*, pages 72–84. IEEE Computer Society, 1992.
- [11] Kemal Bicakci, Nart Bedin Atalay, Mustafa Yuceel, and Paul C. van Oorschot. Exploration and field study of a browser-based password manager using icon-based passwords. In *Financial Cryptography and Data Security*, volume 7126 of *Lecture Notes in Computer Science*, pages 104–118. Springer, 2012.
- [12] Joseph Bonneau, Cormac Herley, Paul C. van Oorschot, and Frank Stajano. The quest to replace passwords: A framework for comparative evaluation of web authentication schemes. In *2012 IEEE Symposium on Security and Privacy*, pages 553–567. IEEE, 2012.
- [13] Xavier Boyen. Hidden credential retrieval from a reusable password. In *Proceedings of the 4th International Symposium on Information, Computer, and Communications Security*, pages 228–238. ACM, 2009.
- [14] Julian Brost, Christoph Egger, Russell W. F. Lai, Fritz Schmid, Dominique Schröder, and Markus Zoppelt. Threshold password-hardened encryption services. In *ACM CCS*, 2020.
- [15] Jan Camenisch, Anja Lehmann, Anna Lysyanskaya, and Gregory Neven. Memento: How to reconstruct your secrets from a single password in a hostile environment. *CRYPTO*, 2014.
- [16] Chenchen Cao, Chunxiang Xu, Changsong Jiang, Zhao Zhang, Xinfeng Dong, and Kefei Chen. LPbT-SSO: Password-based threshold single-sign-on authentication from LWE. *IEEE Transactions on Dependable and Secure Computing*, 22(1):491–504, 2025.

- [17] Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI 1999)*, pages 173–186, New Orleans, LA, USA, February 1999. USENIX Association.
- [18] Debajyoti Das, Julian Hesse, and Anja Lehmann. Dpase: Distributed password-hardened encryption. In *ACM SIGSAC*, 2022.
- [19] Shimon Even, Oded Goldreich, and Abraham Lempel. A randomized protocol for signing contracts. *Communications of the ACM*, 28(6):637–647, 1985.
- [20] Adam Everspaugh, Rahul Chatterjee, Samuel Scott, Ari Juels, and Thomas Ristenpart. The pythia PRF service. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 547–562. USENIX Association, 2015.
- [21] Dinei Florencio and Cormac Herley. A large-scale study of web password habits. In *Proceedings of the 16th International Conference on World Wide Web*, pages 657–666. ACM, 2007.
- [22] Michael J. Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold. Keyword search and oblivious pseudorandom functions. In *Theory of Cryptography Conference*, volume 3378 of *Lecture Notes in Computer Science*, pages 303–324. Springer, 2005.
- [23] Juan Garay, Aggelos Kiayias, and Nikos Leonardos. The bitcoin backbone protocol: Analysis and applications. In *Advances in Cryptology – EUROCRYPT 2015*, volume 9057 of *Lecture Notes in Computer Science*, pages 281–310. Springer, 2015.
- [24] Rosario Gennaro, Stanisław Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. *Journal of Cryptology*, 20(1):51–83, 2007.
- [25] David K. Gifford. Weighted voting for replicated data. In *Proceedings of the 7th ACM Symposium on Operating Systems Principles*, pages 150–162. ACM, 1979.

- [26] Oded Goldreich. *Foundations of Cryptography: Volume 1, Basic Tools*. Cambridge University Press, 2001.
- [27] Oded Goldreich. *Foundations of Cryptography: Volume 2, Basic Applications*. Cambridge University Press, 2004.
- [28] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, 1986.
- [29] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game. In *Proceedings of the 19th Annual ACM Symposium on Theory of Computing*, pages 218–229. ACM, 1987.
- [30] Shafi Goldwasser and Silvio Micali. Probabilistic encryption. *Journal of Computer and System Sciences*, 28(2):270–299, 1984.
- [31] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing*, 17(2):281–308, 1988.
- [32] James Heather and David Lundin. The append-only web bulletin board. In Pierpaolo Degano, Joshua D. Guttman, and Fabio Martinelli, editors, *Formal Aspects in Security and Trust: 5th International Workshop, FAST 2008, Revised Selected Papers*, volume 5491 of *Lecture Notes in Computer Science*, pages 242–256, Berlin, Heidelberg, 2009. Springer.
- [33] Andrea Huszti, Szabolcs Kovács, and Norbert Oláh. Scalable, password-based and threshold authentication for smart homes. *International Journal of Information Security*, 21(4):707–723, 2022.
- [34] Devriş İşler, HamidReza Saadi Dadmarzi, and Alptekin Küpçü. Secure and updatable single password authentication. Cryptology ePrint Archive, Paper 2026/784, 2026.
- [35] Devriş İşler and Alptekin Küpçü. Threshold single password authentication. In *Data Privacy Management, Cryptocurrencies and Blockchain Technology*,

- volume 10436 of *Lecture Notes in Computer Science*, pages 143–162. Springer, 2017.
- [36] Devriş İşler and Alptekin Küpçü. Distributed single password protocol framework. Cryptology ePrint Archive, Paper 2018/976, 2018.
- [37] Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. Round-optimal password-protected secret sharing and t-pake in the password-only model. In *Advances in Cryptology–ASIACRYPT 2014*, 2014.
- [38] Stanisław Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. Toppss: Cost-minimal password-protected secret sharing based on threshold oprf. In *Applied Cryptography and Network Security*, volume 10355 of *Lecture Notes in Computer Science*, pages 39–58. Springer, 2017.
- [39] Stanislaw Jarecki, Hugo Krawczyk, and Jason Resch. Updatable oblivious key management for storage systems. In *ACM CCS*, 2019.
- [40] Stanislaw Jarecki, Hugo Krawczyk, Maliheh Shirvanian, and Nitesh Saxena. Device-enhanced password protocols with optimal online-offline protection. In *Proceedings of the 11th ACM on Asia Conference on Computer and Communications Security*, pages 177–188. ACM, 2016.
- [41] Changsong Jiang, Chunxiang Xu, Zhen Liu, Xinfeng Dong, and Wenzheng Zhang. Threshold password-hardening updatable oblivious key management. *T-IFS*, 2025.
- [42] Changsong Jiang, Chunxiang Xu, and Guomin Yang. Augsso: Secure threshold single-sign-on authentication with popular password collection. *IEEE Transactions on Mobile Computing*, 2025.
- [43] Changsong Jiang, Chunxiang Xu, Guomin Yang, Zhao Zhang, and Jie Chen. Device-enhanced password-based threshold single-sign-on authentication. *T-IFS*, 2025.

- [44] Jingwei Jiang, Ding Wang, Guoyin Zhang, and Zhiyuan Chen. Quantum-resistant password-based threshold single-sign-on authentication with updatable server private key. In *Computer Security – ESORICS 2022*, volume 13555 of *Lecture Notes in Computer Science*, pages 295–316. Springer, 2022.
- [45] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. CRC Press, 2014.
- [46] Aggelos et al. Kiayias. Ouroboros: A provably secure proof-of-stake blockchain protocol. In *CRYPTO*, 2017.
- [47] Russell W. F. Lai, Christoph Egger, Manuel Reinert, Sherman S. M. Chow, Matteo Maffei, and Dominique Schröder. Simple Password-Hardened encryption services. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 1405–1421. USENIX Association, 2018.
- [48] Ben Laurie, Emilia Kasper Messeri, and Rob Stradling. Certificate transparency version 2.0. RFC 9162, 2021.
- [49] Shanshan Li and Chunxiang Xu. Puokms: Password-protected updatable oblivious key management system for cloud storage. In *Information Security Practice and Experience: International Conference, ISPEC*, 2021.
- [50] Dahlia Malkhi and Michael K. Reiter. Byzantine quorum systems. *Distributed Computing*, 11(4):203–213, 1998.
- [51] Marcela S. Melara, Aaron Blankstein, Joseph Bonneau, Edward W. Felten, and Michael J. Freedman. Coniks: Bringing key transparency to end users. In *24th USENIX Security Symposium*, pages 383–398. USENIX Association, 2015.
- [52] Silvio Micali, Charles Rackoff, and Bob Sloan. The notion of security for probabilistic cryptosystems. *SIAM Journal on Computing*, 17(2):412–426, 1988.
- [53] Jelena Mirkovic and Peter L. Reiher. A taxonomy of DDoS attack and DDoS defense mechanisms. *ACM SIGCOMM Computer Communication Review*, 34(2):39–53, 2004.

- [54] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. <https://bitcoin.org/bitcoin.pdf>, 2008.
- [55] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Advances in Cryptology – CRYPTO ’91*, volume 576 of *Lecture Notes in Computer Science*, pages 129–140. Springer, 1992.
- [56] Rachit Rawat and Mahabir Prasad Jhanwar. Pas-ta-u: Password-based threshold authentication with password update. In *International Conference on Security, Privacy, and Applied Cryptography Engineering*, 2020.
- [57] Phillip Rogaway and Thomas Shrimpton. Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance. In *Fast Software Encryption*, volume 3017 of *Lecture Notes in Computer Science*, pages 371–388. Springer, 2004.
- [58] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
- [59] Andrew C. Yao. Theory and applications of trapdoor functions. In *Proceedings of the 23rd Annual Symposium on Foundations of Computer Science*, pages 80–91. IEEE, 1982.
- [60] Xun Yi, Zahir Tari, Feng Hao, Liqun Chen, Joseph K. Liu, Xuechao Yang, Kwok-Yan Lam, Ibrahim Khalil, and Albert Y. Zomaya. Efficient threshold password-authenticated secret sharing protocols for cloud computing. *Journal of Parallel and Distributed Computing*, 128:57–70, 2019.
- [61] Yuan Zhang, Chunxiang Xu, Hongwei Li, Kan Yang, Nan Cheng, and Xuemin Shen. Protect: efficient password-based threshold single-sign-on authentication for mobile users against perpetual leakage. *IEEE Transactions on Mobile Computing*, 2020.
- [62] Zhao Zhang, Chunxiang Xu, Changsong Jiang, and Kefei Chen. Tsapp: Threshold single-sign-on authentication preserving privacy. *IEEE Trans. Dependable Secur. Comput.*, 2024.